

Alexandre Meslin
meslin@puc-rio.br

Redes de Comunicação de Dados

Ementa

Conceitos de Redes de Comunicação de Dados

- PAN
- LAN
- MAN
- WAN

Topologias

- Física
- Lógica

Modelo de Referência OSI

- Funcionalidades das 7 camadas

Arquitetura TCP/IP

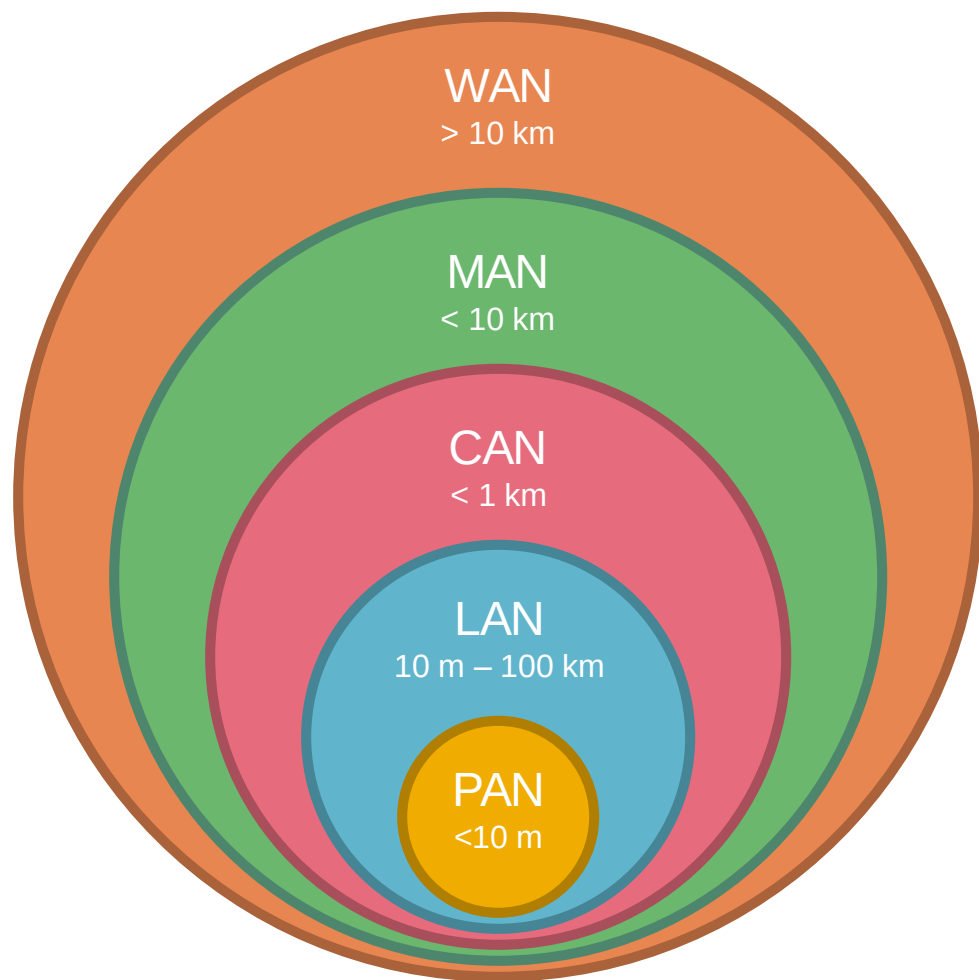
- Operação básica dos protocolos de rede (ARP, IP)
- Transporte (TCP, UDP)
- Aplicação (DHCP, HTTP, FTP, SMTP, POP3, IMAP)

Domain Name System (DNS)

Conceitos de Redes de Comunicação de Dados

- PAN
- LAN
- MAN
- WAN

Tipos de Redes

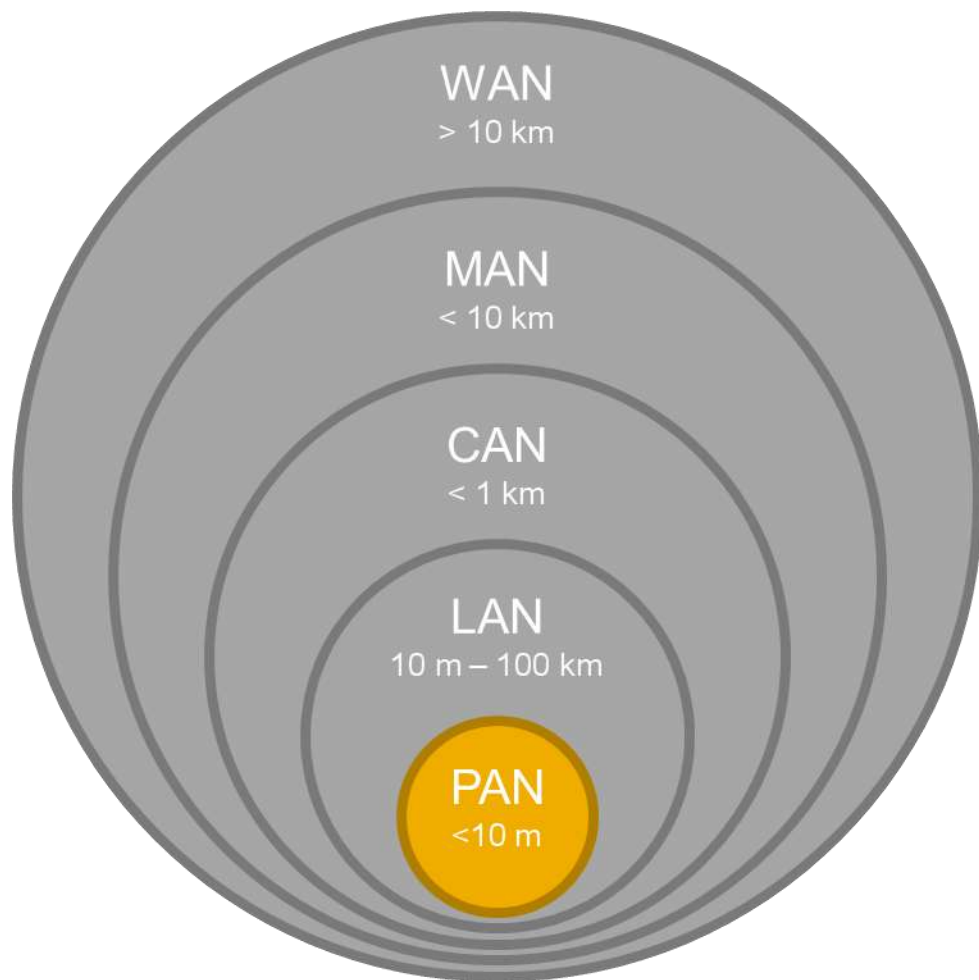


Escala de Alcance

Cada tipo de rede cobre uma faixa geográfica específica, desde conexões pessoais até redes globais

- **PAN** — até 10 m
- **LAN** — 10 m a 100 m
- **CAN** — até 1 km
- **MAN** — até 10 km
- **WAN** — acima de 10 km

Tipos de Redes



PAN: Personal Area Network

- Rede pessoal, de alcance muito pequeno, normalmente ao redor de uma pessoa
- Conecta periféricos próximos para troca de dados

Tipos de Redes

PAN: Personal Area Network

- Rede pessoal, de alcance muito pequeno, normalmente ao redor de uma pessoa
- Conecta periféricos próximos para troca de dados

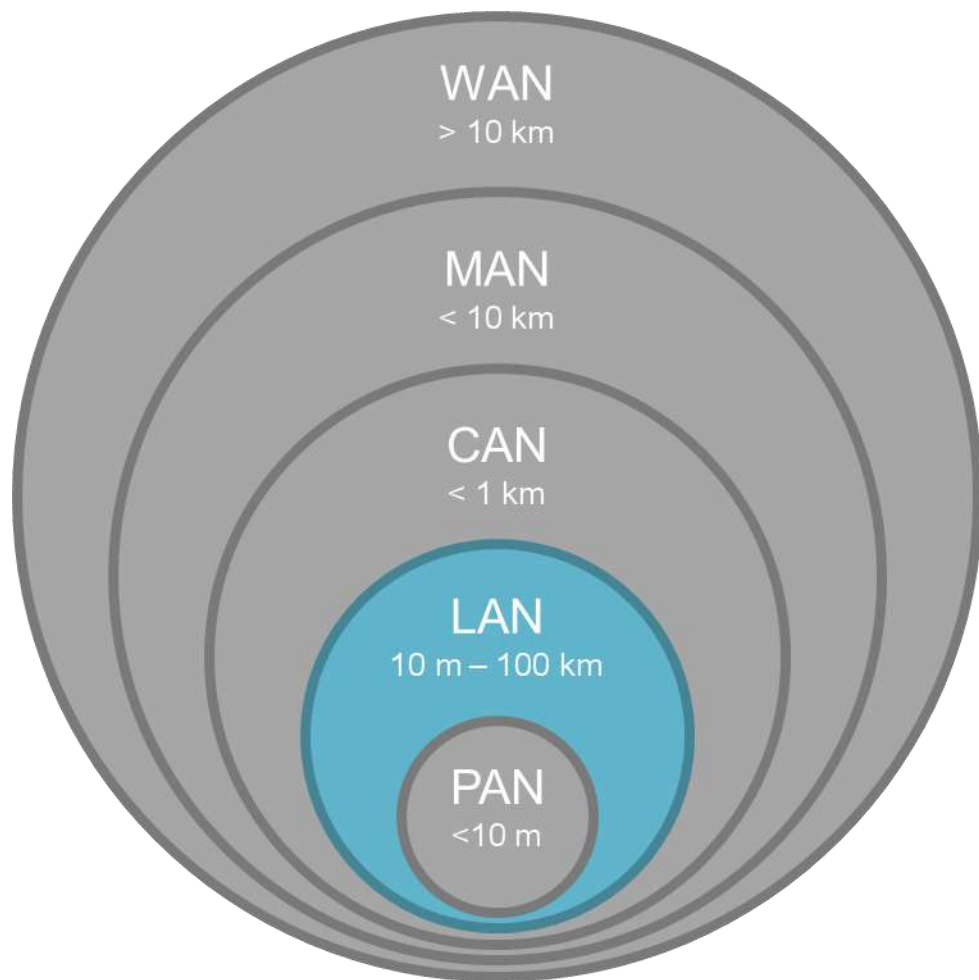


Bluetooth
Conexão sem fio de curto alcance

USB
Conexão cabeada

Wi-Fi Direct
Comunicação sem infraestrutura

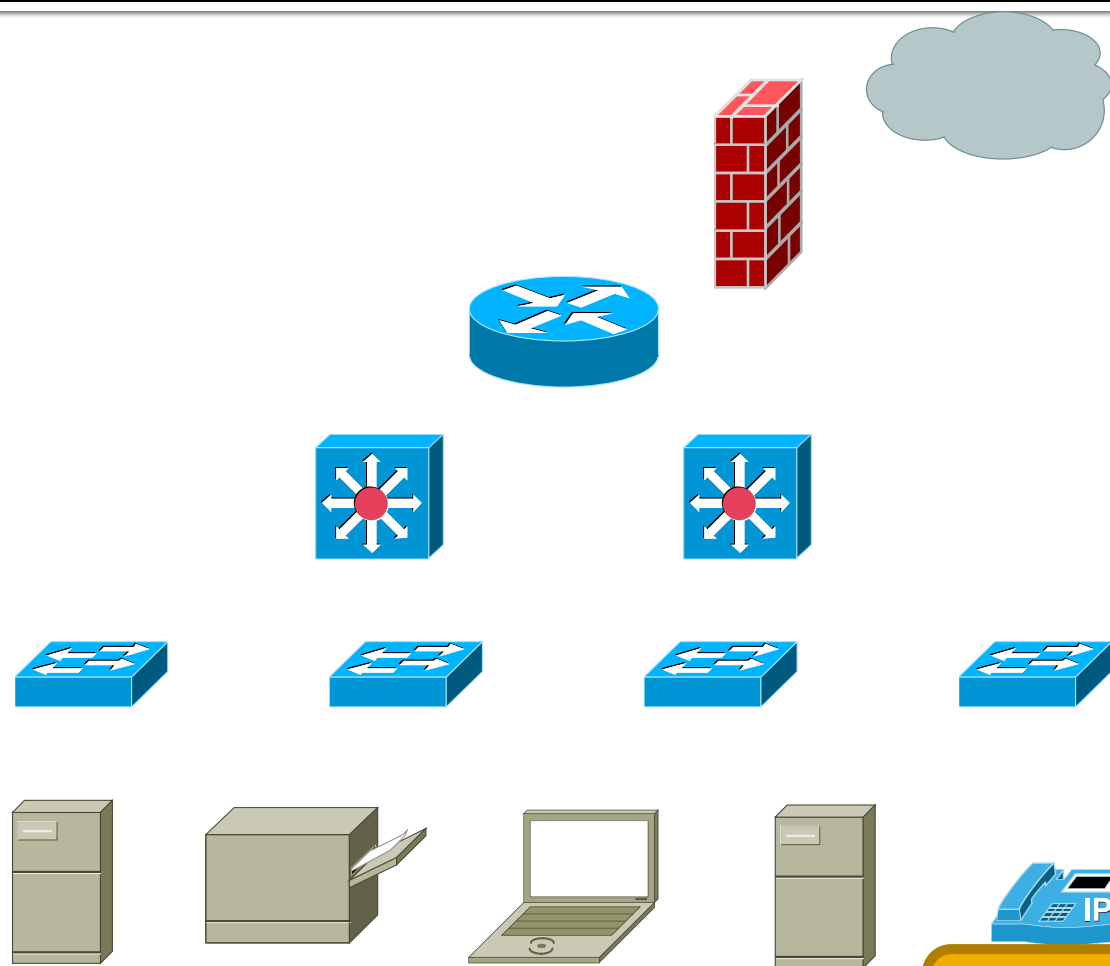
Tipos de Redes



LAN: Local Area Network

- Rede local limitada a uma residência, escritório, laboratório ou prédio
- Área relativamente pequena
- Alta velocidade
- Normalmente administrada por uma única organização ou pessoa

Tipos de Redes



LAN: Local Area Network

- Rede local limitada a uma residência, escritório, laboratório ou prédio
- Área relativamente pequena
- Alta velocidade
- Normalmente administrada por uma única organização ou pessoa

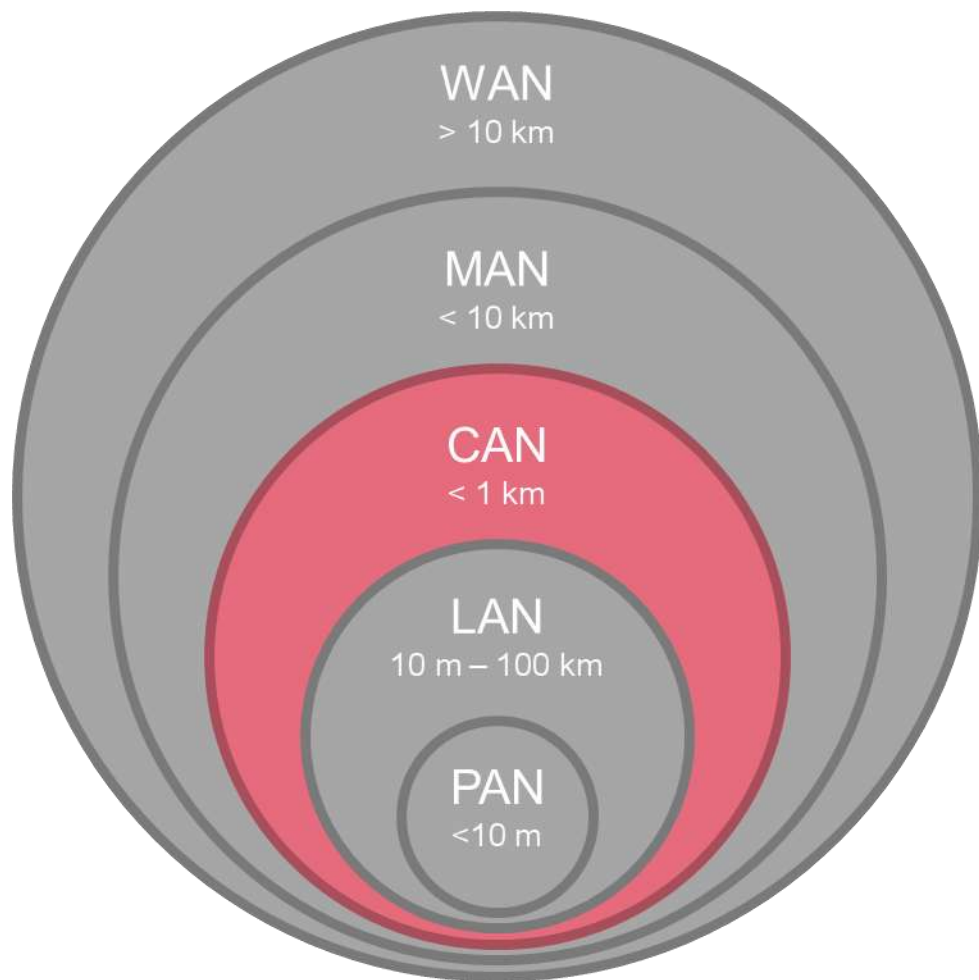
Ethernet

Velocidades entre 100 Mb/s a 10 Gb/s

Wi-Fi

Padrão IEEE 802.11

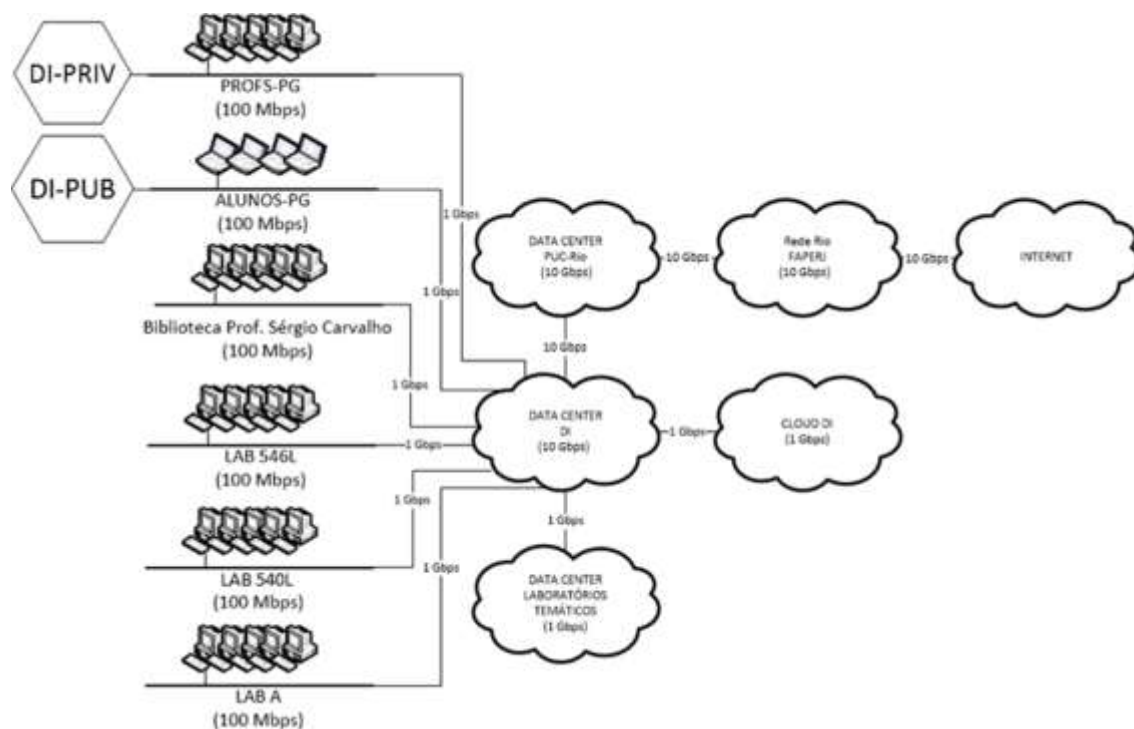
Tipos de Redes



CAN: Campus Area Network

- Conecta várias LANs dentro de uma área relativamente próxima, como:
 - Universidade
 - Campus empresarial
 - Complexo hospitalar
 - Parque industrial

Tipos de Redes

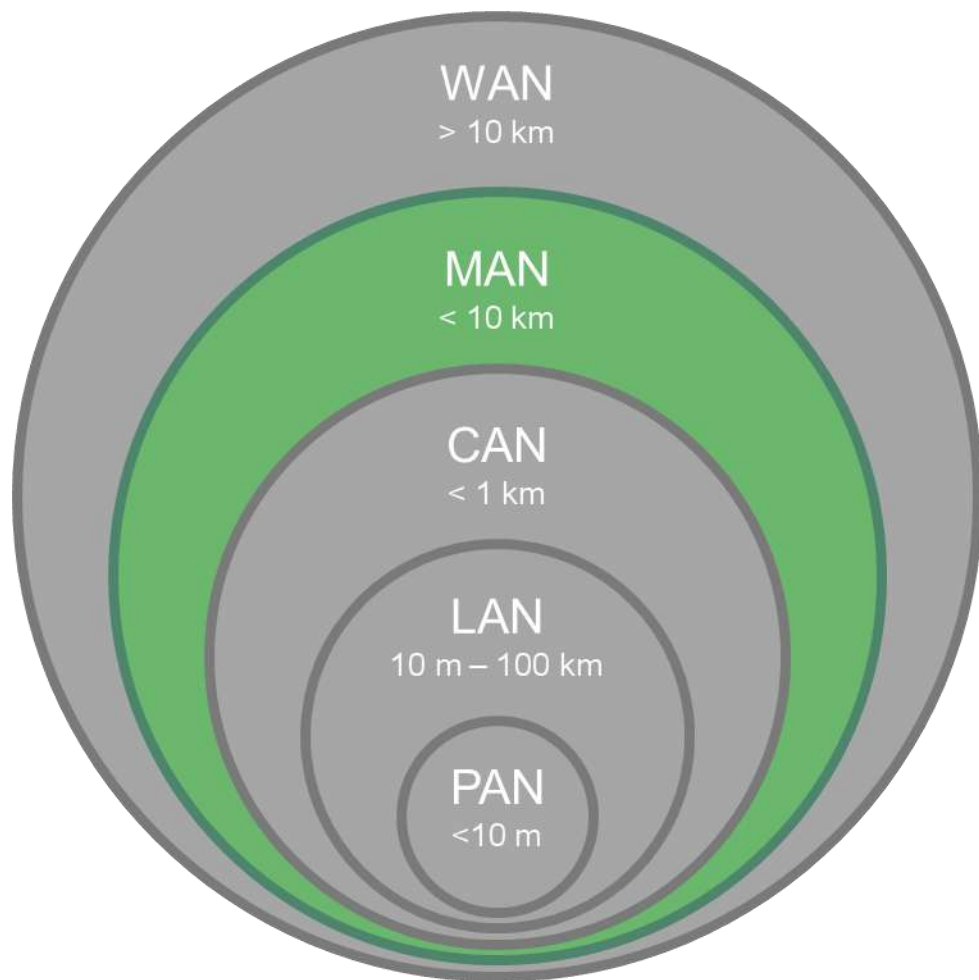


CAN: Campus Area Network

- Conecta várias LANs dentro de uma área relativamente próxima, como:
 - Universidade
 - Campus empresarial
 - Complexo hospitalar
 - Parque industrial.

Exemplo prático: A rede da **PUC-Rio** é um caso real de **CAN**, interligando prédios, laboratórios e instalações universitárias.

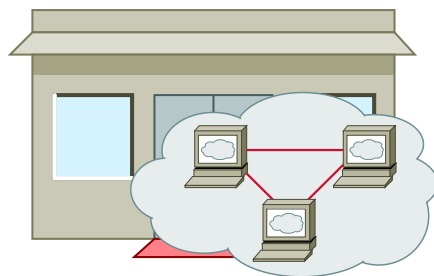
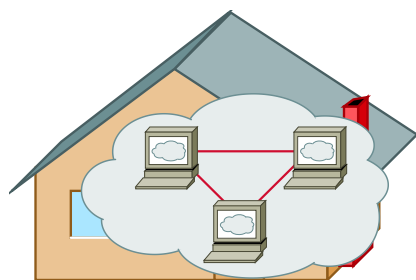
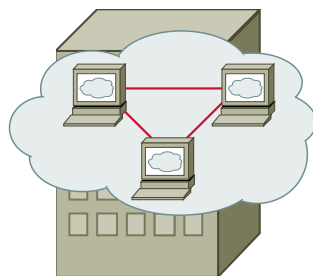
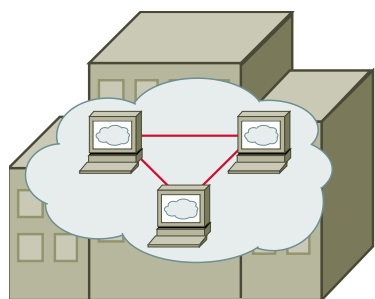
Tipos de Redes



MAN: Metropolitan Area Network

- Cobre uma área maior, normalmente uma cidade ou região metropolitana.
- Por exemplo, uma empresa pode ter escritórios em diferentes pontos de uma cidade:
- Utiliza infraestrutura de telecomunicações para conectar redes que estão geograficamente separadas dentro da mesma região.

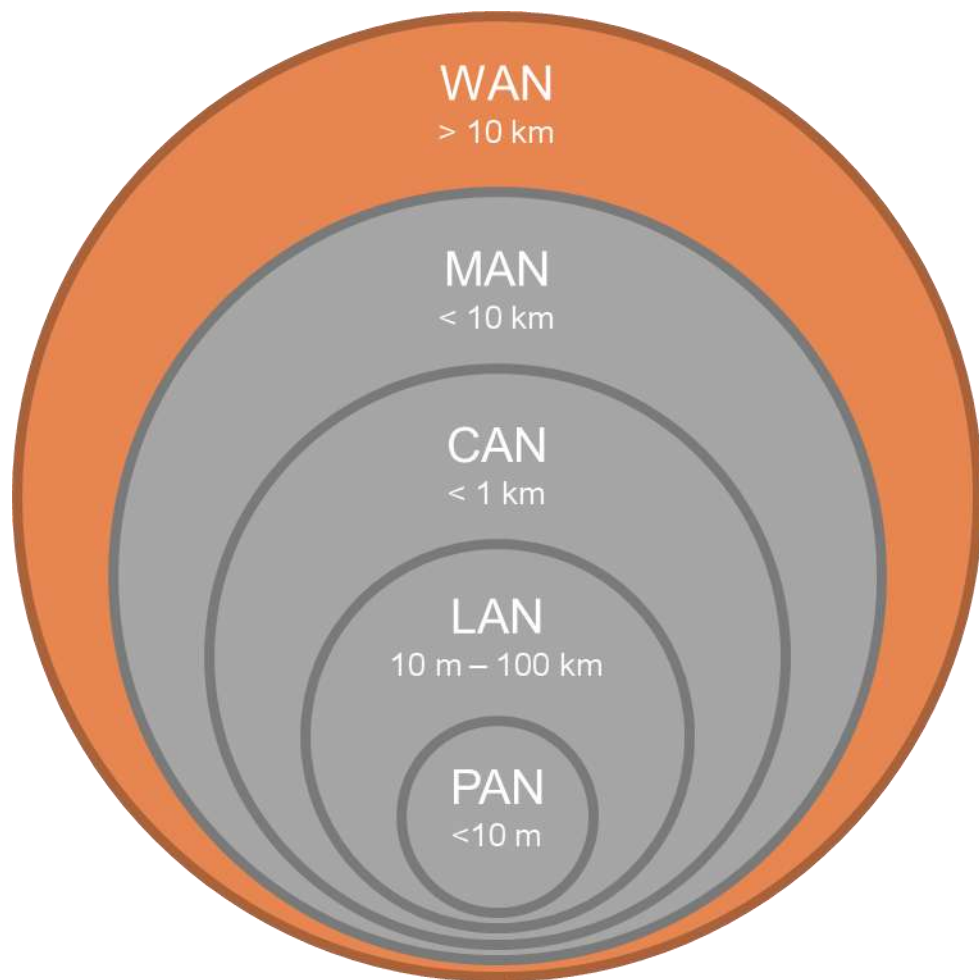
Tipos de Redes



MAN: Metropolitan Area Network

- Cobre uma área maior, normalmente uma cidade ou região metropolitana.
- Por exemplo, uma empresa pode ter escritórios em diferentes pontos de uma cidade:
- Utiliza infraestrutura de telecomunicações para conectar redes que estão geograficamente separadas dentro da mesma região.

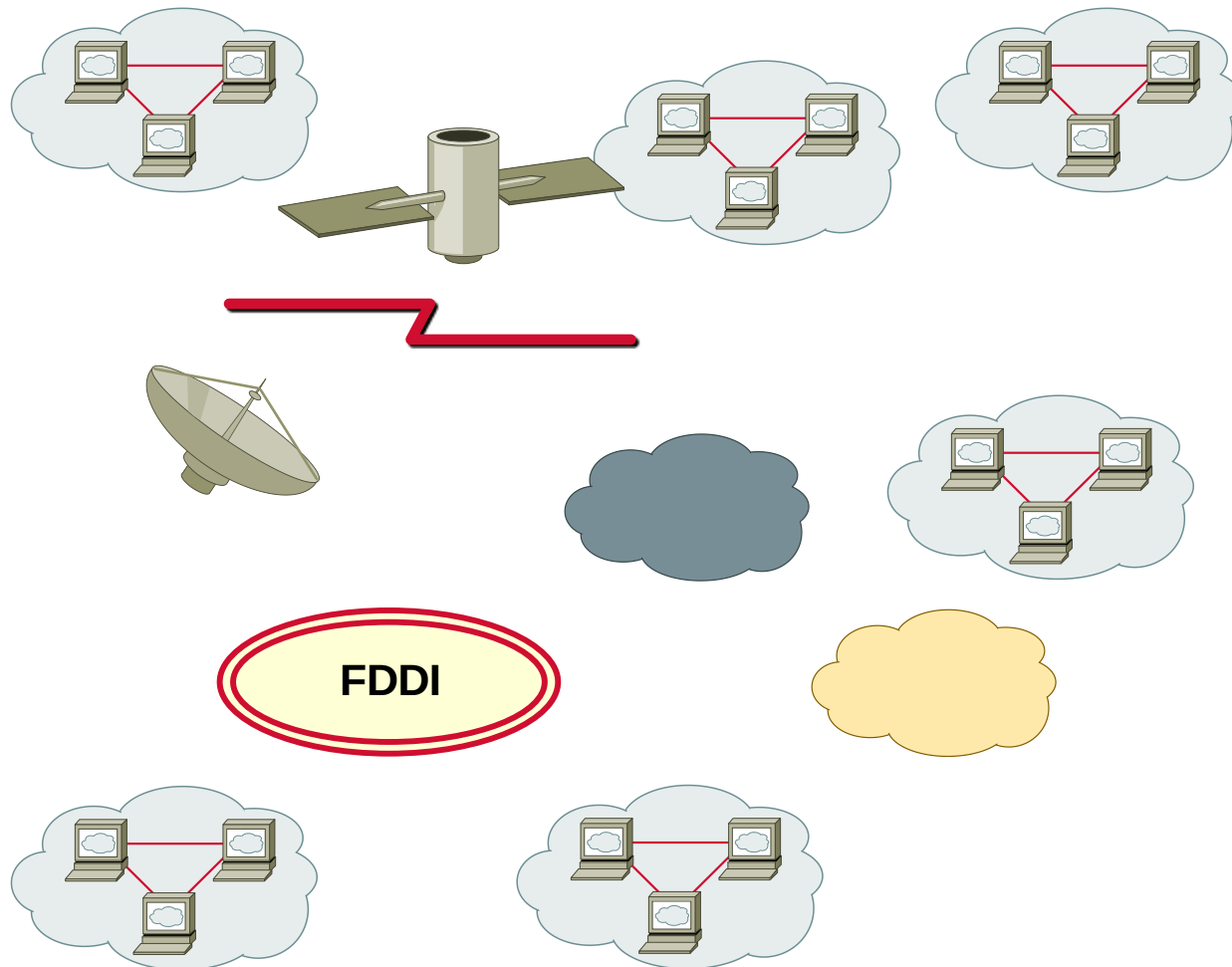
Tipos de Redes



WAN: Wide Area Network

- Rede de longa distância
- Interliga redes em grandes distâncias geográficas.
- Uma WAN pode conectar:
 - Cidades
 - Estados
 - Países
 - Continentes
- A Internet é um exemplo de uma WAN (tecnicamente é correto considerar uma enorme interconexão de diferentes redes)

Tipos de Redes



WAN: Wide Area Network

- Rede de longa distância
- Interliga redes em grandes distâncias geográficas.
- Uma WAN pode conectar:
 - Cidades
 - Estados
 - Países
 - Continentes
- A Internet é um exemplo de uma WAN (tecnicamente é correto considerar uma enorme interconexão de diferentes redes)

Topologías

- Física
- Lógica

Topologias de Redes

Topologia descreve como os elementos de uma rede estão organizados e relacionados

- Topologia física: como os dispositivos e cabos estão fisicamente conectados
- Topologia lógica: como os dados circulam pela rede, independentemente da disposição física

Uma mesma rede pode ter uma topologia **física** e uma **topologia** lógica diferentes

Topologia

Topologia Física

- Representa a estrutura física da rede
- Mostra a posição dos equipamentos e os enlaces que os conectam
- Exemplos: computadores, switches, roteadores, pontos de acesso e cabos

Topologia física: ajuda no planejamento de cabeamento e infraestrutura.

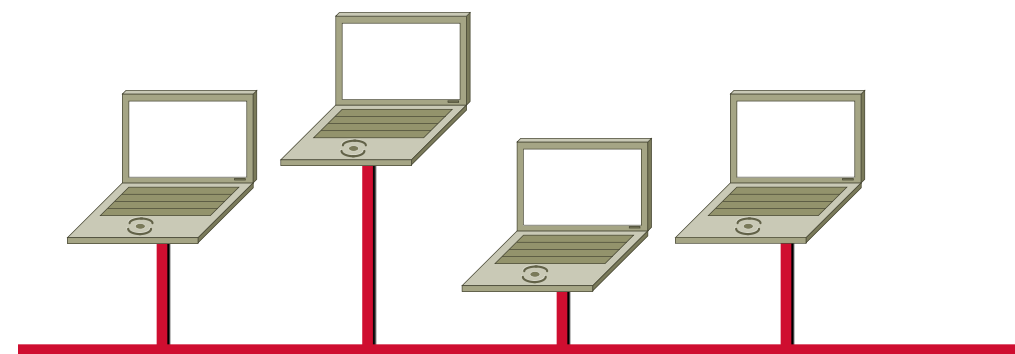
Topologia lógica: essencial para entender o comportamento do tráfego, configurar protocolos e segmentar a rede com VLANs.

Topologia Lógica

- Representa a forma como os dados percorrem a rede.
- Não precisa coincidir com a disposição física dos equipamentos.
- Pode depender de protocolos, VLANs, endereçamento e mecanismos de controle de acesso.
- É útil para entender o caminho e o comportamento do tráfego.

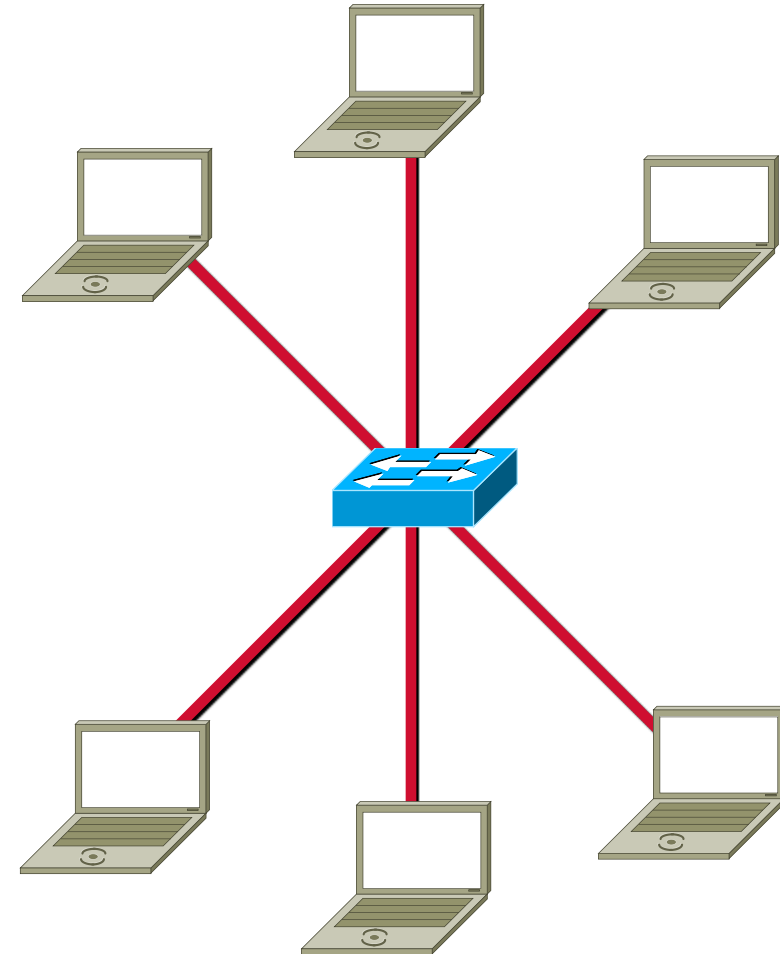
Topologia de Barramento

- Usa um único segmento de backbone (comprimento do cabo) ao qual todos os hosts se conectam diretamente
- Todos os dispositivos compartilham um meio de transmissão comum
- Foi muito utilizada em redes Ethernet antigas com cabo coaxial
- Vantagem: estrutura simples e pouco cabeamento
- Desvantagens
 - Falha no meio compartilhado pode afetar toda a rede
 - Desempenho limitado
- Topologia lógica predominante em redes locais



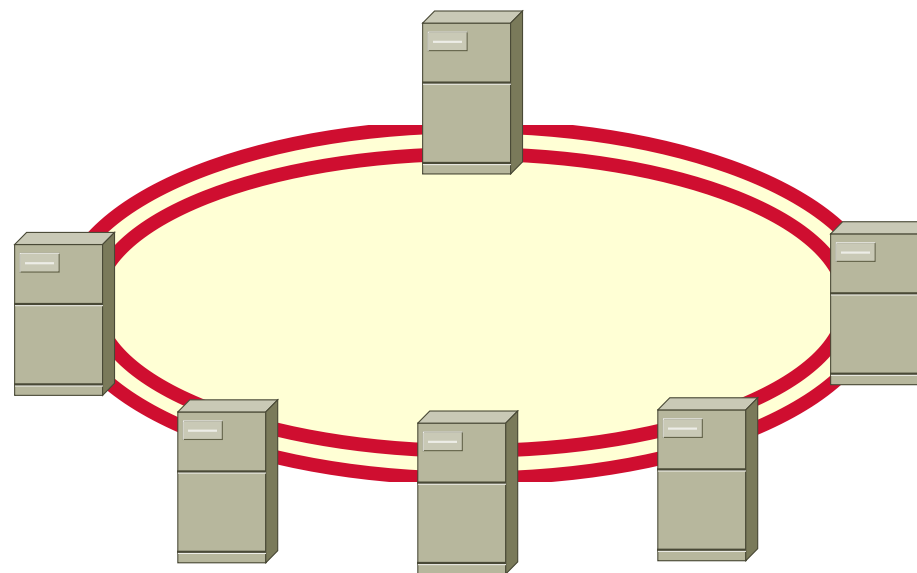
Topologia em Estrela

- Conecta todos os cabos ao ponto central de concentração
- Em Ethernet moderna, o equipamento central normalmente é um switch
- Falha de um cabo geralmente afeta apenas o dispositivo conectado a ele
- Facilita expansão, manutenção e gerenciamento
- É a topologia física predominante em LANs Ethernet atuais



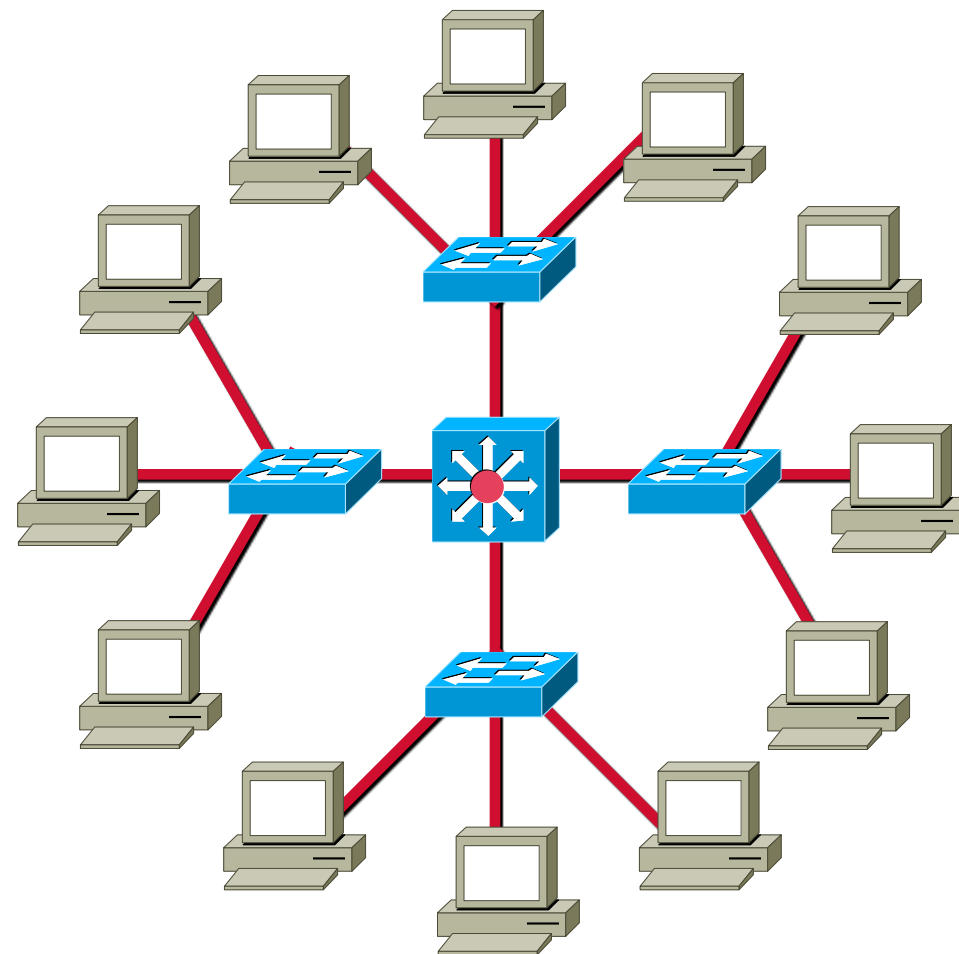
Topologia em Anel

- Os dispositivos formam um circuito fechado
- Cada nó possui conexão com os nós vizinhos
- O tráfego pode seguir um ou ambos os sentidos, conforme a tecnologia
- Exemplos históricos incluem Token Ring e algumas redes de fibra.



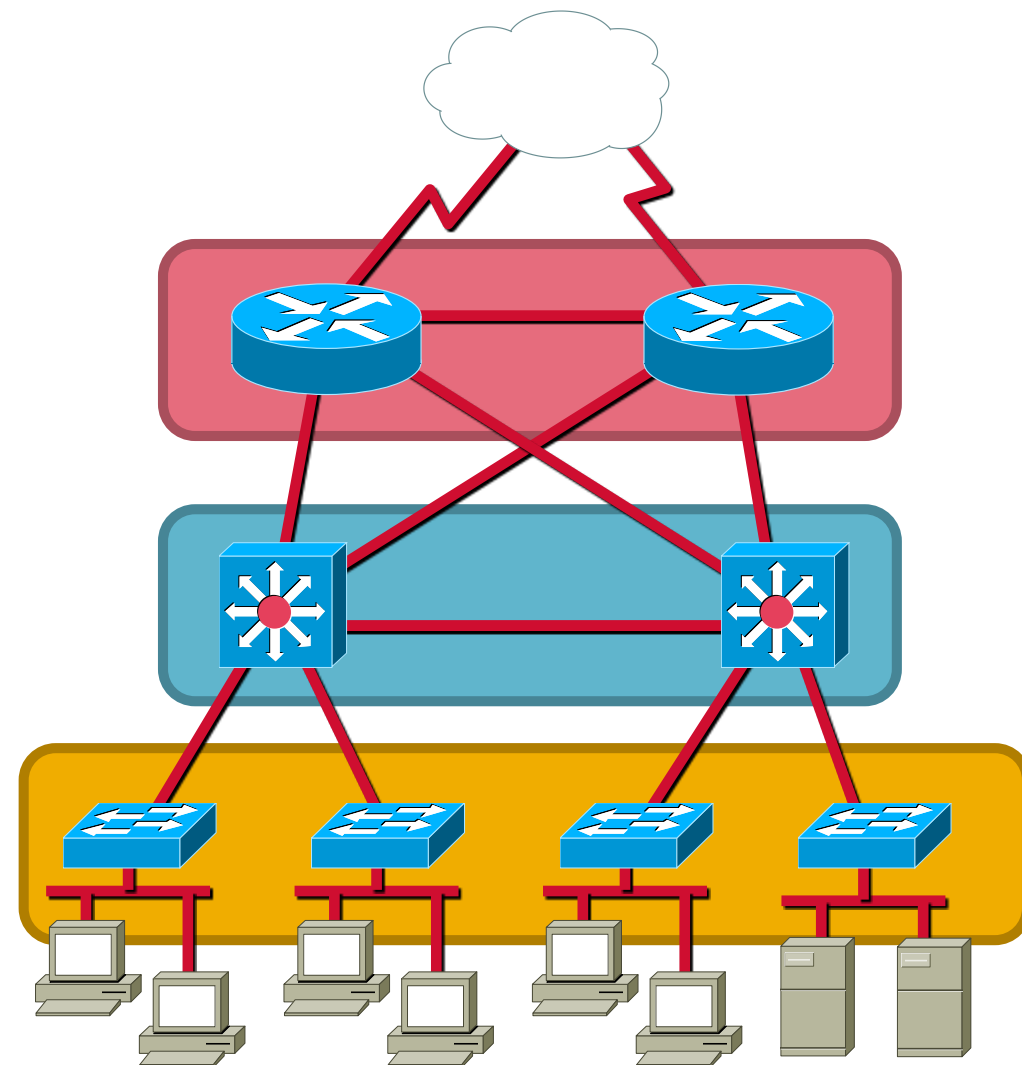
Topologia em Estrela Estendida

- Usa a topologia em estrela para ser criada para ser criada
- Une as estrelas individuais vinculando os switches, estendendo o comprimento e o tamanho da rede



Topologia Hierárquica

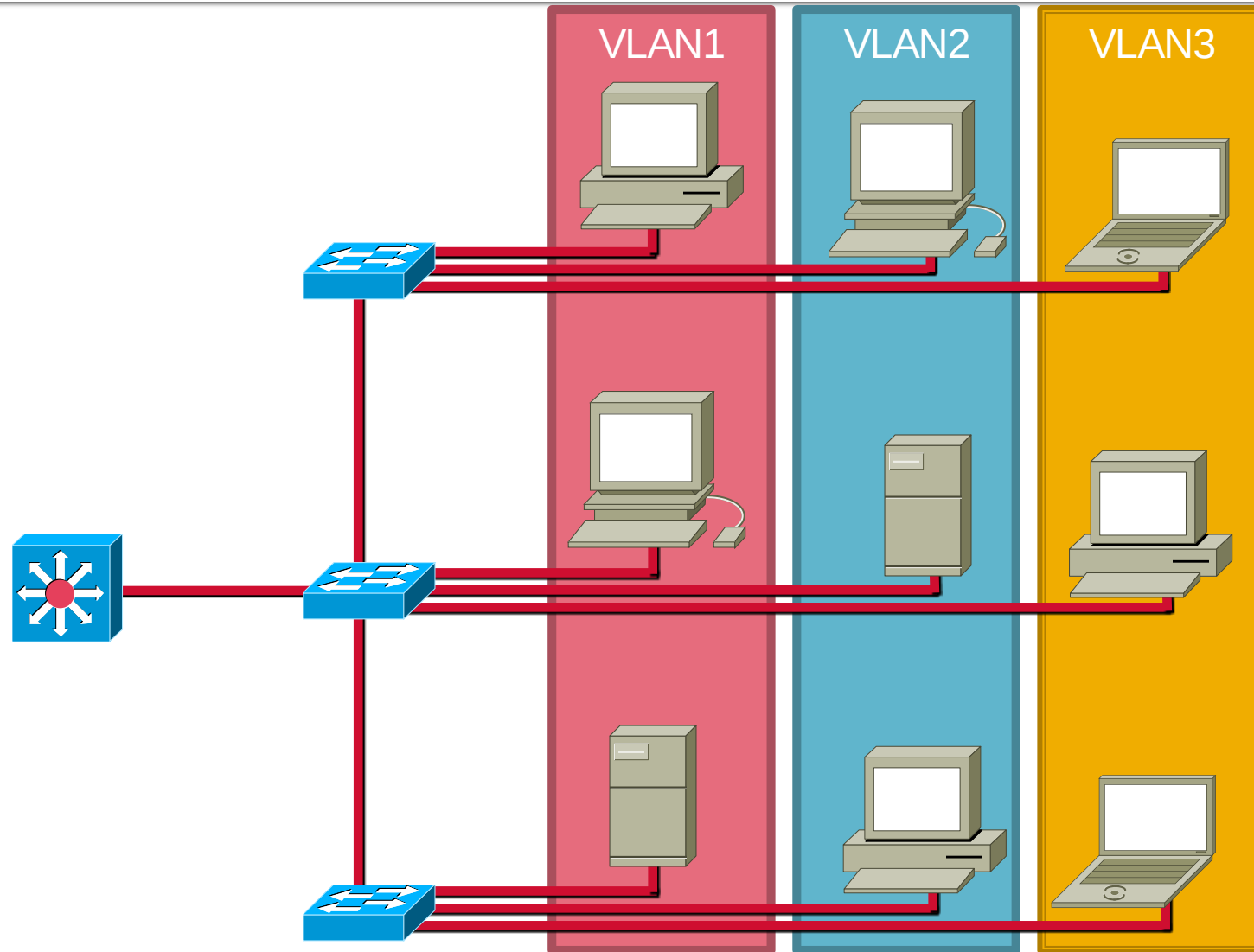
- Semelhante à Estrela Estendida
- Organiza a rede em camadas funcionais
- Modelo de 3 camadas recomendado para redes corporativas
 - Camada do núcleo (core)
 - Camada de distribuição
 - Camada de acesso



VLAN (Virtual LAN)

- Estações distribuídas pela rede se comunicam como se estivessem **conectadas em um mesmo segmento físico**
- **Domínio de broadcast lógico**
- Tráfego **broadcast** e **multicast** originado de uma estação membro da VLAN é repassado apenas para as portas do switch que estejam conectadas às estações desta VLAN
- Não há interferência do tráfego **multicast** e **broadcast** de um usuário nos demais
- Deve haver **suporte nos switches**

VLAN (Virtual LAN)



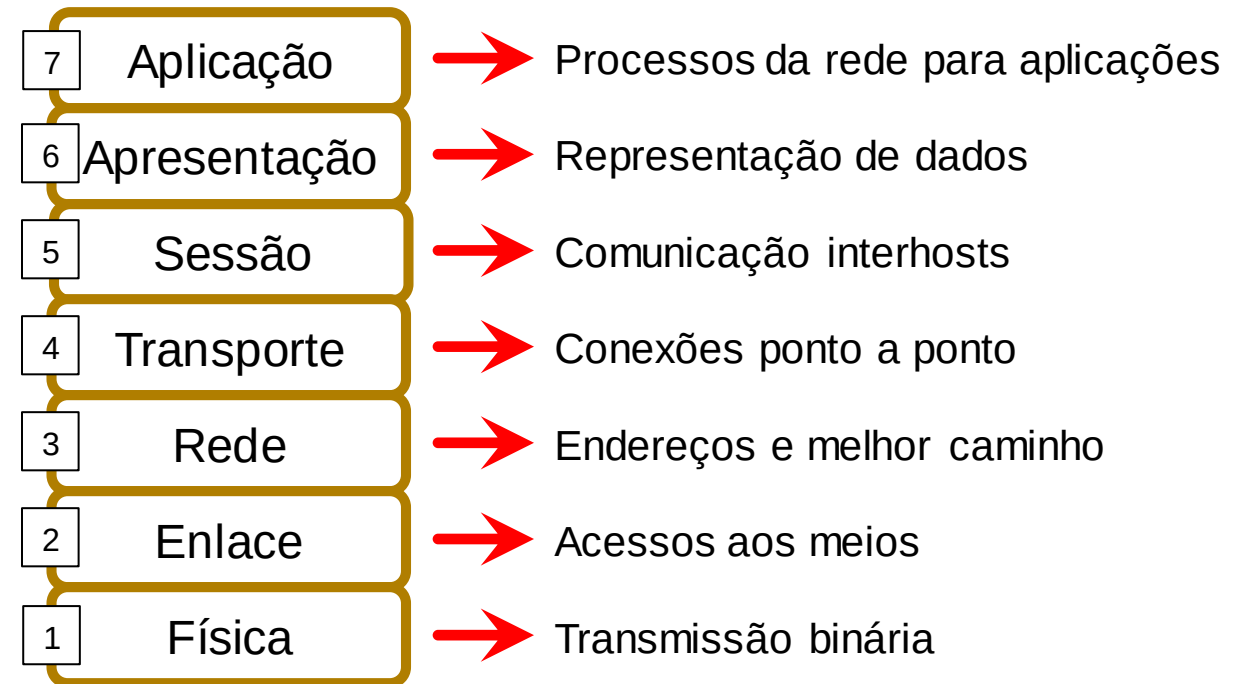
Modelo de Referência OSI

- Funcionalidades das 7 camadas

O Modelo OSI

As 7 camadas da comunicação em redes

- **OSI = Open Systems Interconnection**
- Padronizada pela **ISO** (International Organization for Standardization)
- Modelo conceitual criado para organizar as funções necessárias à comunicação entre sistemas.
- Cada camada oferece serviços à camada acima e utiliza serviços da camada abaixo.

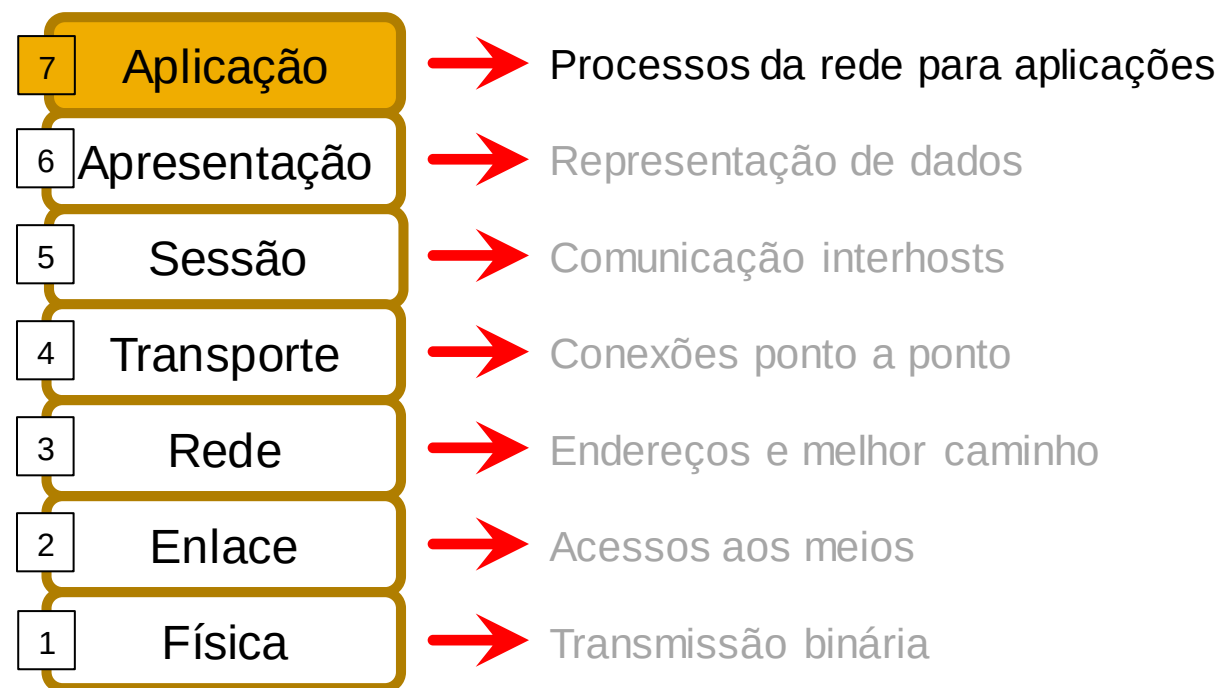


O Modelo OSI

Camada 7 — Aplicação

Onde ficam os serviços de rede percebidos pelas aplicações

- Fornece serviços de rede para os programas que precisam se comunicar.
- Não significa necessariamente a própria aplicação: são protocolos e serviços utilizados por ela.
- Exemplos: HTTP/HTTPS, DNS, SMTP, IMAP, FTP, SSH.
- Exemplo: um navegador utiliza HTTP/HTTPS para solicitar uma página web.

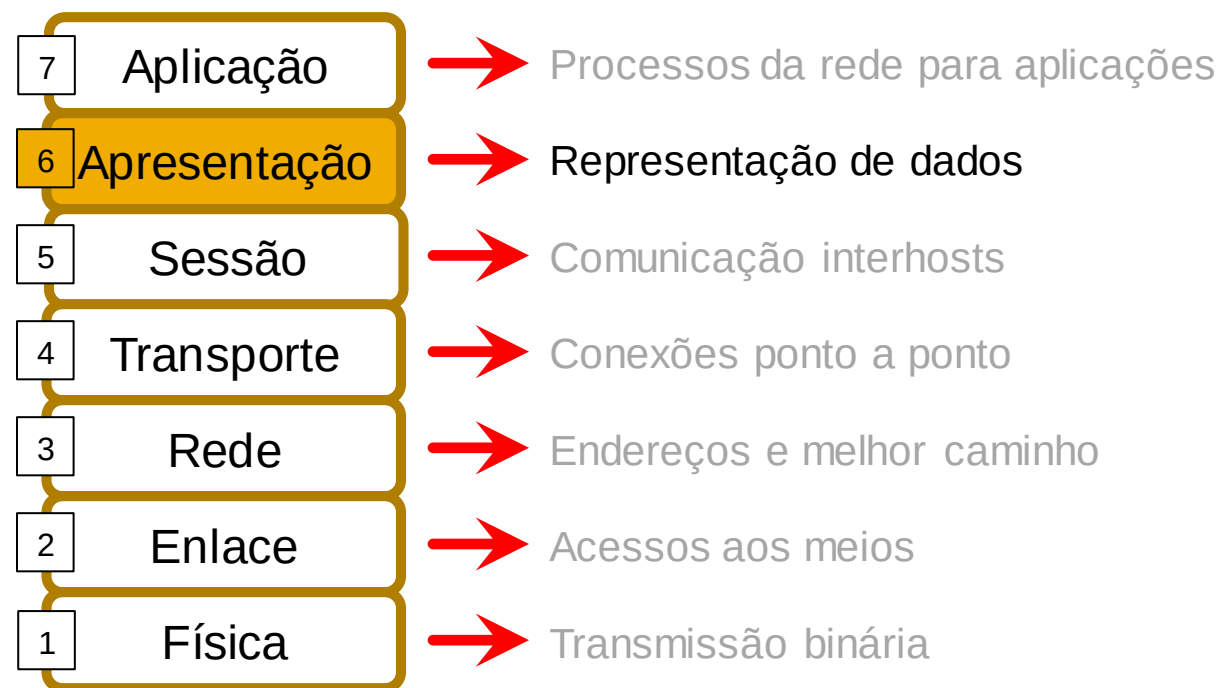


O Modelo OSI

Camada 6 — Apresentação

Como os dados são representados

- Define transformações necessárias para que os sistemas interpretem os dados de forma compatível.
- Funções tradicionalmente associadas: tradução de formatos, codificação, compressão e criptografia.
- Exemplos de conceitos: UTF-8, formatos de representação e codificação de dados.
- Na Internet moderna, muitas dessas funções aparecem incorporadas aos protocolos das aplicações.

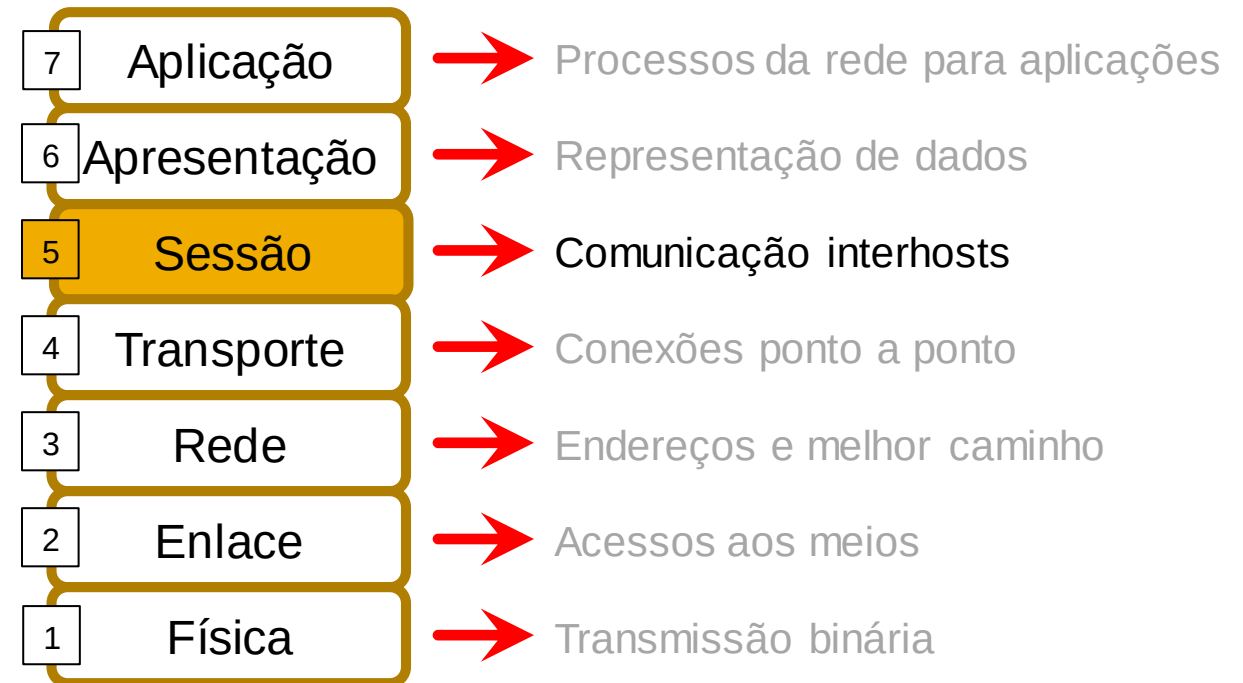


O Modelo OSI

Camada 5 — Sessão

Gerenciamento da comunicação entre aplicações

- Estabelece, mantém e encerra sessões de comunicação.
- Pode controlar o diálogo e a sincronização entre os participantes.
- Permite organizar uma comunicação longa em uma sequência lógica de interação.
- No conjunto TCP/IP moderno, muitas funções de sessão são implementadas pelas próprias aplicações ou por protocolos específicos.

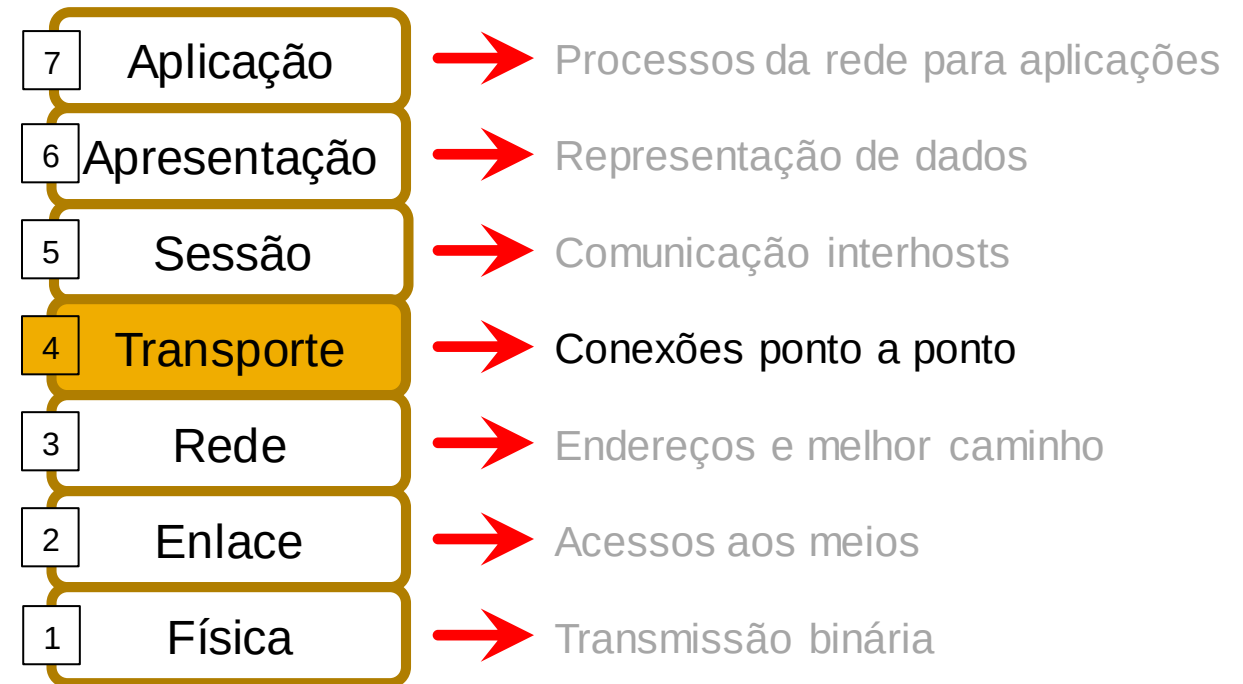


O Modelo OSI

Camada 4 — Transporte

Comunicação fim a fim

- Transporta dados entre processos de origem e destino.
- Pode oferecer confiabilidade, ordenação, controle de fluxo e controle de congestionamento.
- TCP: orientado à conexão e confiável.
- UDP: não estabelece conexão e possui menor sobrecarga.
- Utiliza números de porta para identificar processos/serviços.

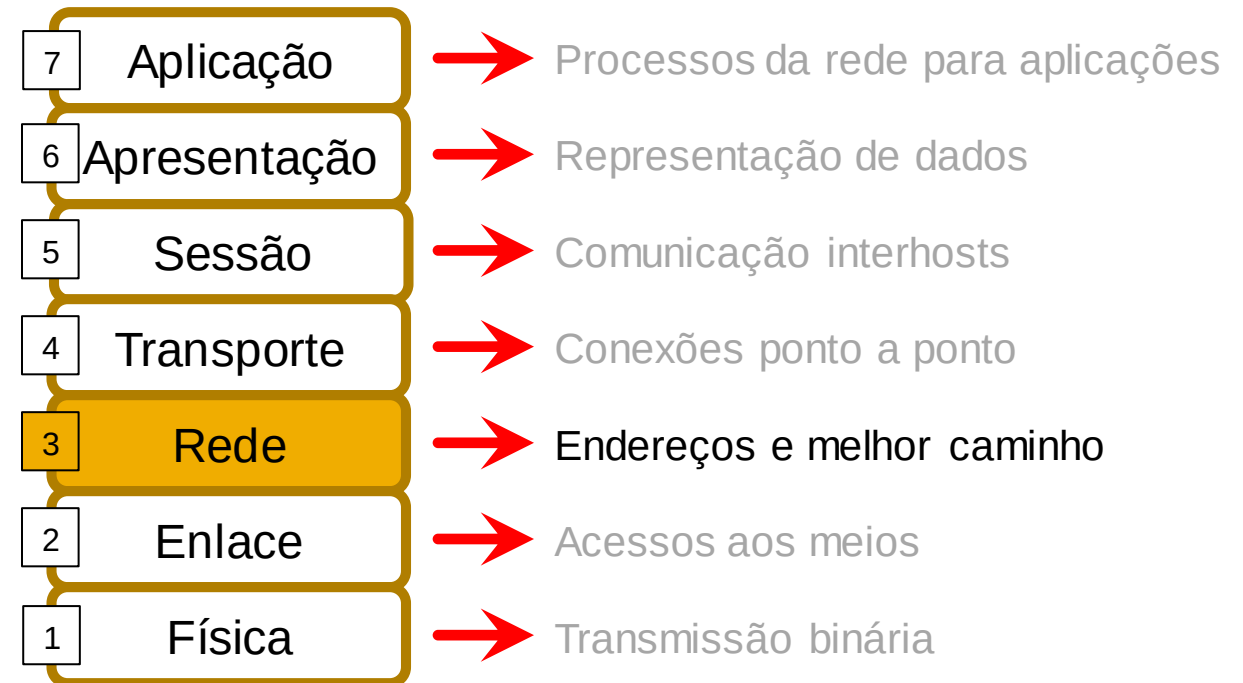


O Modelo OSI

Camada 3 — Rede

Entrega de pacotes entre redes

- Responsável pelo endereçamento lógico e pela escolha de caminhos entre redes.
- O protocolo IP é o principal exemplo no modelo TCP/IP.
- Roteadores operam principalmente nesta camada.
- Exemplo: um pacote destinado a outra rede precisa ser encaminhado por um ou mais roteadores.

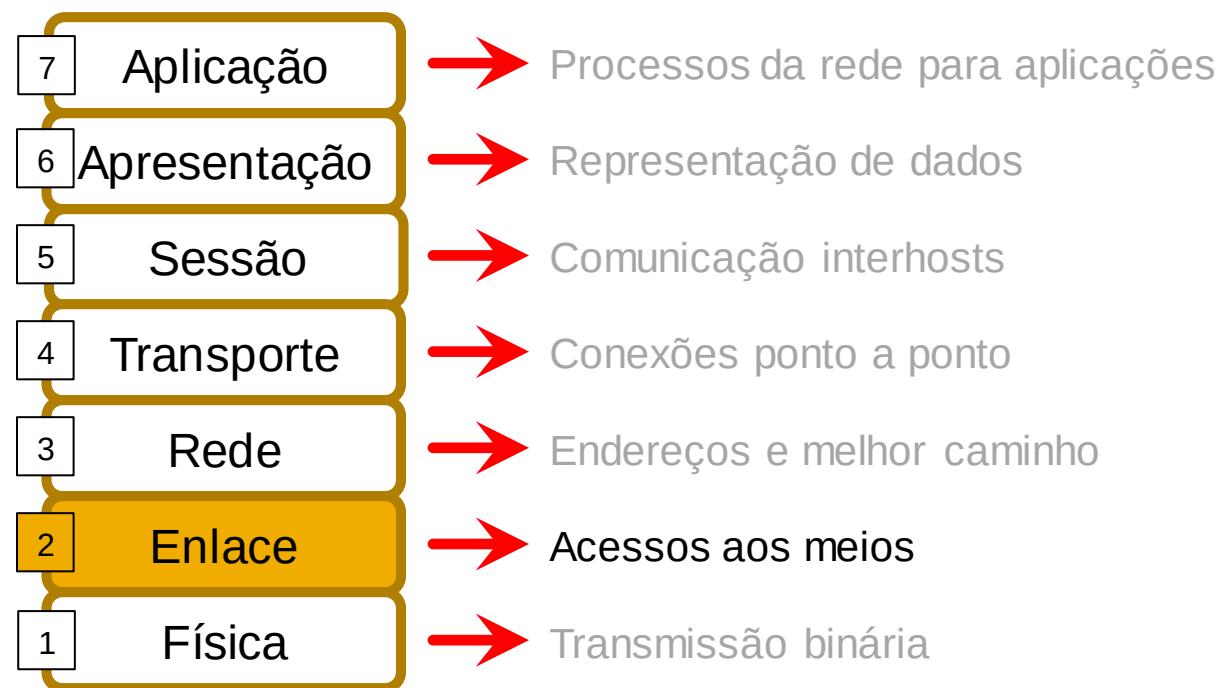


O Modelo OSI

Camada 2 — Enlace de Dados

Comunicação entre nós diretamente conectados

- Organiza a transmissão em quadros (frames).
- Utiliza endereços físicos, como endereços MAC em redes Ethernet e Wi-Fi.
- Detecta determinados erros de transmissão e controla o acesso ao meio, conforme a tecnologia.
- Switches Ethernet operam principalmente nesta camada.

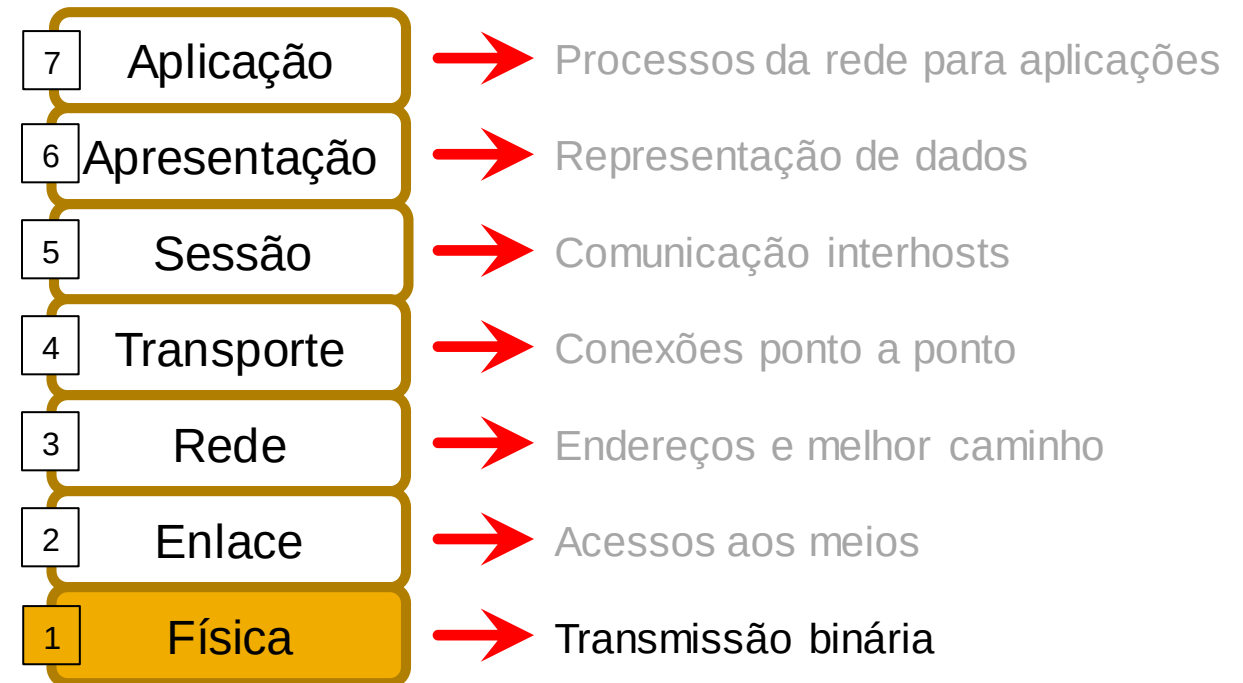


O Modelo OSI

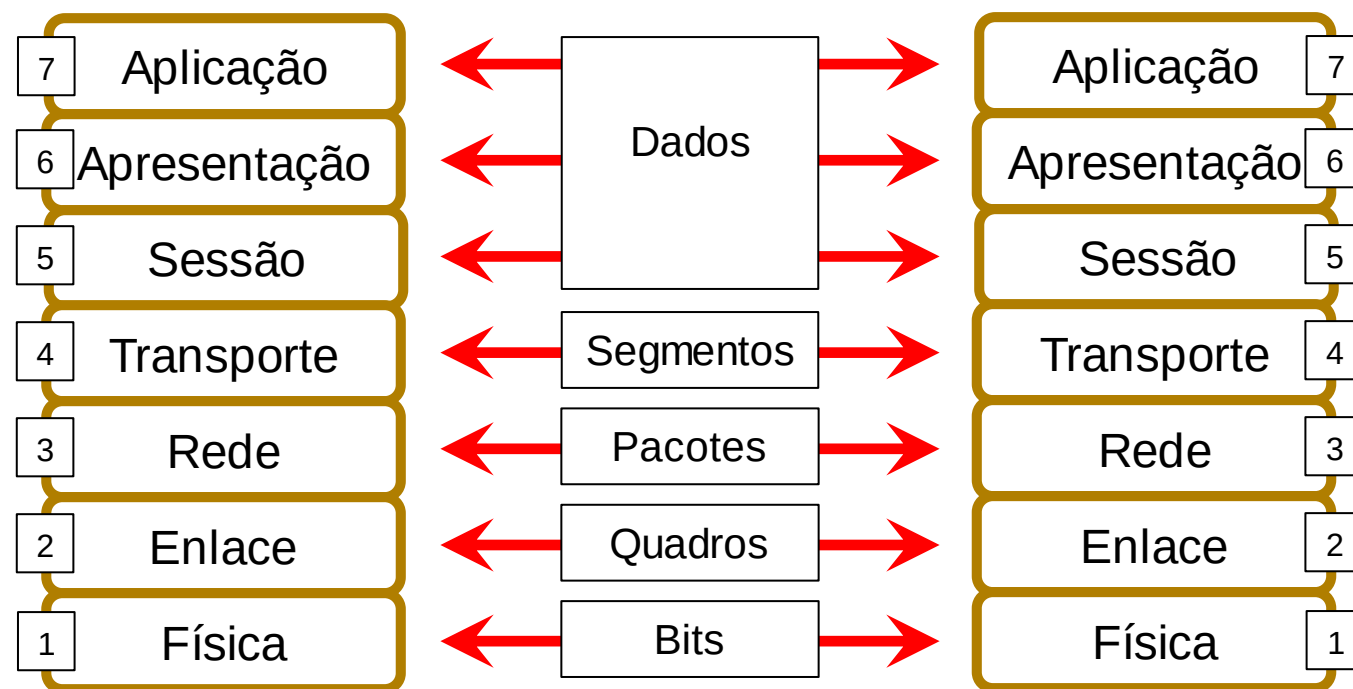
Camada 1 — Física

Transmissão dos bits

- Define como os bits são representados e transmitidos pelo meio.
- Envolve sinais elétricos, ópticos ou de rádio.
- Abrange características do meio, conectores, frequências, modulação e taxas de transmissão, dependendo da tecnologia.
- Exemplos de meios: cabo de cobre, fibra óptica e ondas de rádio.



O Modelo OSI



Arquitetura TCP/IP

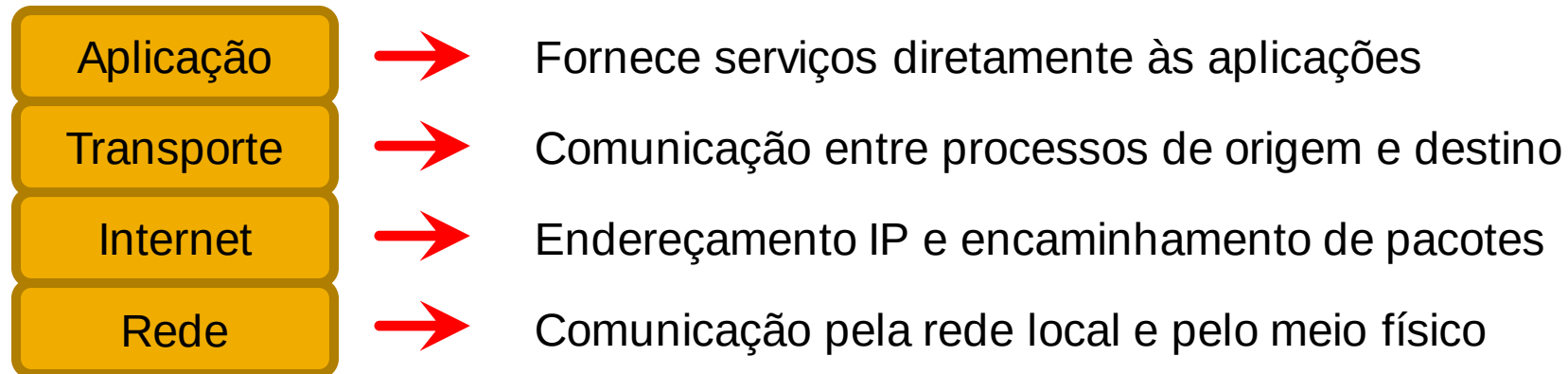
- Protocolos de Rede
- Protocolos de Internet
- Protocolos de Transporte
- Protocolos de Aplicação

Arquitetura TCP/IP

- TCP/IP é a arquitetura de protocolos que sustenta a comunicação na Internet.
- A representação tradicional possui quatro camadas.
- Cada camada agrupa protocolos e responsabilidades relacionadas.
- A separação em camadas permite desenvolver e substituir protocolos de forma independente.

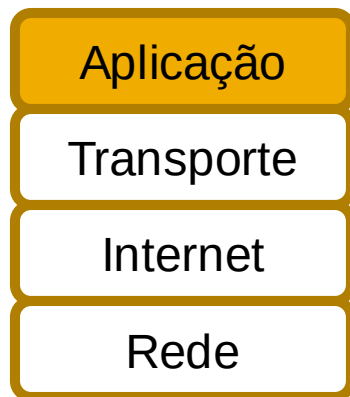


Arquitetura TCP/IP



Arquitetura TCP/IP

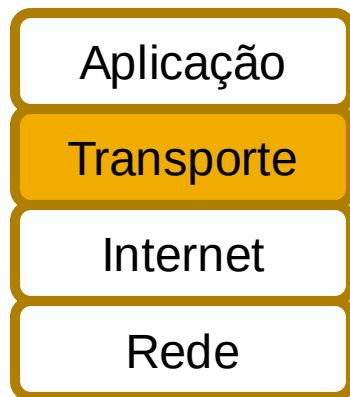
Camada de Aplicação



- É a camada mais próxima dos programas utilizados pelo usuário
- Define protocolos para diferentes tipos de serviços de rede
 - HTTP/HTTPS → Web
 - DNS → resolução de nomes
 - SMTP/IMAP → correio eletrônico
 - SSH → acesso remoto

Arquitetura TCP/IP

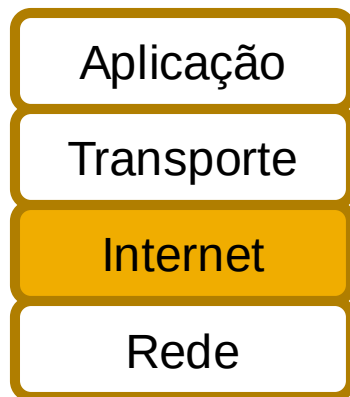
Camada de Transporte



- Fornece comunicação lógica fim a fim entre processos.
- Utiliza números de porta para identificar os processos envolvidos.
- TCP: orientado à conexão, confiável, ordenado e com controle de fluxo e congestionamento.
- UDP: sem conexão, menor sobrecarga e sem garantia de entrega ou ordenação.
- Exemplo: um servidor Web pode receber conexões TCP na porta 443.

Arquitetura TCP/IP

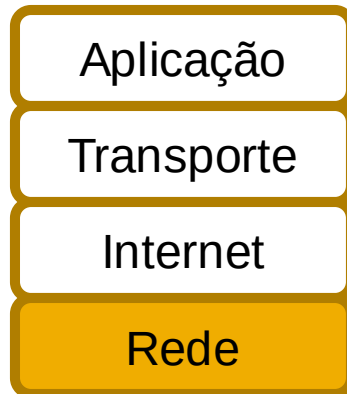
Camada Internet



- É responsável pelo endereçamento lógico e pelo encaminhamento entre redes
- Os principais protocolos são IP, nas versões IPv4 e IPv6
- Roteadores examinam endereços IP para decidir para onde encaminhar os pacotes
- O IP oferece entrega de melhor esforço: não garante, sozinho, que o pacote chegará

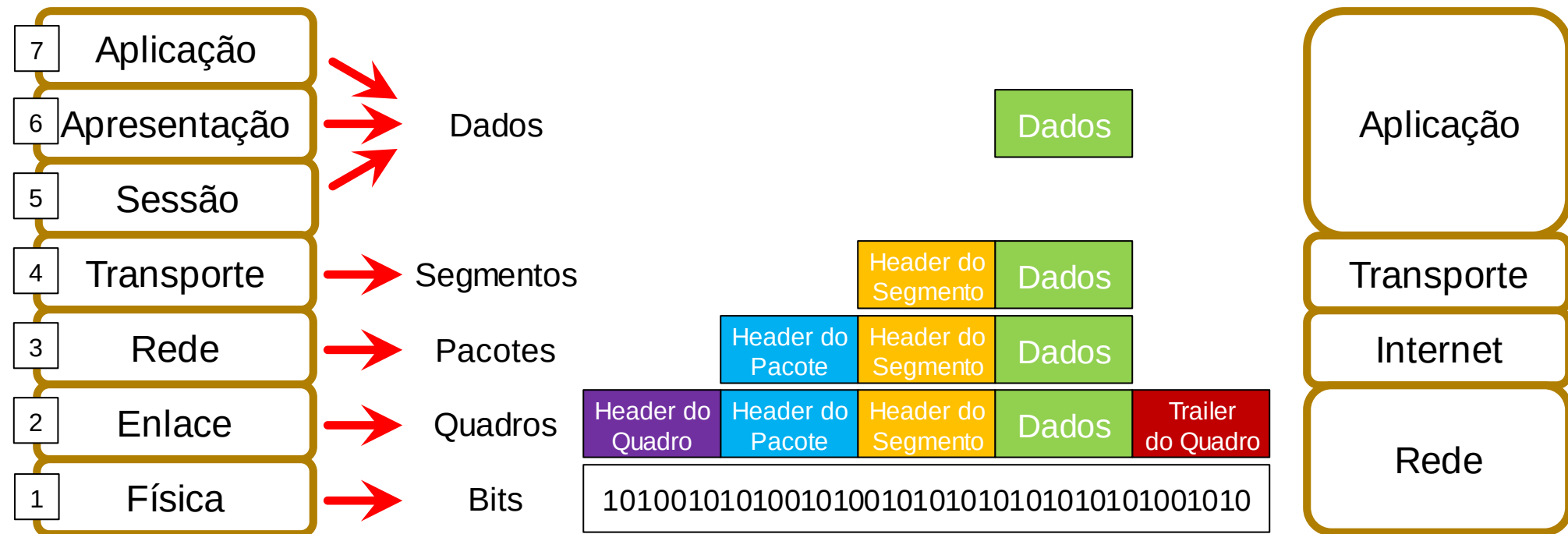
Arquitetura TCP/IP

Camada de Acesso à Rede



- Responsável pela comunicação através da tecnologia de rede utilizada no enlace
- Inclui funções de enlace e aspectos físicos
- Ethernet e Wi-Fi são exemplos de tecnologias associadas a esta camada
- Em uma LAN Ethernet, os dados são transportados em quadros
- O endereço MAC identifica interfaces na rede local

TCP/IP e Modelo OSI



Arquitetura TCP/IP

Protocolos de rede

- ARP
- IP

Camada de Rede

Camada OSI Física

Tecnologia	Meio	Descrição
10BASE-T	Par trançado	10 Mb/s
100BASE-TX	Par trançado	100 Mb/s
1000BASE-T	Par trançado	1 Gb/s
10GBASE-T	Par trançado	10 GB/s
Fibra óptica Ethernet	Fibra	Velocidade e distância
IEEE802.11	Rádio	Sem fio
Bluetooth	Rádio	PAN
DSL	Par metálico	Telefonia existente
DOCSIS	Coaxial 75 Ω	TV a cabo
SONET/SDH	Fibra	Multiplexado
5G	Rádio	Rede celular

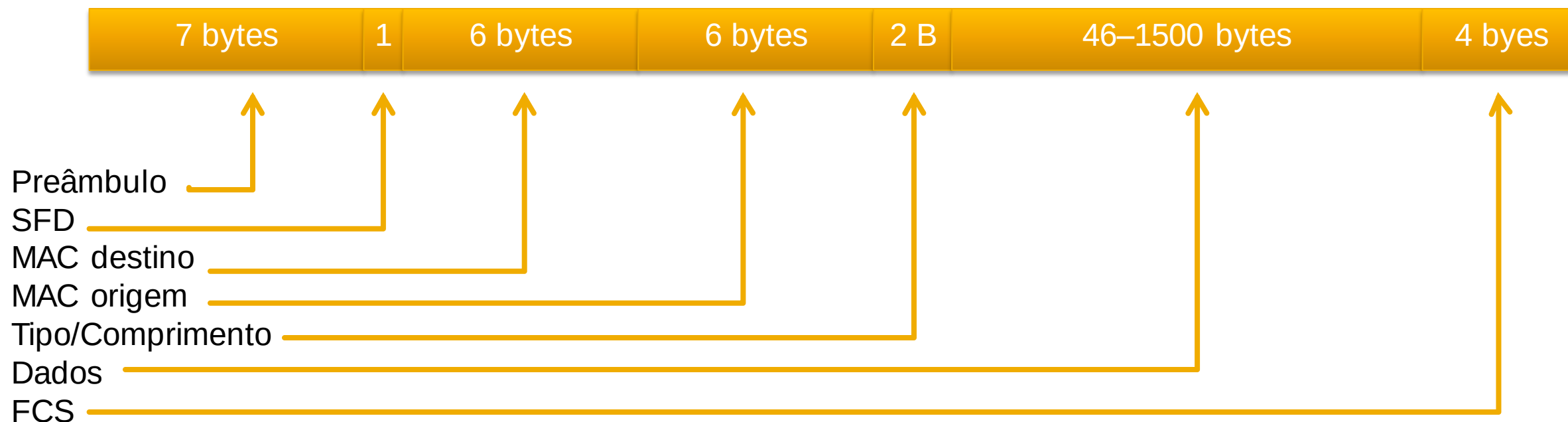
Camada OSI Enlace

Padrão	Nome
IEEE 802.3	Ethernet
IEEE 802.11	Wi-Fi
RFC 826	ARP
IEEE 802.1D	STP
IEEE 802.1W	RSTP
IEEE 802.1Q	VLAN

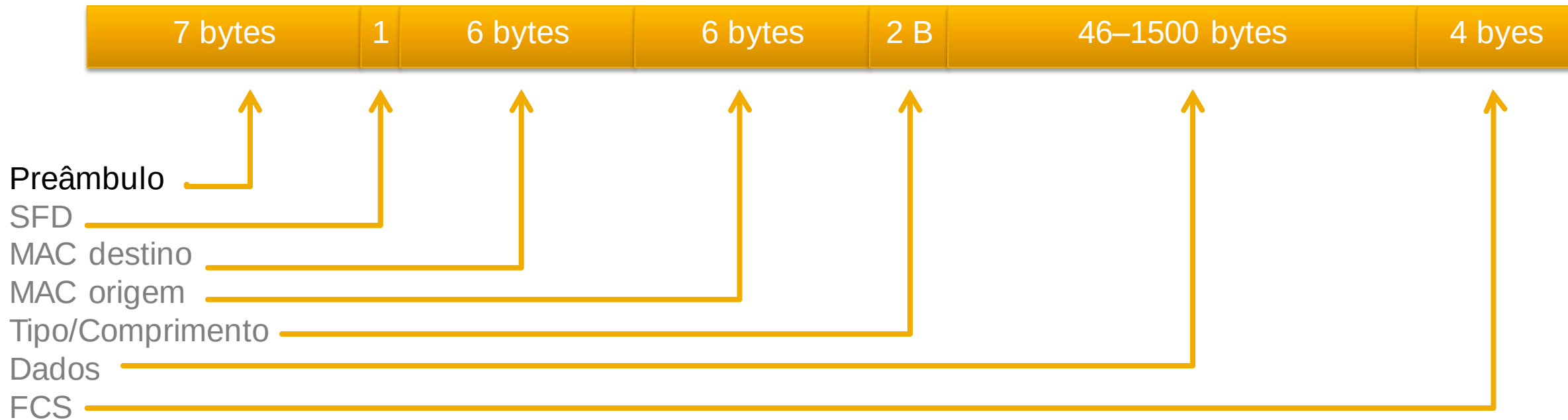
Quadros Ethernet e Wi-Fi

- IEEE 802.3 (Ethernet): redes cabeadas
- IEEE 802.11 (Wi-Fi): redes sem fio
- O frame é a unidade de transmissão da camada de enlace
- Frame transporta um pacote da camada de rede entre dispositivos diretamente conectados

Frame Ethernet — estrutura básica



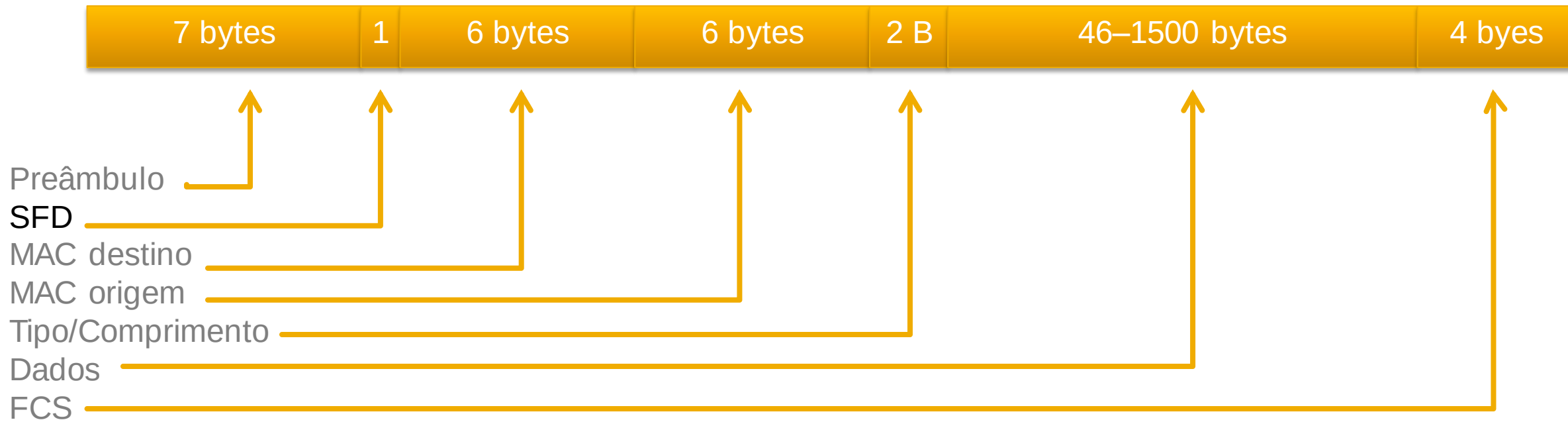
Frame Ethernet — estrutura básica



■ Preâmbulo

- Sequência de 7 bytes para sincronizar origem e destino
- 10101010 10101010 10101010 10101010 10101010 10101010 10101010

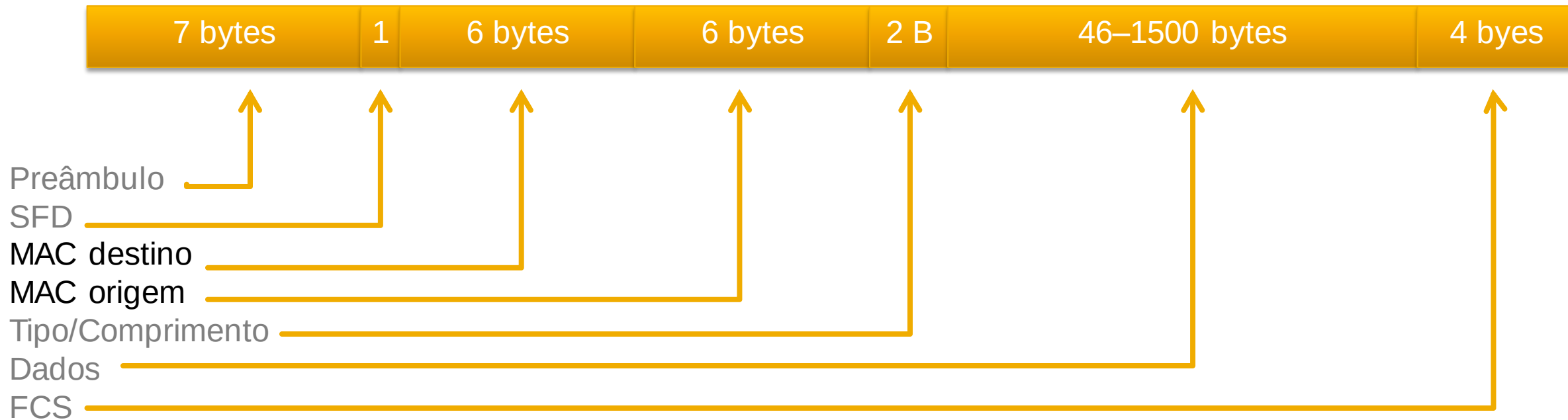
Frame Ethernet — estrutura básica



■ SFD

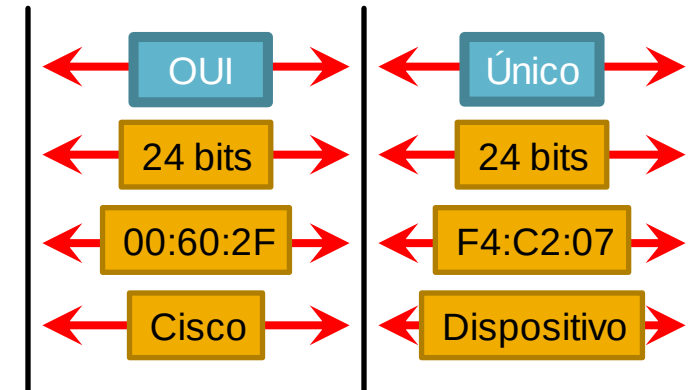
- Start Frame Delimiter
- Padrão 10101011 (0xD5)

Frame Ethernet — estrutura básica

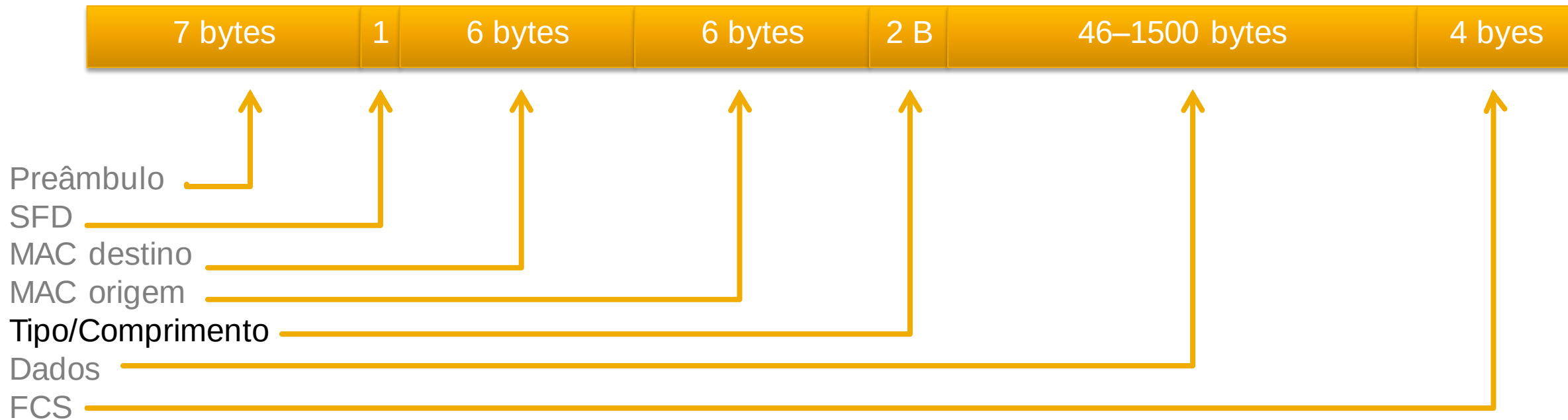


- MAC (Media Access Control) destino/origem

- Endereço físico não hierárquico de 48 bits
- Endereço MAC de broadcast: FF:FF:FF:FF:FF:FF



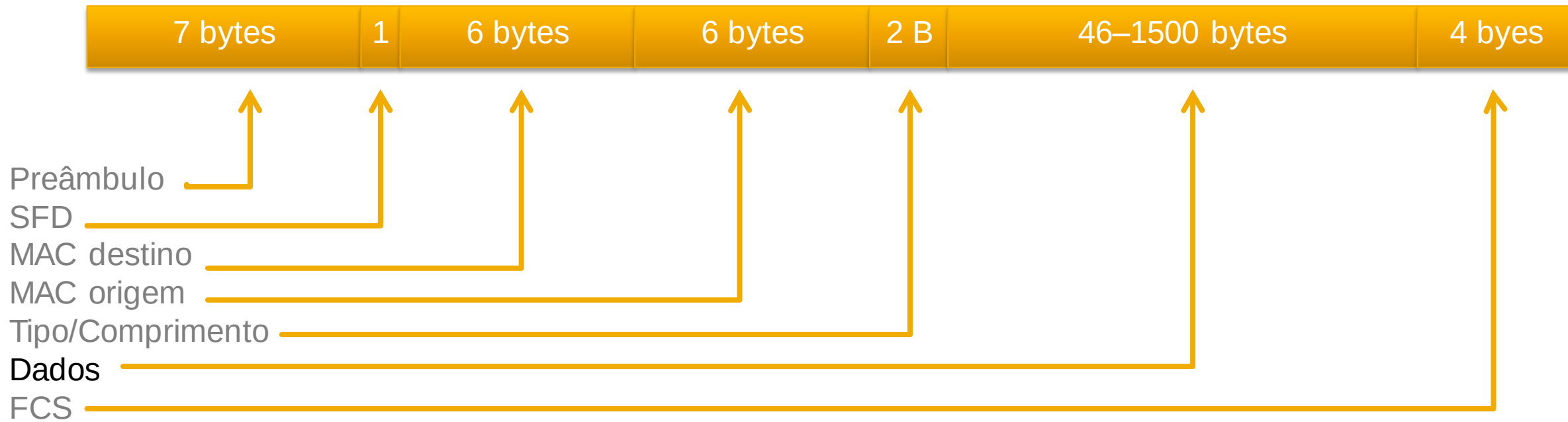
Frame Ethernet — estrutura básica



■ Tipo/Comprimento

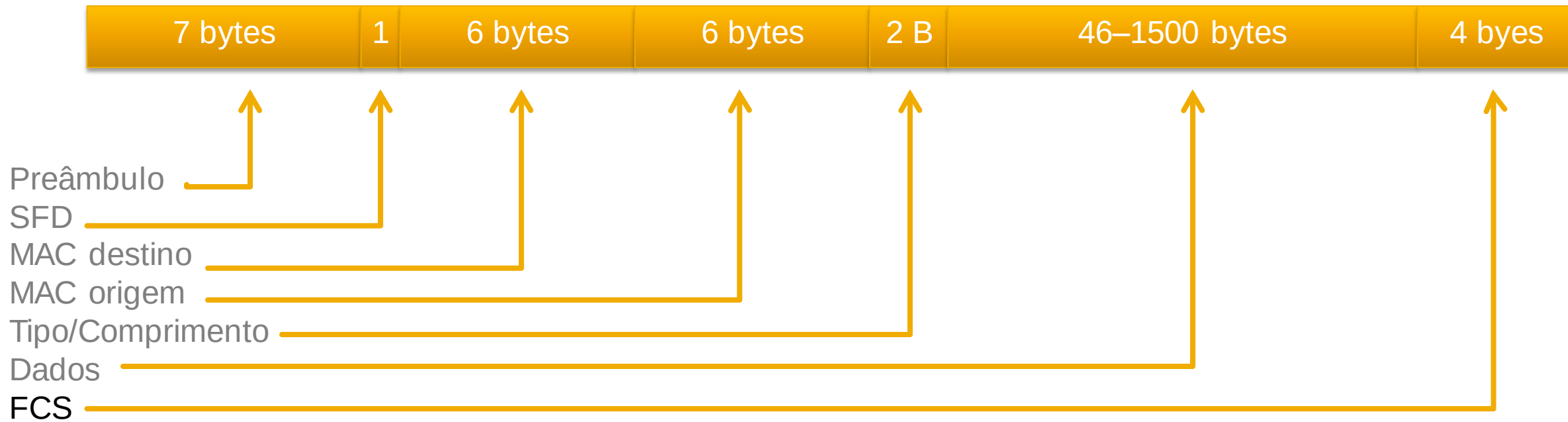
- ≤ 1500 : Comprimento
- ≥ 1536 : Tipo do payload
- Se o tamanho for menor do que 46 bytes, o restante do payload é preenchido com padding

Frame Ethernet — estrutura básica



- Dados (payload)
 - Pacote IP (incluindo os cabeçalhos)

Frame Ethernet — estrutura básica



- FCS (Frame Check Sequence)

- Verificação de erro do frame
- CRC-32

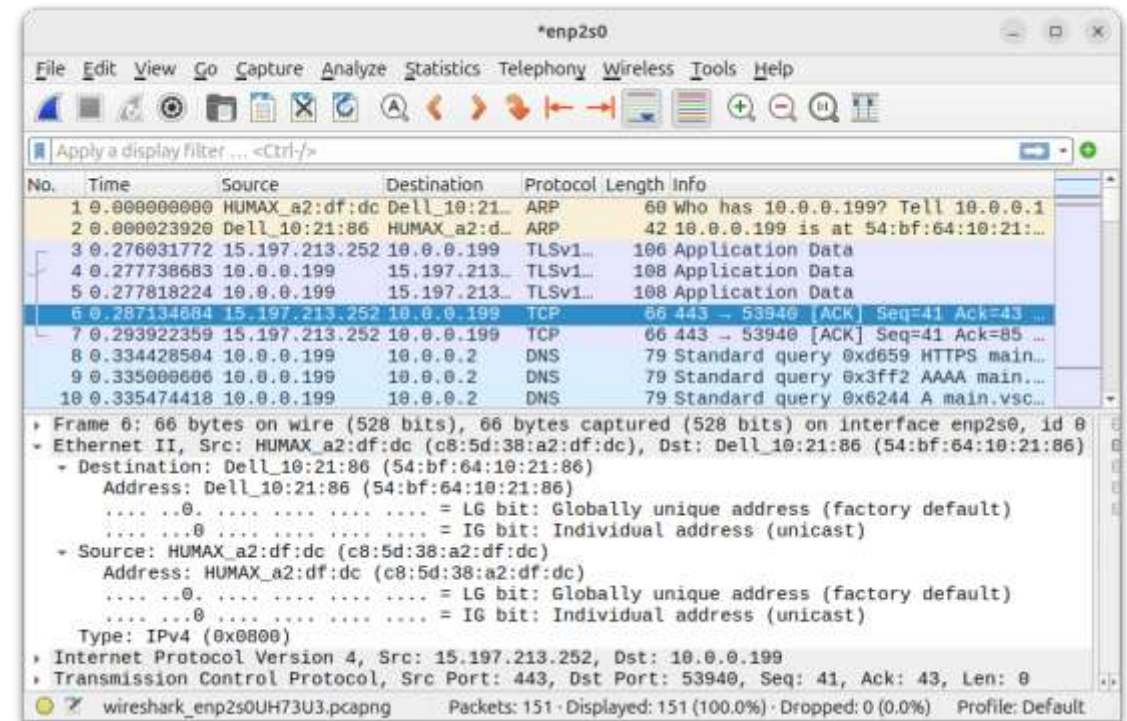
Frame Wireless — IEEE 802.11

Frame Control	Duration / ID	Address 1	Address 2	Address 3	Sequence Control	Address 4	Payload	FCS
2 bytes	2 bytes	6 bytes	6 bytes	6 bytes	0/2 bytes	6 bytes	0-2318 bytes	4 bytes

- O Wi-Fi possui um cabeçalho mais complexo que o Ethernet
- Address 4 aparece em determinados modos de operação
- 802.11 pode precisar de até quatro endereços MAC porque há **estação, access point e distribuição de frames**

Frame Ethernet: exemplo em Wireshark

- Ethernet II
 - Destination
 - MAC do próximo dispositivo na LAN
 - Source
 - MAC da interface transmissora
 - Type
 - 0x0800 → IPv4
 - 0x86DD → IPv6
 - 0x0806 → ARP
- Depois do cabeçalho Ethernet vem o protocolo encapsulado



Endereço IP

Endereçamento IP

- Endereço IP identifica uma interface em uma rede IP
- Máscara de rede determina qual parte do endereço representa a rede
- Sub-rede é uma divisão lógica de uma rede IP em redes menores
- CIDR (/n) é a forma moderna de representar a máscara

Endereço IPv4

- 32 bits escrito como 4 octetos (4 bytes)
- Cada octeto possui 8 bits e varia de 0 a 255
- Exemplo: 192.168.10.25

192	168	10	25
-----	-----	----	----

Máscara de Rede

- A máscara indica onde termina a parte da rede e começa a parte dos hosts
- Hosts da mesma sub-rede compartilham o mesmo prefixo de rede
- Exemplo:
 - 192.168.10.25/24
 - rede 192.168.10.0
 - host 25

Máscara de Rede

- A máscara possui 32 bits, assim como o endereço IPv4
- Bits 1 indicam a parte da rede
- Bits 0 indicam a parte dos hosts
- /n informa quantos bits iniciais da máscara são 1
- Exemplos:
 - /24 = 255.255.255.0
 - /22 = 255.255.255.252
 - /16 = 255.255.0.0

Endereço de Rede

- Operação AND bit a bit entre endereço e máscara

- Exemplo:

- 192.168.10.25/24

192.168.10.25 &

255.255.255.0

192.168.10.0

11000000.10101000.00001010.00011001

11111111.11111111.11111111.00000000

11000000.10101000.00001010.00000000

Endereço de Broadcast

- Endereço de rede: todos os bits da parte de host são 0
- Broadcast da sub-rede: todos os bits da parte de host são 1
- Os dois delimitam o intervalo de endereços da sub-rede
- Exemplo:
 - 192.168.10.64/26
 - rede .64
 - broadcast .127

Quantidade de endereços em uma sub-rede

- Se o prefixo é /n, existem $(32 - n)$ bits para hosts
- Total de endereços = $2^{(32 - n)}$
- /24 $\rightarrow 2^8 = 256$ endereços
- /26 $\rightarrow 2^6 = 64$ endereços

Em IPv4 tradicional, rede e broadcast não são atribuídos a hosts

Exemplo Completo

Endereço IP	192.168.10.25
Máscara	255.255.255.0 (/24)
Endereço de rede	192.168.10.0
Endereço de broadcast	192.168.10.255
Endereços de host utilizáveis	192.168.10.1 – 192.168.10.254

Sub-Rede

- Uma rede IP pode ser dividida em várias redes menores
- Para criar sub-redes, aumentamos o número de bits do prefixo
- Exemplo:
 - Uma rede /24 pode ser dividida em redes /26
 - Isso cria 4 sub-redes, cada uma com 64 endereços
 - 64 endereços = 62 host + broadcast + rede

Sub-Rede

- 172.16.0.0/16
 - Criando sub-redes com máscara /18 (255.255.192.0)

Rede	Endereço de rede	Endereço de hosts	Endereço de Broadcast
172.16.0.0/18	172.16.0.0	172.16.0.1 → 172.16.63.254	172.16.63.255
172.16.64.0/18	172.16.64.0	172.16.64.1 → 172.16.127.254	172.16.127.255
172.16.128.0/18	172.16.128.0	172.16.128.1 → 172.16.191.254	172.16.191.255
172.16.192.0/18	172.16.192.0	172.16.192.1 → 172.16.255.254	172.16.255.255

Comunicação dentro da Sub-Rede

- Configuração do host origem:
 - Endereço IP: 10.0.0.146
 - Máscara: 255.255.255.0 (/24)
- Destino:
 - Endereço IP: 10.0.0.252
- Cálculo do endereço de rede
 - Origem: $10.0.0.146 \& 255.255.255.0 = 10.0.0.0$
 - Destino: $10.0.0.252 \& 255.255.255.0 = 10.0.0.0$
 - Conclusão: mesma rede, comunicação direta!

Comunicação entre Sub-Redes

- Configuração do host origem:
 - Endereço IP: 10.0.0.146
 - Máscara: 255.255.255.0 (/24)
- Destino:
 - Endereço IP: 10.0.15.252
- Cálculo do endereço de rede
 - Origem: $10.0.0.146 \& 255.255.255.0 = 10.0.0.0$
 - Destino: $10.0.15.252 \& 255.255.255.0 = 10.0.15.0$
 - Conclusão: redes diferentes, comunicação via gateway!

Wireshark

Inspeção de datagramas em tempo real

Wireshark

- Analisador de protocolos de rede (Packet Sniffer) gratuito e de código aberto.
- Captura pacotes em tempo real que trafegam pela placa de rede (Ethernet/Wi-Fi).
- Permite decodificar e examinar cabeçalhos e conteúdo de centenas de protocolos.
- Usado por administradores de redes, profissionais de segurança e desenvolvedores.



Interface Principal

Lista de Pacotes

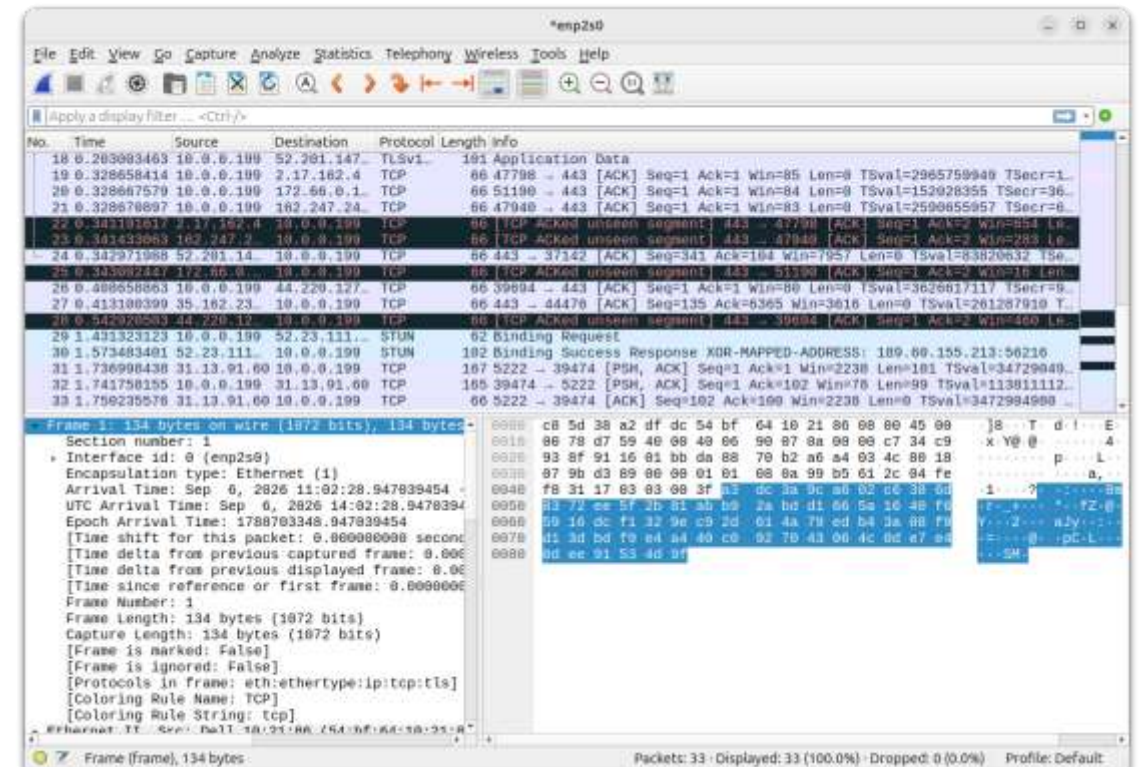
- Exibe todos os pacotes capturados em ordem cronológica, com origem, destino, protocolo e informações resumidas.

Detalhes do Pacote

- Mostra a estrutura em camadas (modelo OSI/TCP-IP) do pacote selecionado, expandindo cada cabeçalho.

Bytes do Pacote

- Exibe os dados brutos do pacote selecionado em formato Hexadecimal e caracteres ASCII correspondentes.



Interface Principal

Lista de Pacotes

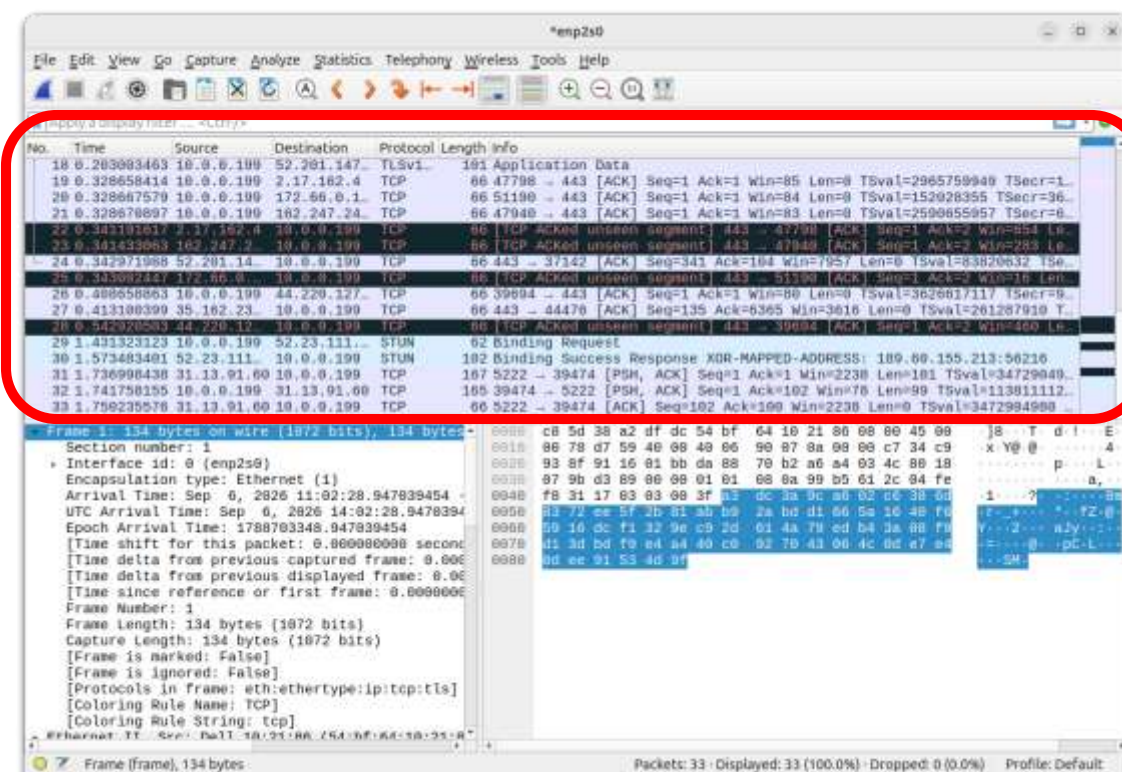
- Exibe todos os pacotes capturados em ordem cronológica, com origem, destino, protocolo e informações resumidas.

Detalhes do Pacote

- Mostra a estrutura em camadas (modelo OSI/TCP-IP) do pacote selecionado, expandindo cada cabeçalho.

Bytes do Pacote

- Exibe os dados brutos do pacote selecionado em formato Hexadecimal e caracteres ASCII correspondentes.



Interface Principal

Lista de Pacotes

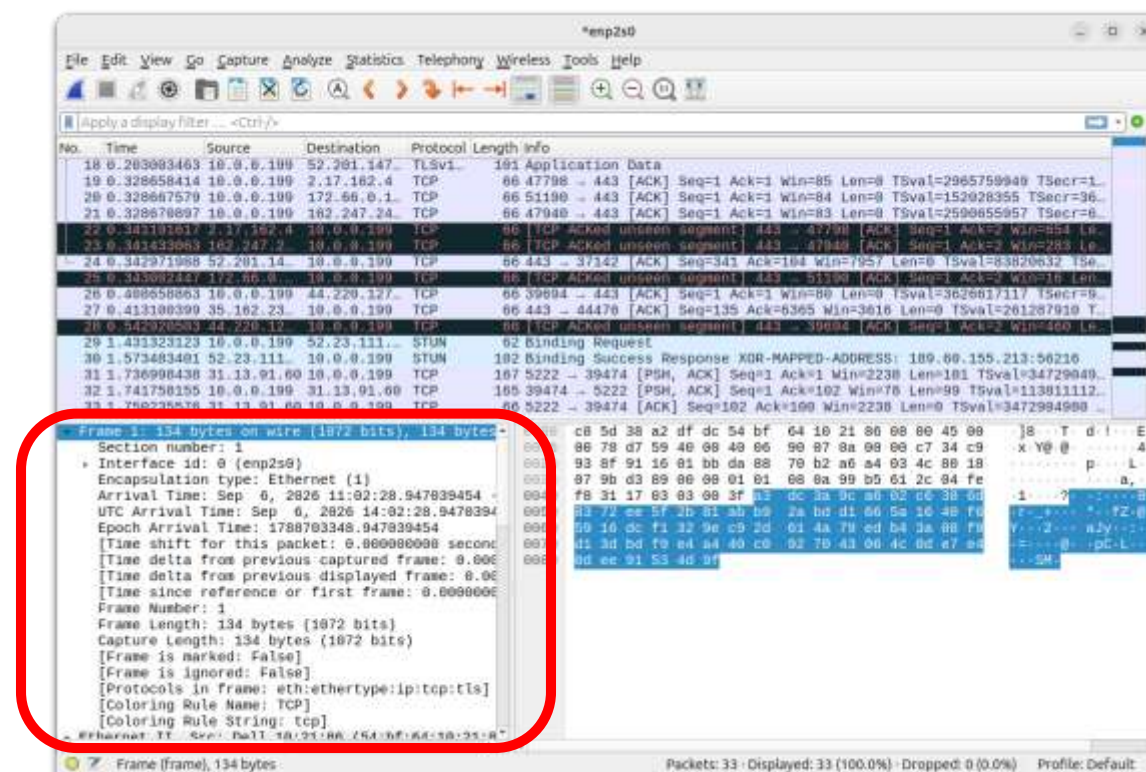
- Exibe todos os pacotes capturados em ordem cronológica, com origem, destino, protocolo e informações resumidas.

Detalhes do Pacote

- Mostra a estrutura em camadas (modelo OSI/TCP-IP) do pacote selecionado, expandindo cada cabeçalho.

Bytes do Pacote

- Exibe os dados brutos do pacote selecionado em formato Hexadecimal e caracteres ASCII correspondentes.



Interface Principal

Lista de Pacotes

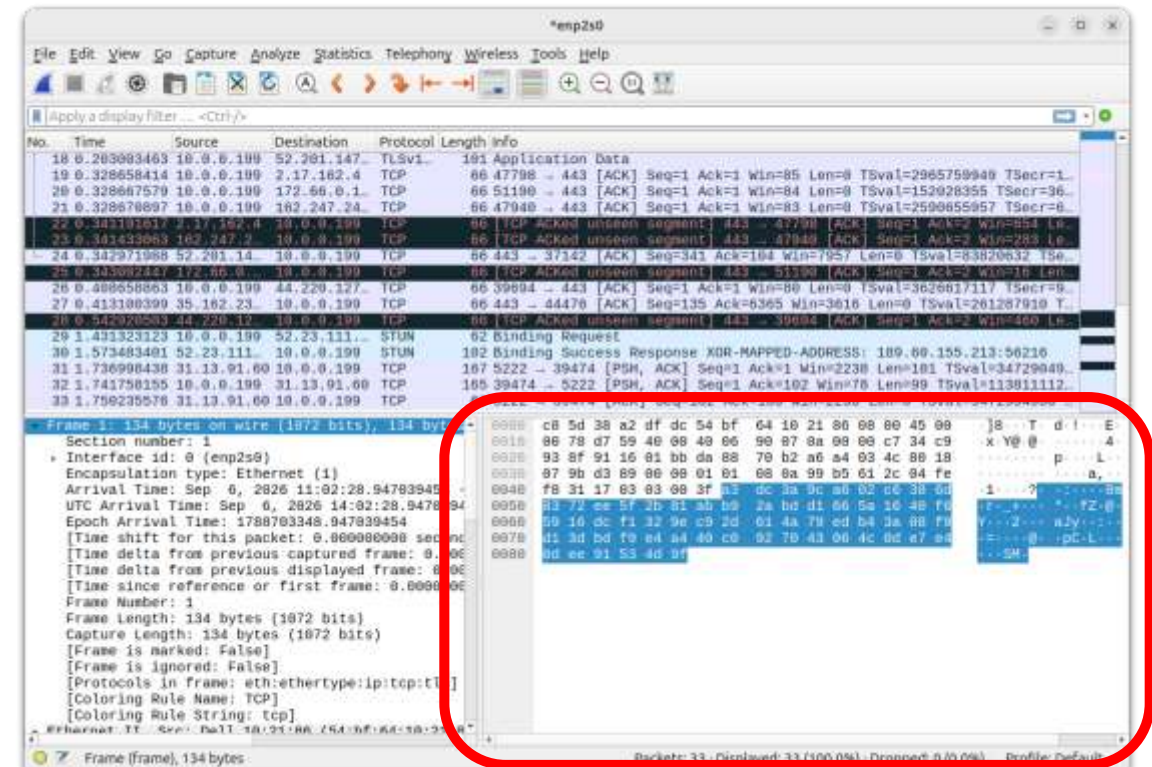
- Exibe todos os pacotes capturados em ordem cronológica, com origem, destino, protocolo e informações resumidas.

Detalhes do Pacote

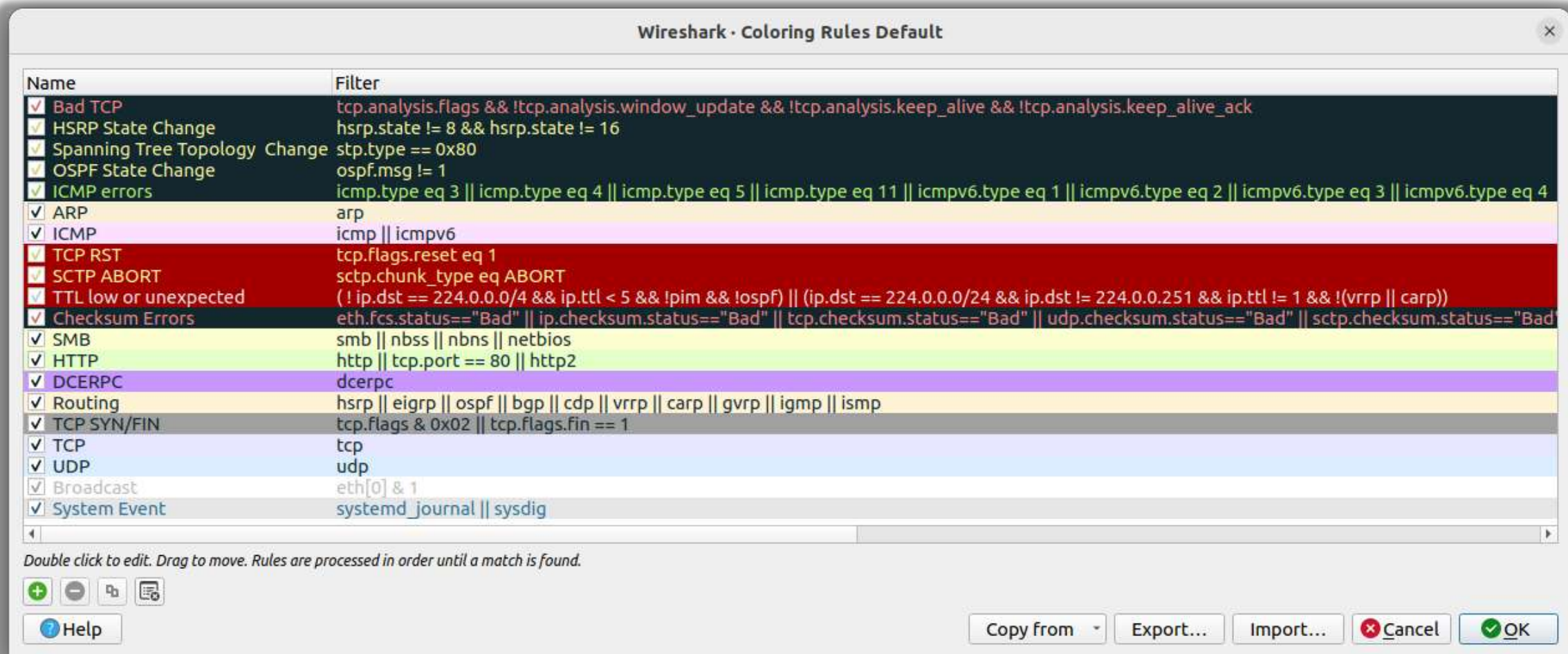
- Mostra a estrutura em camadas (modelo OSI/TCP-IP) do pacote selecionado, expandindo cada cabeçalho.

Bytes do Pacote

- Exibe os dados brutos do pacote selecionado em formato Hexadecimal e caracteres ASCII correspondentes.



Código de Cores



Iniciando a Captura

Escolha a Interface

- Na tela inicial, selecione a interface com tráfego ativo (Wi-Fi, Ethernet, Bluetooth).

Inicie a Captura

- Clique no ícone da *Barbatana de Tubarão Azul* no canto superior esquerdo.

Pare a Captura

- Clique no ícone do *Quadrado Vermelho* assim que coletar o tráfego desejado.



Iniciando a Captura

Escolha a Interface

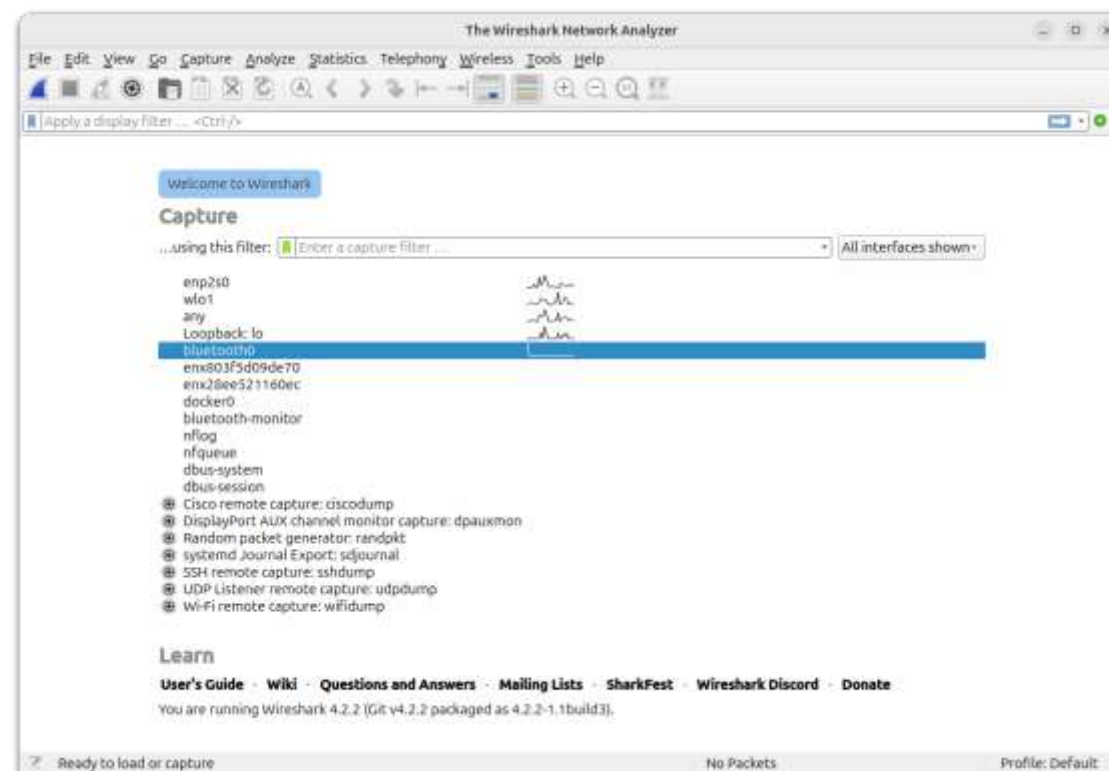
- Na tela inicial, selecione a interface com tráfego ativo (Wi-Fi, Ethernet, Bluetooth).

Inicie a Captura

- Clique no ícone da *Barbatana de Tubarão Azul* no canto superior esquerdo.

Pare a Captura

- Clique no ícone do *Quadrado Vermelho* assim que coletar o tráfego desejado.



Iniciando a Captura

Escolha a Interface

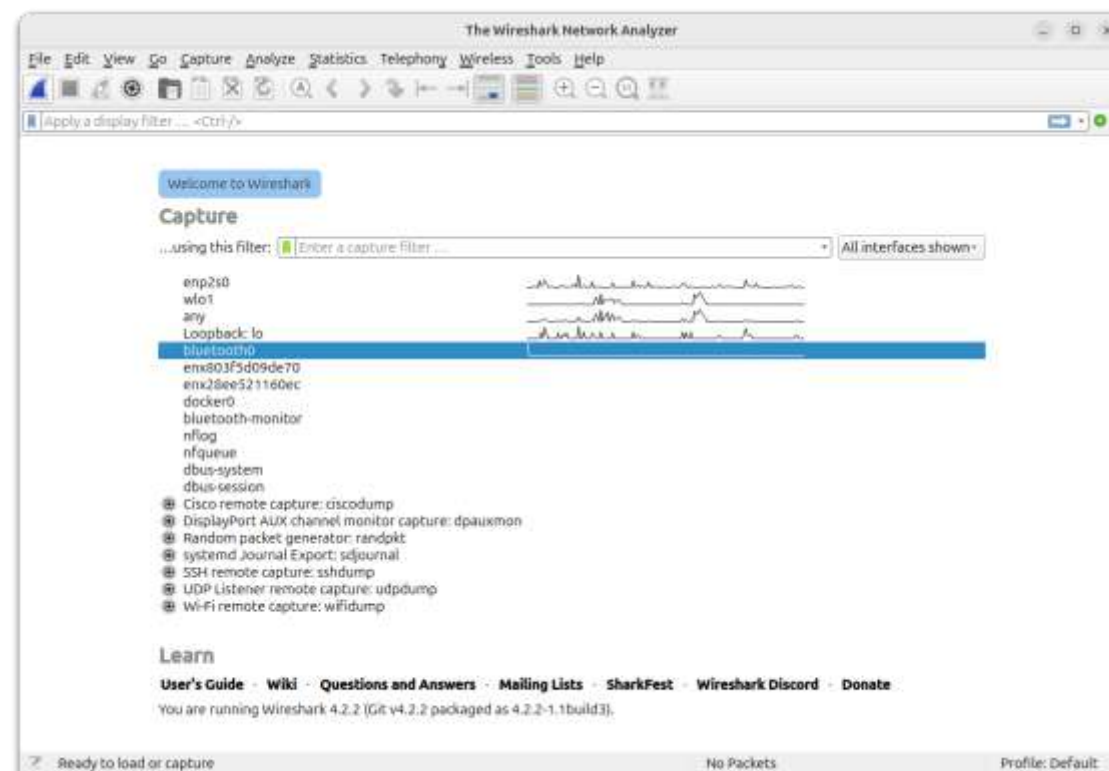
- Na tela inicial, selecione a interface com tráfego ativo (Wi-Fi, Ethernet, Bluetooth).

Inicie a Captura

- Clique no ícone da *Barbatana de Tubarão Azul* no canto superior esquerdo.

Pare a Captura

- Clique no ícone do *Quadrado Vermelho* assim que coletar o tráfego desejado.



Iniciando a Captura

Escolha a Interface

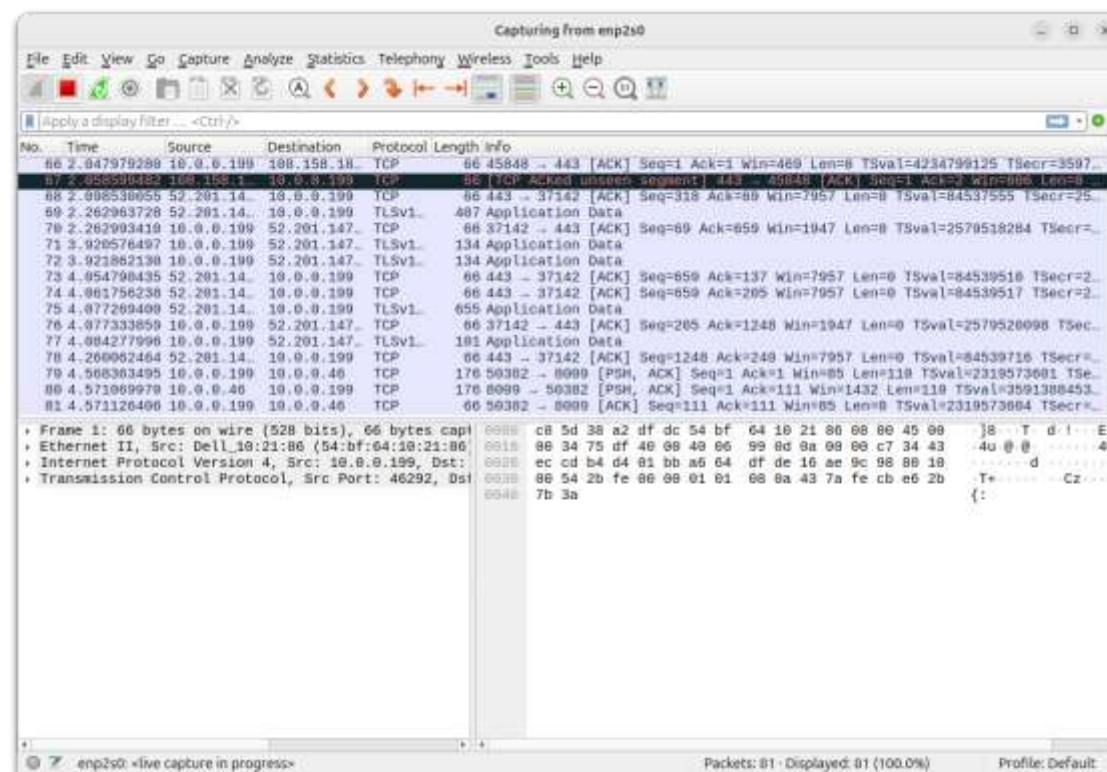
- Na tela inicial, selecione a interface com tráfego ativo (Wi-Fi, Ethernet, Bluetooth).

Inicie a Captura

- Clique no ícone da *Barbatana de Tubarão Azul* no canto superior esquerdo.

Pare a Captura

- Clique no ícone do *Quadrado Vermelho* assim que coletar o tráfego desejado.



Filtros Básicos

- **Filtrar Protocolo TCP**

- Digite tcp na barra de filtros para exibir apenas segmentos do protocolo TCP.

- **Filtrar Protocolo HTTP**

- Digite http para filtrar requisições e respostas da Web não criptografadas.

- **Filtrar por Endereço IP**

- Use ip.addr == 192.168.1.1 para isolar tráfego de um host específico.

- **Filtrar por Porta Específica**

- Use tcp.port == 80 ou tcp.port == 443 para inspecionar tráfegos por portas específicas.

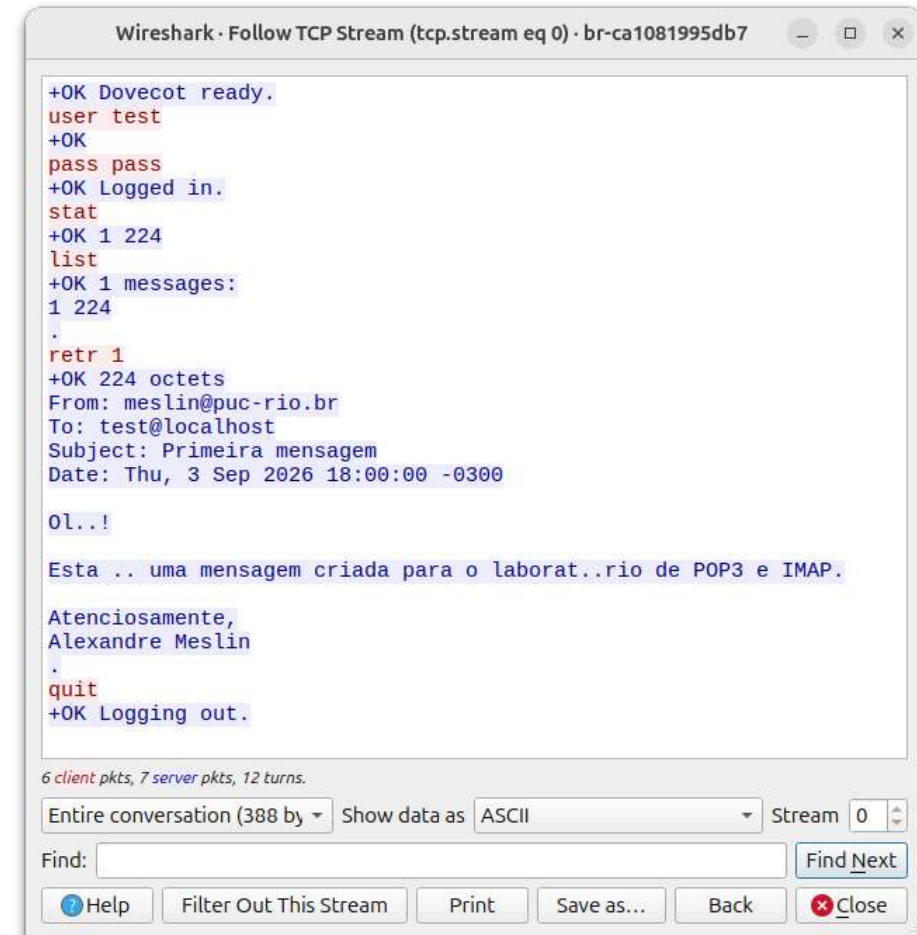
Fluxo TCP

■ TCP Stream

- O Wireshark reconstrói toda a conversa bidirecional entre cliente e servidor, unindo os pacotes individuais em um único diálogo contínuo.

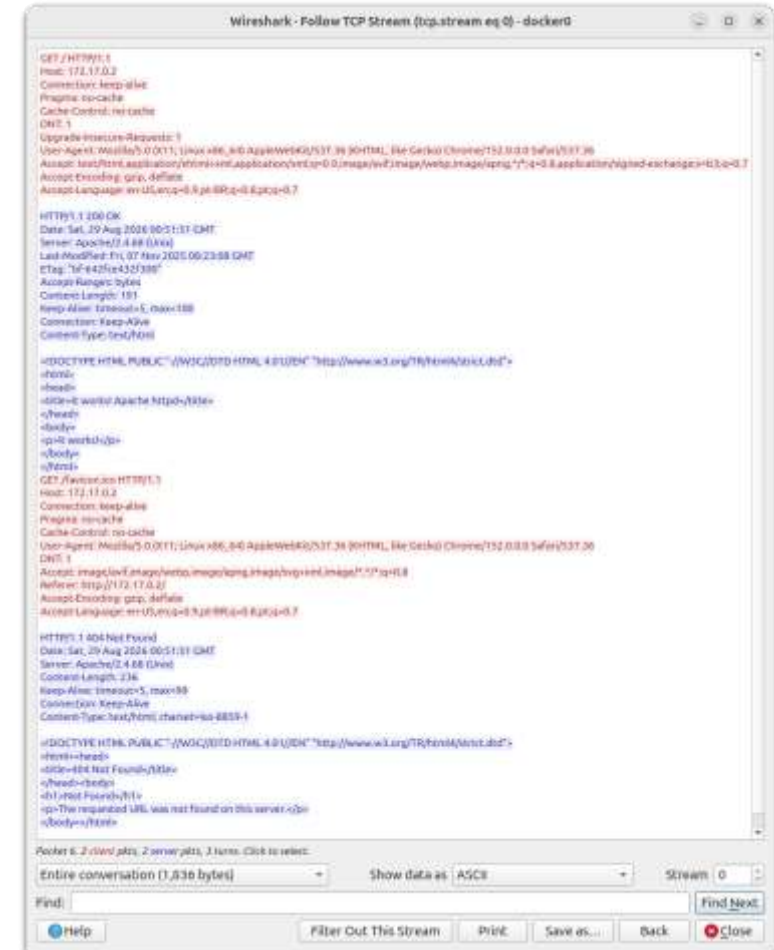
■ Seguindo fluxo

- Clique com o **botão direito** em qualquer pacote TCP.
- Navegue até **Follow > TCP Stream**.
- Uma nova janela será exibida mostrando a troca de dados em texto simples (Cliente em Vermelho, Servidor em Azul).



Fluxo HTTP

- **Seguir o fluxo HTTP**
 - Após aplicar o filtro **http**, identifique uma requisição (ex: GET ou POST). Clique com o botão direito sobre o pacote e selecione **Follow > HTTP Stream**.
- **Identificar no fluxo**
 - Você visualizará os cabeçalhos da requisição (Host, User-Agent), as respostas do servidor (200 OK, 404 Not Found) e o conteúdo transmitido (HTML, JSON, formulários).



Algumas Dicas

■ Salvar Arquivos

- Guarde as capturas no formato nativo .pcapng para posterior análise ou compartilhamento com a equipe.

■ Criptografia TLS/HTTPS

- Tráfego HTTPS é criptografado
- Para ler o conteúdo, use o fluxo HTTP em sites de teste (faremos isso)

■ Pacote Exibido vs Pacote Capturado

- O Wireshark captura TODOS os pacotes, mesmo os que não aparecem no filtro – cuidado com a memória

TCPDump

TCPDump

tcpdump is a data-network packet analyzer computer program that runs under a command line interface. It allows the user to display TCP/IP and other packets being transmitted or received over a network to which the computer is attached. Distributed under the BSD license, tcpdump is free software.

Tcpdump works on most Unix-like operating systems: Linux, Solaris, FreeBSD, DragonFly BSD, NetBSD, OpenBSD, OpenWrt, macOS, HP-UX 11i, and AIX. In those systems, tcpdump uses the libpcap library to capture packets. The port of tcpdump for Windows is called WinDump; it uses WinPcap, the Windows version of libpcap.

Instalação

- Atualize a base de dados dos pacotes
- Atualize os pacotes já instalados
- Instale o TCPdump

```
$ sudo apt update
```

```
$ sudo apt upgrade -y
```

```
$ sudo apt install tcpdump -y
```

Exemplo de Uso

```
$ sudo tcpdump -i br-844bdf0db4d4 -n port 53
```

```
tcpdump: verbose output suppressed, use -v[v]... for full protocol decode  
listening on br-844bdf0db4d4, link-type EN10MB (Ethernet), snapshot length 262144 bytes
```

```
19:47:54.453445 IP 172.20.0.2.45678 > 172.20.0.10.53: 25172+ A? www.meslin.com.br. (35)
```

```
19:47:54.453756 IP 172.20.0.10.53 > 172.20.0.2.45678: 25172 6/0/0 CNAME git.meslin.com.br.,  
CNAME alexandremeslin.github.io., A 185.199.109.153, A 185.199.111.153, A 185.199.108.153, A  
185.199.110.153 (156)
```

```
19:47:54.454888 IP 172.20.0.2.38772 > 172.20.0.10.53: 19292+ AAAA? alexandremeslin.github.io.  
(43)
```

```
19:47:54.455021 IP 172.20.0.10.53 > 172.20.0.2.38772: 19292 4/0/0 AAAA 2606:50c0:8003::153,  
AAAA 2606:50c0:8000::153, AAAA 2606:50c0:8001::153, AAAA 2606:50c0:8002::153 (155)
```

Exemplo de Uso

```
$ tcpdump -i eth1 -n -e -vvv port 67 or port 68
tcpdump: listening on eth1, link-type EN10MB (Ethernet), snapshot length 262144 bytes
00:08:07.677586 aa:36:11:5d:0d:22 > da:1e:92:0a:d5:70, ethertype IPv4 (0x0800), length 342: (tos 0x0, ttl
64, id 54131, offset 0, flags [DF], proto UDP (17), length 328)
    192.168.50.3.67 > 192.168.50.2.67: [udp sum ok] BOOTP/DHCP, Request from a2:35:db:5c:58:54, length 300,
hops 1, xid 0x7db0524e, Flags [none] (0x0000)
    Gateway-IP 10.10.0.3
    Client-Ethernet-Address a2:35:db:5c:58:54
    Vendor-rfc1048 Extensions
    Magic Cookie 0x63825363
    DHCP-Message (53), length 1: Discover
    Hostname (12), length 12: "dhcp-client1"
    Parameter-Request (55), length 13:
        Subnet-Mask (1), BR (28), Time-Zone (2), Default-Gateway (3)
        Domain-Name (15), Domain-Name-Server (6), Unknown (119), Hostname (12)
        Netbios-Name-Server (44), Netbios-Scope (47), MTU (26), Classless-Static-Route (121)
        NTP (42)
    END (255), length 0
    PAD (0), length 0, occurs 27
00:08:08.679547 da:1e:92:0a:d5:70 > aa:36:11:5d:0d:22, ethertype IPv4 (0x0800), length 342: (tos 0x0, ttl
64, id 52128, offset 0, flags [DF], proto UDP (17), length 328)
    192.168.50.2.67 > 10.10.0.3.67: [bad udp cksum 0xfdfc -> 0x8f65!] BOOTP/DHCP, Reply, length 300, hops
1, xid 0x7db0524e, Flags [none] (0x0000)
```

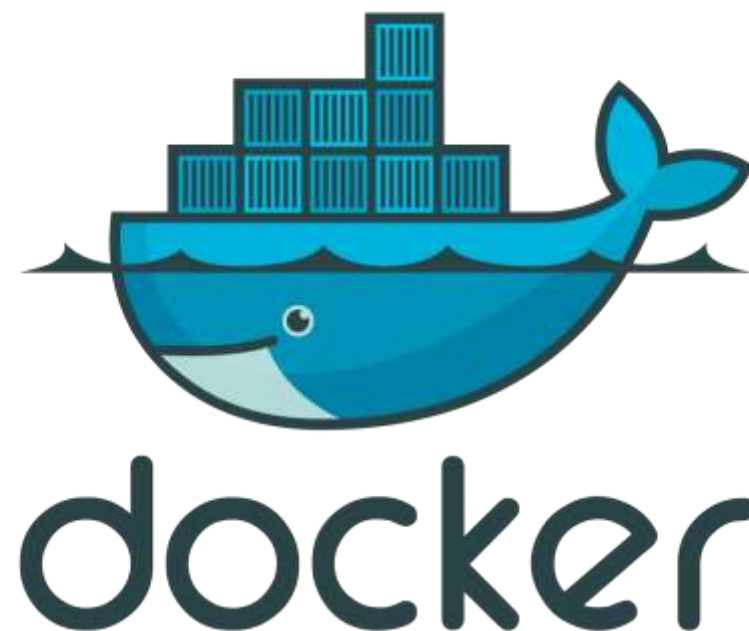
Containers

Docker

- Images
- Containers
- Docker
- Docker Compose
- Dockerfile

Container

- Empacota o código da aplicação e todas as suas dependências em uma **única unidade**.
- Elimina o famoso problema do "**na minha máquina funciona**".
- Garante comportamento idêntico no ambiente **local, staging e produção**.
- **Início ultrarrápido** em segundos e uso altamente eficiente de recursos.



Arquitetura Básica

Docker Client

- Interface de linha de comando (CLI) usada pelo desenvolvedor para enviar ordens ao sistema.

Docker Host (Daemon)

- O processo de segundo plano (dockerd) que constrói, executa e gerencia imagens e containers.

Docker Registry

- Repositório de imagens (ex: Docker Hub) onde imagens públicas e privadas são armazenadas.

Containers vs Máquinas Virtuais

- **Máquina virtual:** virtualiza um computador e executa um sistema operacional convidado
- **Container:** isola processos usando recursos do kernel do sistema hospedeiro
- Containers normalmente são mais leves e iniciam rapidamente
- Para Redes, Docker permite criar vários hosts e servidores no mesmo computador

Containers não são máquinas virtuais completas

Containers vs Máquinas Virtuais

Vantagens dos Containers:

- Kernel Compartilhado
 - O Docker compartilha o Kernel do sistema operacional hospedeiro, dispensando um SO completo por aplicação
- Leveza
 - Ocupam apenas megabytes de espaço e inicializam instantaneamente (em milissegundos)
- Alto Desempenho
 - Executam com sobrecarga (overhead) quase nula no hardware

O problema que o Docker resolve

- Aplicações dependem de código, bibliotecas, versões e configuração
- Instalar tudo diretamente no computador pode gerar conflitos
- Máquinas virtuais resolvem parte do problema, mas incluem um sistema operacional completo
- Docker empacota a aplicação e suas dependências em imagens e executa containers

Resultado: ambientes mais reproduzíveis e isolados

Imagens e Containers

Imagem Docker

- Modelo estático, somente leitura, contendo o código, bibliotecas e configurações necessárias
- Funciona como uma "planta de construção" ou classe.

Container Docker

- Instância em execução criada a partir de uma Imagem
- Possui uma camada editável e isolada
- É o objeto em execução na prática.

Primeiros Comandos

Baixar e Executar

- `docker run -d -p 8080:80 nginx` (baixa e executa o Nginx em background)

Listar Containers

- `docker ps` para ativos e `docker ps -a` para ver todos

Parar e Remover

- `docker stop <id>`
- `docker rm <id>` para limpar o ambiente

Acessar o Terminal

- `docker exec -it <id> bash` para interagir dentro do container.

Examinar logs

- `docker logs <id>` e `docker logs -f <id>` para análise contínua

Funcionamento

- O Docker procura a imagem localmente
- Se necessário, obtém a imagem de um registry
- Cria um container a partir da imagem
- Executa o processo definido pela imagem
- O container termina quando seu processo principal termina

```
$ sudo docker run hello-world
```

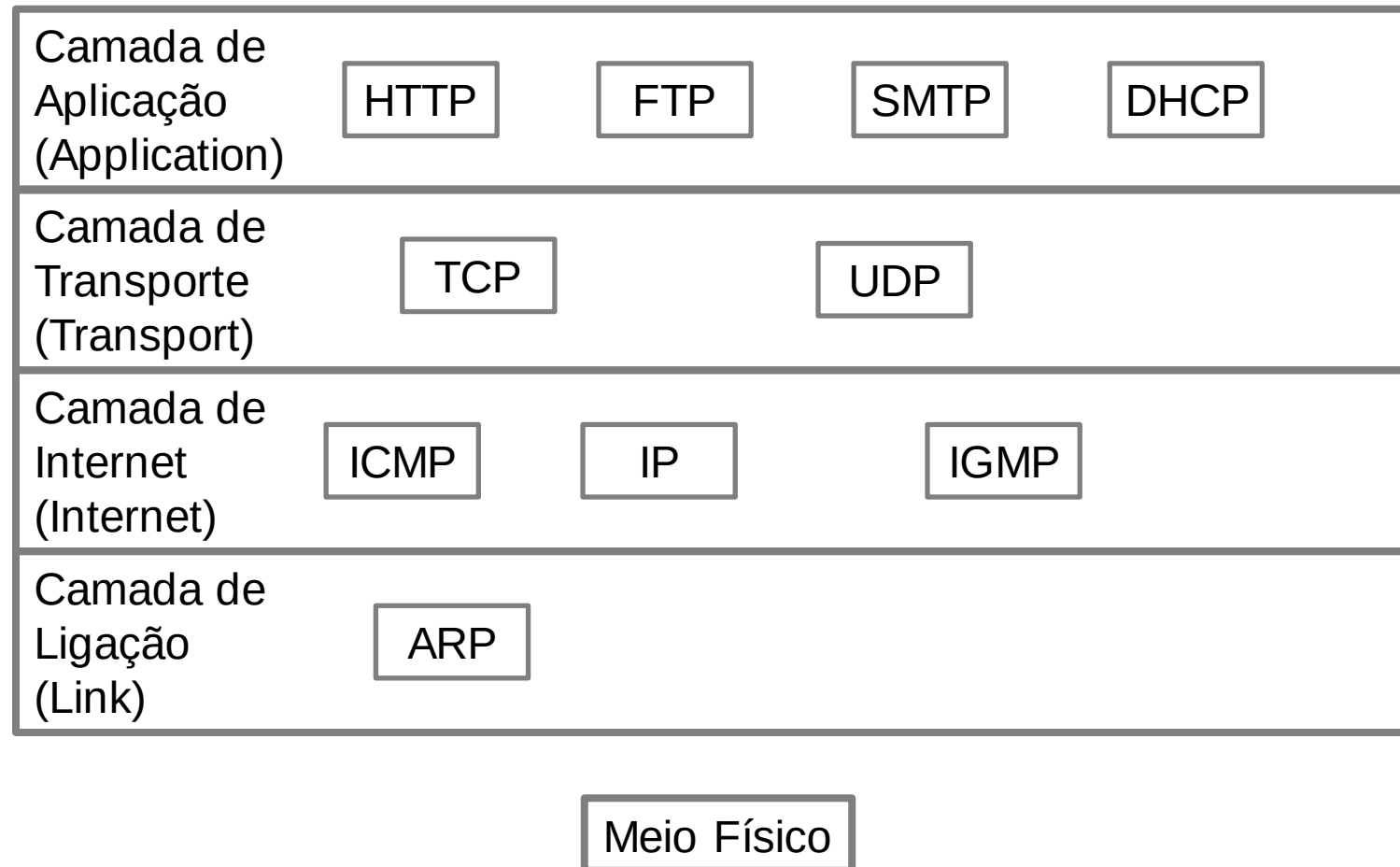
Protocolo ARP – (Address Resolution Protocol)

Comunicação na Camada de Rede

Como o endereço MAC é utilizado

- O frame Ethernet contém o endereço MAC da origem e do destino.
- A placa de rede verifica o MAC de destino de cada frame recebido.
 - MAC de destino = próprio MAC → aceita o frame.
 - MAC de destino $\neq$ próprio MAC → descarta o frame.
- O frame aceito é desencapsulado, e seu *payload* é entregue às camadas superiores.

Overview



Endereçamento

- Endereço IP (Lógico - Camada de Rede)
 - Usado para roteamento através de diferentes redes.
 - Exemplo: 192.168.1.50
- Endereço MAC (Físico - Camada de Enlace):
 - Gravado na placa de rede (NIC), usado para entregar quadros no mesmo segmento local.
 - Exemplo: 00:1A:2B:3C:4D:5E
- A Pergunta Fundamental:
 - "Se meu computador sabe o IP do destino, como ele descobre o endereço MAC da placa de rede para onde enviar o pacote localmente?"

ARP

- **ARP**

- *Address Resolution Protocol*
- RFC 826

- **Objetivo**

- Mapear um **Endereço IP conhecido** em um **Endereço MAC desconhecido**

- **Atuação**

- Opera entre a Camada 2 (Enlace) e a Camada 3 (Rede) do modelo OSI

- **Cenário típico**

- Comunicação em uma rede local (LAN)

Funcionamento



Host A: 192.168.1.10/24
(quer enviar para host B)

Host B: 192.168.1.20/24
(lê e responde)

Consulta à Tabela ARP

- O Host A verifica se já possui o MAC do Host B em seu cache local.

ARP Request (Broadcast)

- Se não tiver, envia um quadro para **FF:FF:FF:FF:FF:FF**
- *"Quem tem o IP 192.168.1.20? Responda para 192.168.1.10!"*

ARP Reply (Unicast)

- O Host B reconhece seu IP e responde diretamente ao Host A
- *"Eu tenho o IP 192.168.1.20! Meu MAC é 00:11:22:33:44:55."*

Atualização da Tabela

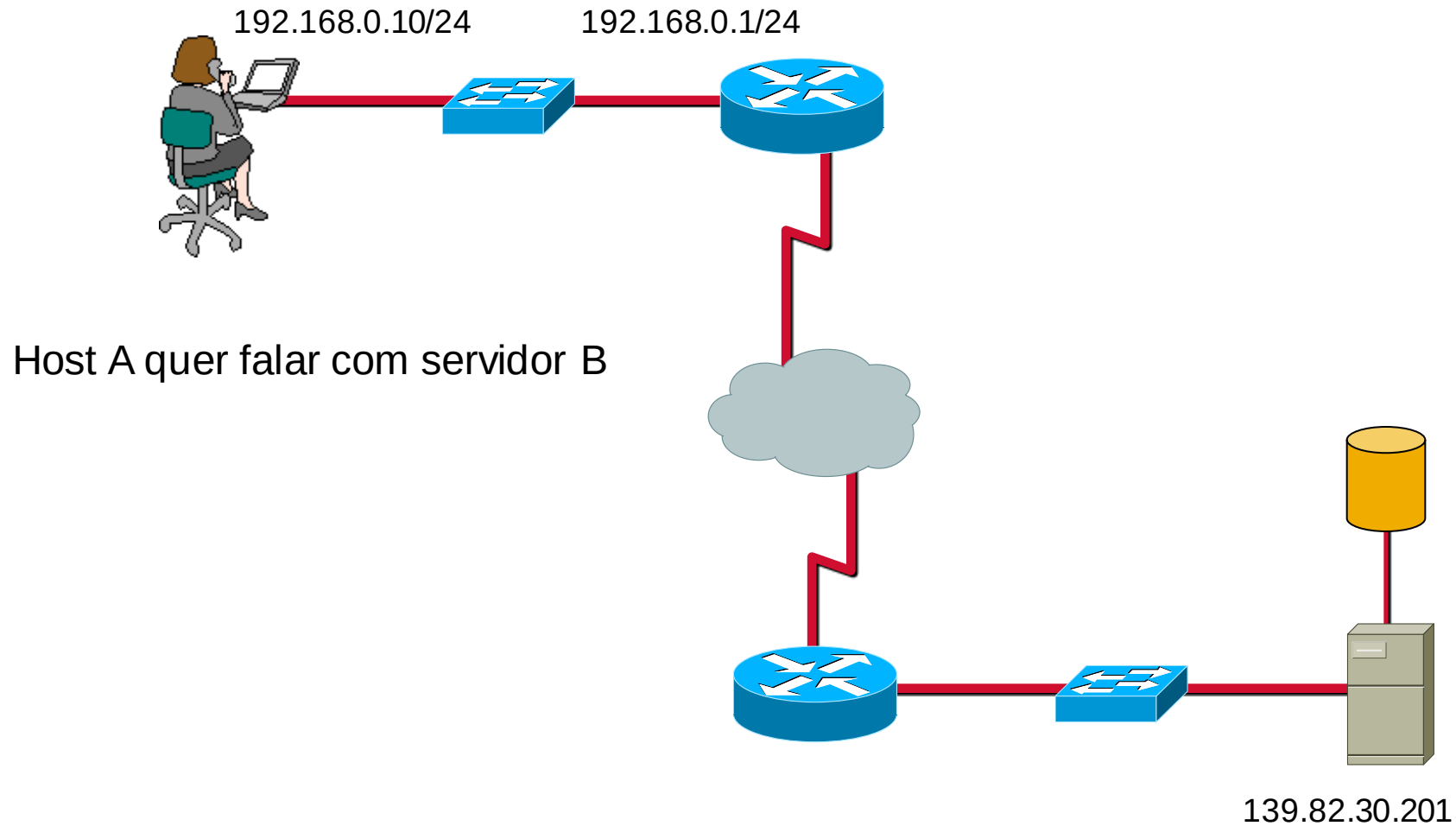
- O Host A guarda essa informação no cache e envia os dados.

Tabela ARP (ARP Cache)

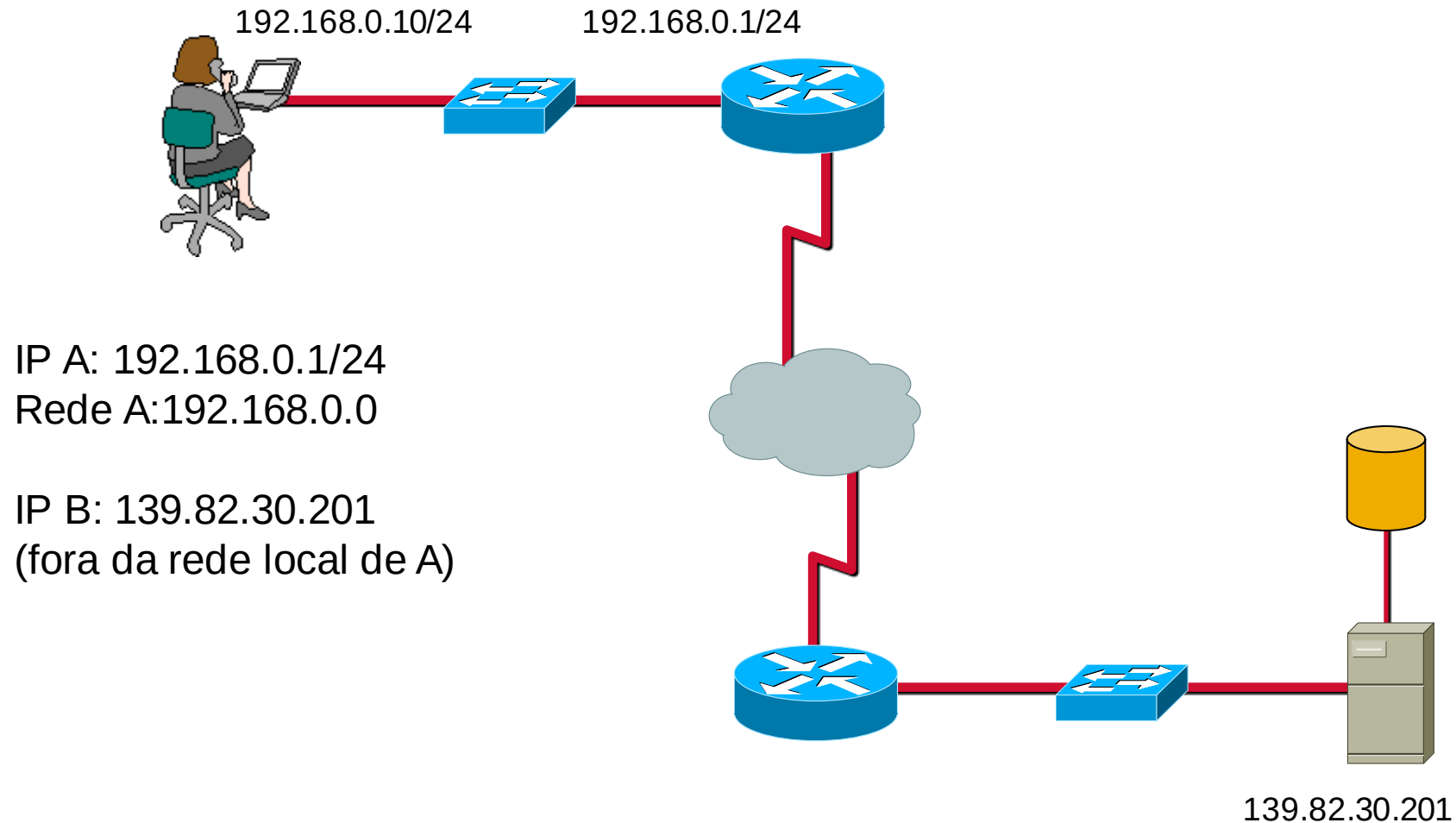
- Para evitar broadcasts constantes, o sistema salva os mapeamentos temporariamente
- Tipos de Entradas:
 - **Dinâmicas:** Criadas automaticamente e expiram após um tempo (ex: 2 a 20 min)
 - **Estáticas:** Inseridas manualmente por administradores para segurança

```
$ arp -a
? (10.0.0.2) at f4:ec:38:fd:c4:96 [ether] on wlo1
? (10.0.0.46) at 48:87:b8:b1:be:5d [ether] on enp2s0
dns.google (8.8.4.4) at <incomplete> on wlo1
dns.google (8.8.8.8) at <incomplete> on wlo1
? (10.0.0.1) at c8:5d:38:a2:df:dc [ether] on wlo1
connectivity-check-ubuntu-com-7.ps6.canonical.com (91.189.91.63) at <incomplete> on enp2s0
dns.google (8.8.4.4) at <incomplete> on enp2s0
dns.google (8.8.8.8) at <incomplete> on enp2s0
ubuntu-content-cache-2.ps6.canonical.com (91.189.91.97) at <incomplete> on wlo1
? (10.0.0.1) at c8:5d:38:a2:df:dc [ether] on enp2s0
? (10.0.0.2) at f4:ec:38:fd:c4:96 [ether] on enp2s0
```

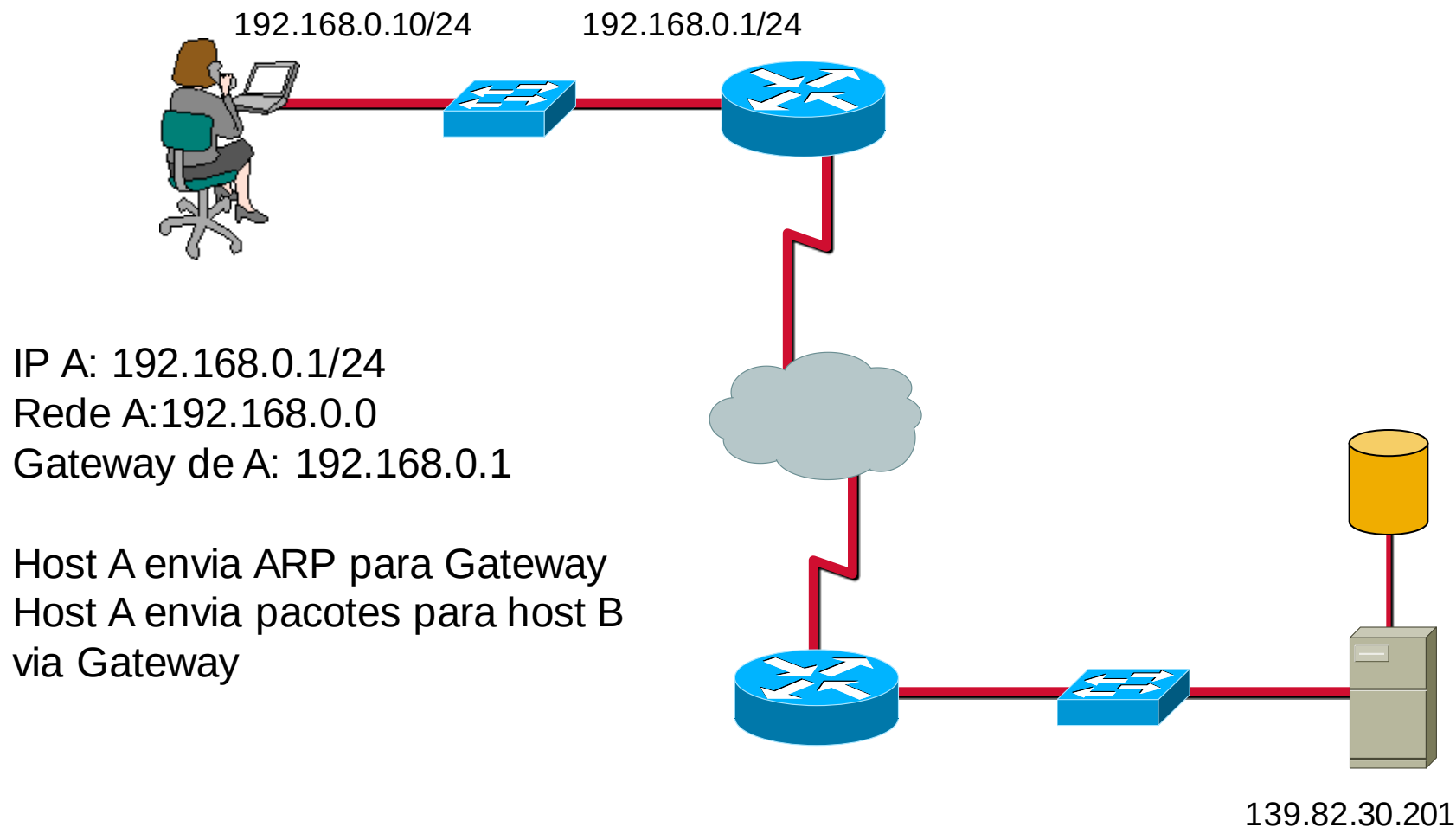
Comunicação entre dois hosts



Comunicação entre dois hosts



Comunicação entre dois hosts



Tópico Avançado - Vulnerabilidades do ARP

- ARP Spoofing / ARP Poisoning:
 - Ataque do tipo Man-in-the-Middle (MitM).
 - Um atacante envia respostas ARP falsas associando o IP do Gateway ao seu próprio MAC.
 - Permite interceptar, modificar ou bloquear o tráfego da rede.
- Mitigação:
 - Uso de entradas ARP estáticas em ambientes críticos.
 - Switches com DAI (Dynamic ARP Inspection).

Arquitetura TCP/IP

Protocolos de transporte

- TCP
- UDP

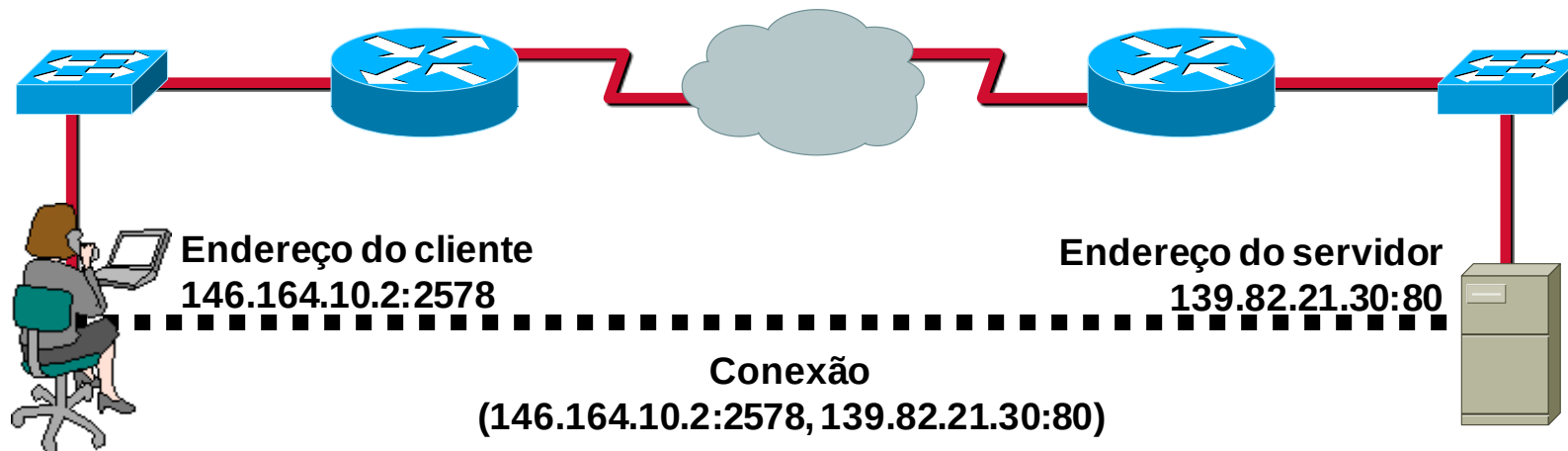
TCP e UDP

- TCP e UDP pertencem à camada de transporte
- Ambos recebem dados das aplicações e os entregam ao IP
- A principal diferença está nos serviços oferecidos às aplicações



Portas

- Um endereço IP identifica um host
- Uma porta identifica um *endpoint* de transporte associado a uma aplicação
- Assim, uma comunicação usa endereços IP e números de porta
- Exemplos:
 - HTTP tradicional usa TCP/80
 - DNS pode usar UDP/53 ou TCP/53.



Multiplexação e demultiplexação

- Um host pode executar várias aplicações simultaneamente
- TCP e UDP usam portas para separar as comunicações
- **Multiplexação:** várias aplicações usam a camada de transporte
- **Demultiplexação:** os dados recebidos são entregues à aplicação correta

UDP — User Datagram Protocol

- UDP é simples e orientado a datagramas
- Não estabelece uma conexão antes de enviar dados
- Não garante entrega, ordem ou retransmissão
- Não implementa controle de congestionamento como o TCP
- Tem pouco overhead e baixa complexidade.

Cabeçalho UDP

- O cabeçalho UDP tem apenas 8 bytes
 - Porta de origem^{1,2}
 - Porta de destino¹
 - Comprimento
 - Checksum

Offset	Octeto
0	Porta origem
1	
2	Porta destino
3	
4	Comprimento
5	
6	Checksum
7	

1 – Opcional em IPv4 (0, se não for usado)

2 – Opcional em IPv6 (0, se não for usado)

Quando usar UDP

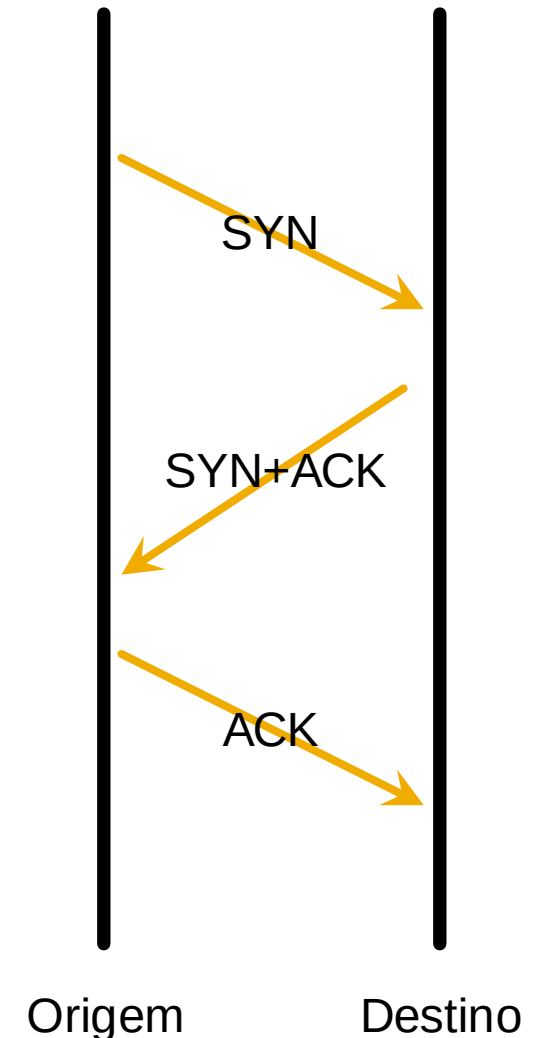
- Quando a aplicação pode tolerar perda de alguns dados
- Quando baixa latência é importante
- Quando a própria aplicação implementa mecanismos de confiabilidade
 - Exemplos: DNS, voz/vídeo em tempo real
- UDP não significa automaticamente “mais rápido” – depende da aplicação e da rede

TCP – Transmission Control Protocol

- TCP é orientado a conexão
- Oferece um fluxo confiável de bytes entre os extremos
- Entrega os dados em ordem à aplicação
- Usa números de sequência, ACKs e retransmissões
- Também possui controle de fluxo e congestionamento
- Janelamento

Estabelecimento da conexão TCP

- Antes de transportar dados, TCP estabelece uma conexão
- O processo é conhecido como **three-way handshake**
 1. Cliente envia **SYN**
 2. Servidor responde **SYN + ACK**
 3. Cliente envia **ACK**
- Depois disso, os dados podem ser transmitidos



Números de Sequência e ACK

- TCP numera os bytes do fluxo
- O receptor informa quais dados recebeu através de ACKs
- Se dados forem perdidos, TCP pode retransmiti-los
- Isso permite que a aplicação receba um fluxo ordenado e confiável

```
3 34398 → 5000 [PSH, ACK] Seq=1 Ack=1 Win=64512 Len=37 TSval=4
6 5000 → 34398 [ACK] Seq=1 Ack=38 Win=65536 Len=0 TSval=4
3 5000 → 34398 [PSH, ACK] Seq=1 Ack=38 Win=65536 Len=37 TSval=4
6 34398 → 5000 [ACK] Seq=38 Ack=38 Win=64512 Len=0 TSval=4
7 34398 → 5000 [PSH, ACK] Seq=38 Ack=38 Win=64512 Len=21 TSval=4
6 5000 → 34398 [ACK] Seq=38 Ack=59 Win=65536 Len=0 TSval=4
```

Controle de Fluxo

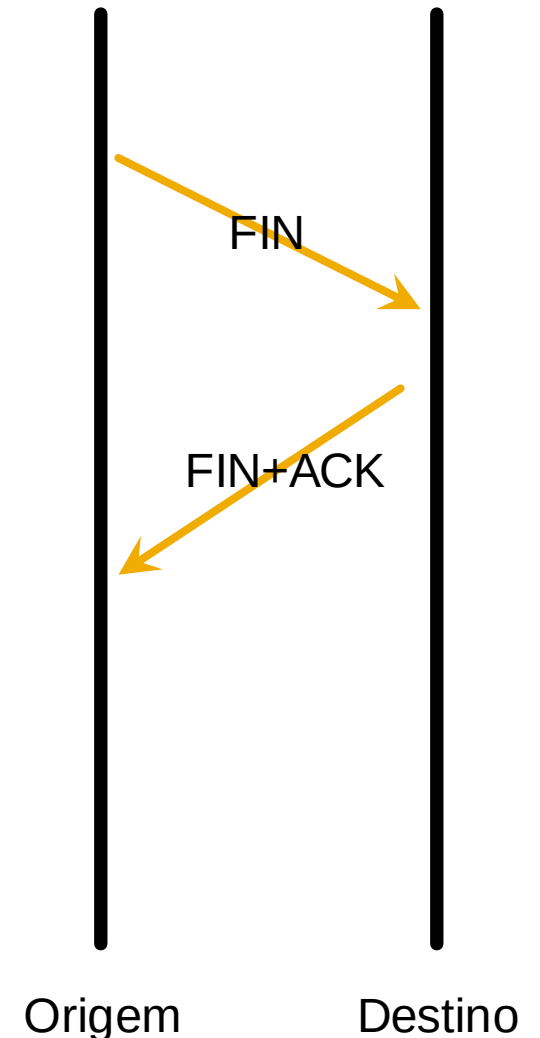
- O receptor possui capacidade limitada de armazenamento
- TCP usa uma janela de recepção (receive window)
- O receptor informa quanto espaço ainda consegue receber
- O emissor limita os dados em trânsito de acordo com essa informação e qualidade da transmissão
- Objetivos
 - Evitar sobrecarregar o receptor
 - Aumento de *throughput*
 - Diminuição de reenvios

Controle de Congestionamento

- Uma rede pode ficar congestionada quando há tráfego excessivo
- TCP ajusta a quantidade de dados em trânsito de acordo com as condições percebidas
- Existem mecanismos como **slow start** e **congestion avoidance**
- Controle de congestionamento é diferente de controle de fluxo

Encerramento da conexão TCP

- Os dois sentidos do fluxo podem ser encerrados separadamente
- **FIN** e **ACK** aparecem normalmente durante o encerramento
- **RST** pode indicar uma interrupção abrupta da conexão
- O fechamento também pode ser observado no Wireshark (veremos em breve)



Arquitetura TCP/IP

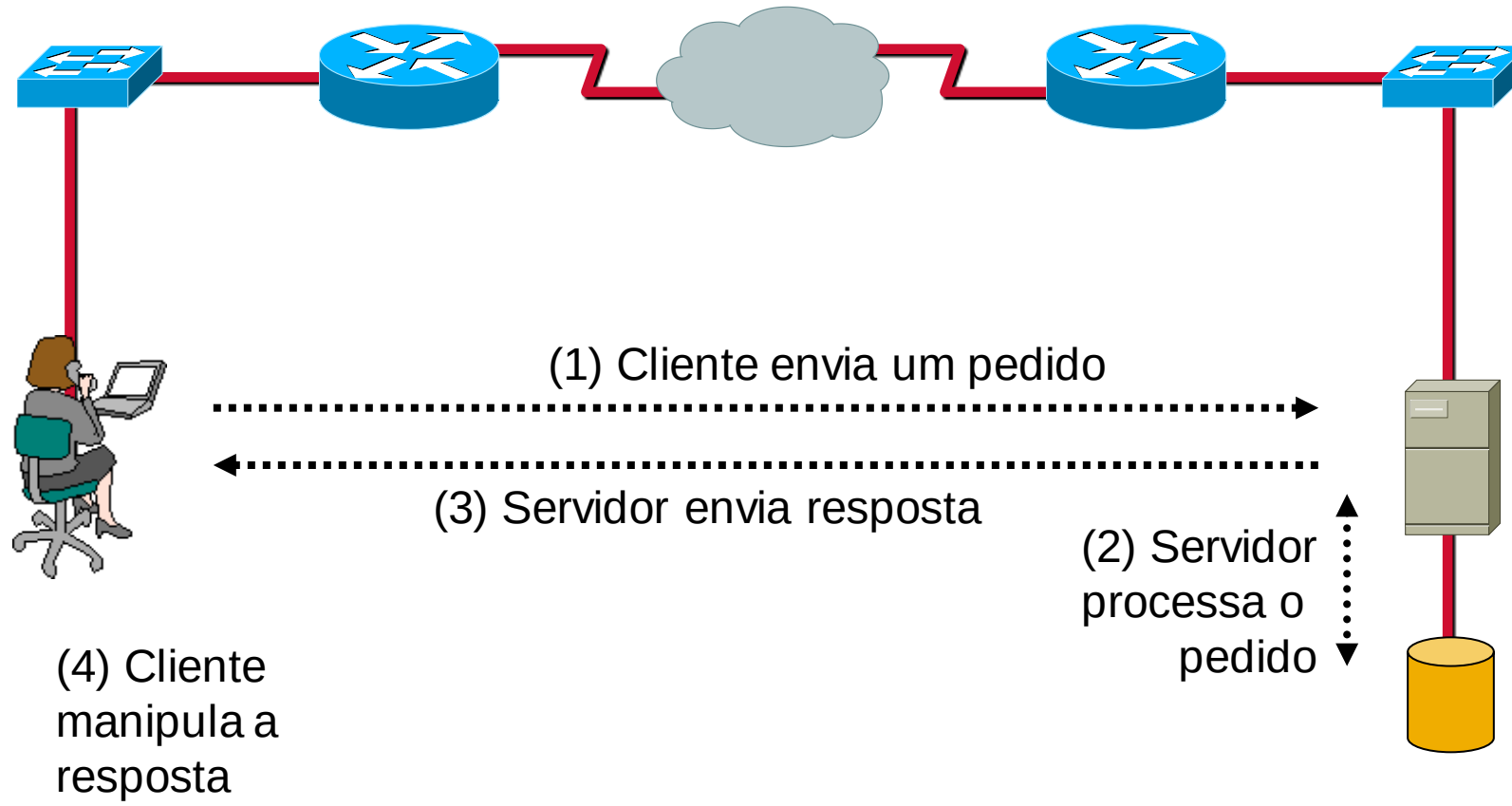
Protocolos de transporte

- Socket

Transações cliente-servidor

- Modelo:
 - 1 processo servidor e um ou mais processos clientes
 - Servidores gerenciam alguns recursos
 - Servidores fornecem serviços manipulando dados para o cliente
- Obs.: clientes e servidores são processos rodando em computadores (hosts). Podem ser no mesmo ou em diferentes hosts.

Um pouco de TCP/IP



Visão do programador

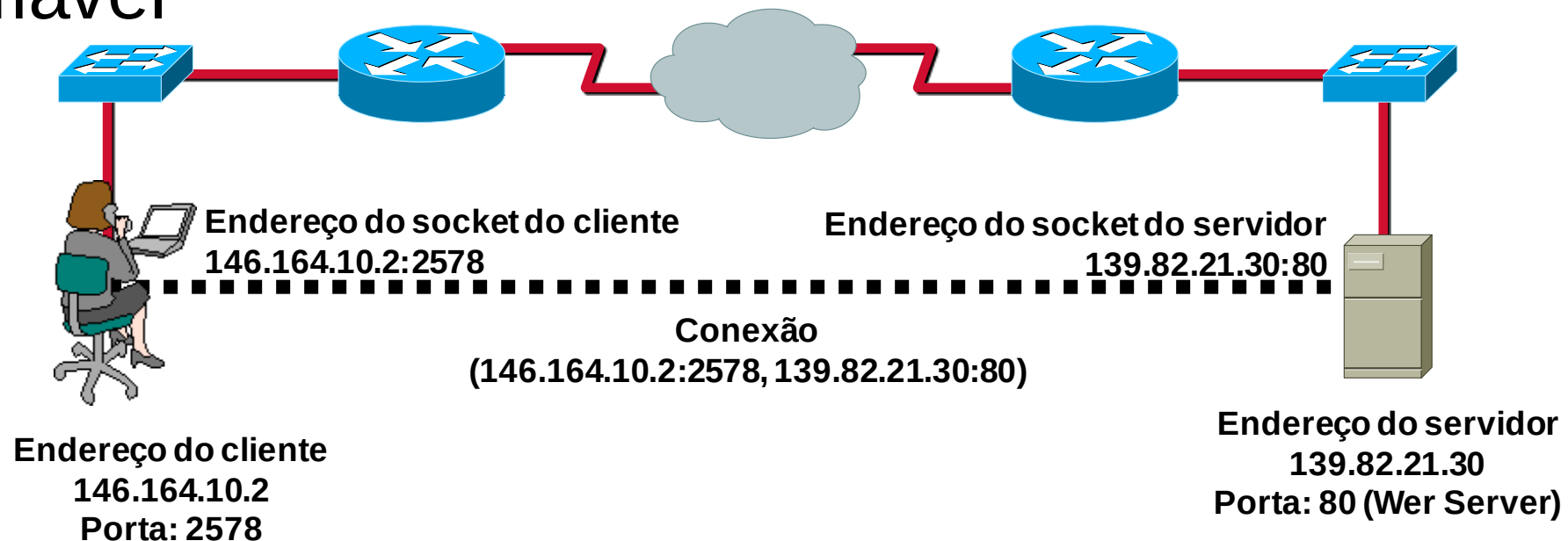
- Host são mapeados através de um endereço IP de 32 bits (IPv4) ou 128 bits (IPv6)
 - 146.164.2.42
 - 139.82.30.201
- Host conectados em uma rede (TCP/IP) possuem endereços IP's
 - 146.164.2.3 mapeia www.nce.ufrj.br
 - 139.82.30.201 mapeia www.puc-rio.br
 - O mapeamento é, geralmente, do tipo many-to-many
- Processos em um host na Internet podem se comunicar com outro processo através de uma conexão

Domain Naming System (DNS)

- A Internet mantém o mapeamento entre o endereço IP e nomes de domínio em um grande banco de dados chamado DNS distribuído pelo mundo
- Conceitualmente, programadores podem ver o banco de dados DNS como uma coleção de milhões de entradas

Conexões Internet

- Clientes e servidores se comunicam enviando fluxos de bytes através de conexões
- Conexões são ponto-a-ponto, full-duplex (bidirecional) e confiável



Clientes

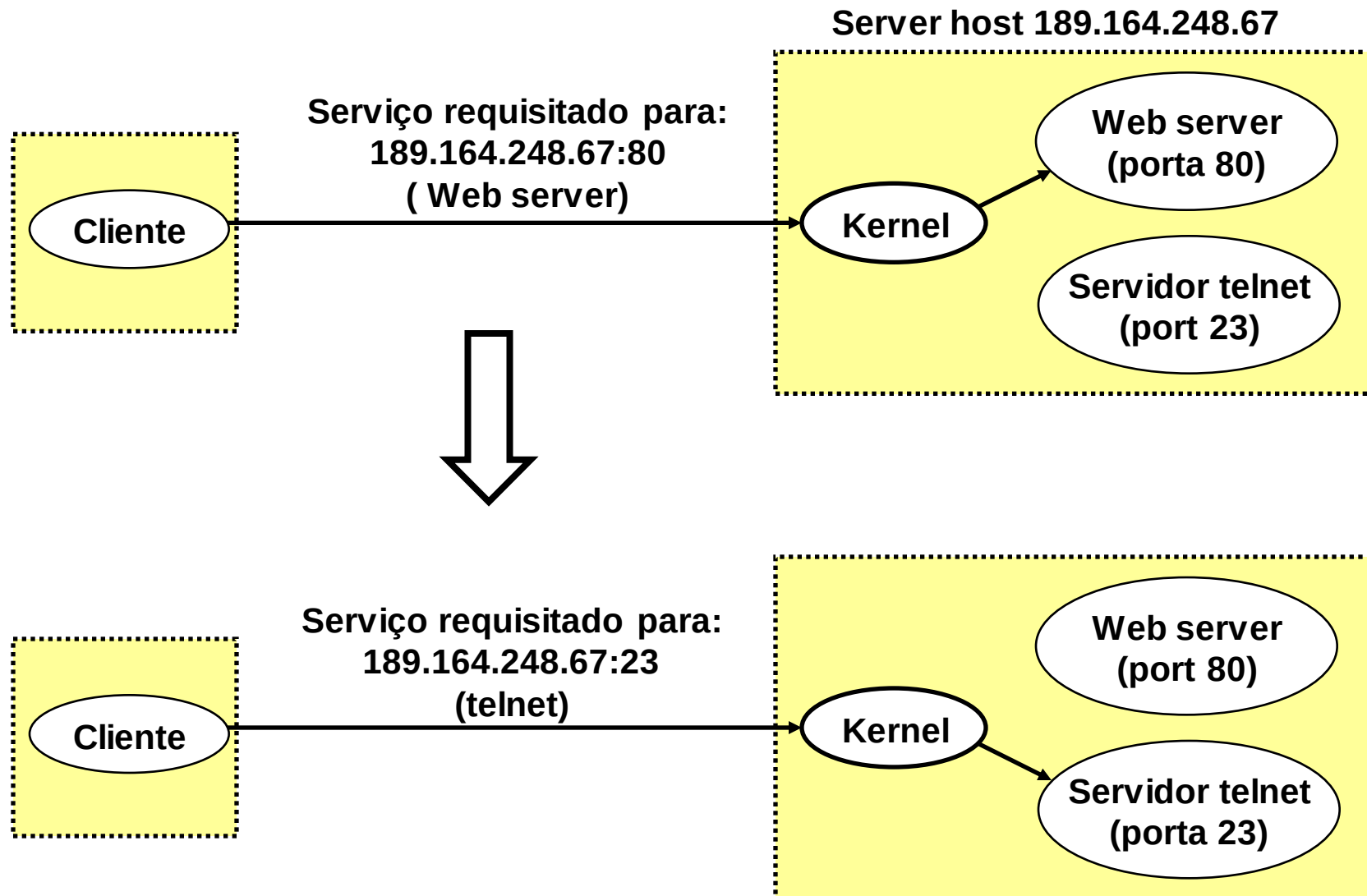
- Exemplos de programas clientes

- Browsers
- ftp
- telnet
- ssh

- Como um cliente localiza um servidor?

- Através do endereço IP do servidor
- Através do nome do servidor, mas tarde convertido para endereço IP
- Em conjunto com o número da porta do servidor
- Exemplo de portas conhecidas (well-known)
 - 20: dados ftp
 - 21: comandos ftp
 - 23: telnet
 - 80: servidor web

Usando portas para identificar um serviço



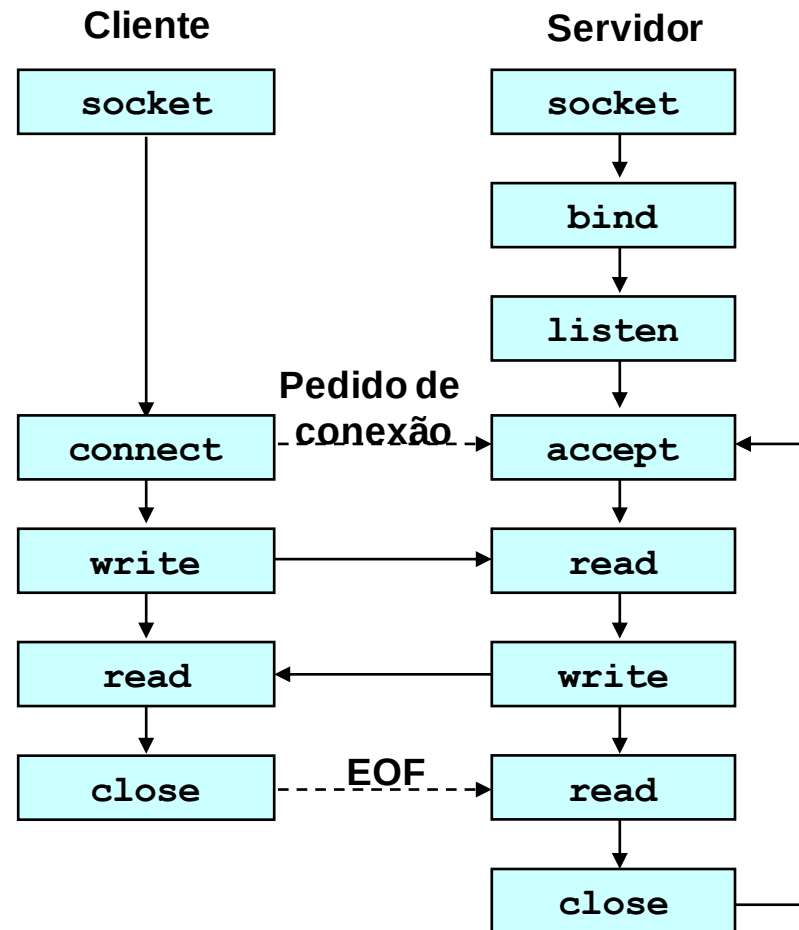
Serviços

- Servidores (software) são processos que executam durante muito tempo
 - Conhecidos como *daemons*
 - Criados (tipicamente) durante o *boot* pelo processo `init` (processo com PID #1)
 - Também podem ser iniciados por `inetd` em resposta a uma conexão em determinada porta
- Cada servidor espera pela chegada de pedidos em portas conhecidas (portas baixas)
- Máquinas que executam um processo servidor também são conhecidas como servidoras

Interface baseada em sockets

- Criada nos anos 80 como parte da distribuição original de Berkeley
- Projetada para “qualquer” protocolo
- Fornece ao usuário uma interface para a rede
- Base para todas as aplicações Internet
- Modelo cliente-servidor

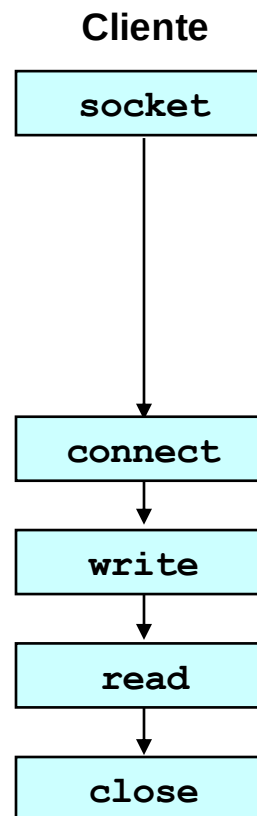
Socket



Socket

- Como um socket é visto:
 - Pelo kernel, socket é um ponto de comunicação
 - Para a aplicação, socket é um descritor de arquivo, no qual a aplicação pode ler e/ou escrever
 - (lembre-se que para o Unix, todas as operações de I/O são mapeadas como um arquivo)
- Clientes e servidores se comunicam através de leituras e escritas nos descritores do socket
- A grande diferença entre um arquivo de verdade e um socket está na forma como o arquivo é aberto

Exemplo: um cliente de chat



Exemplo: um cliente de chat

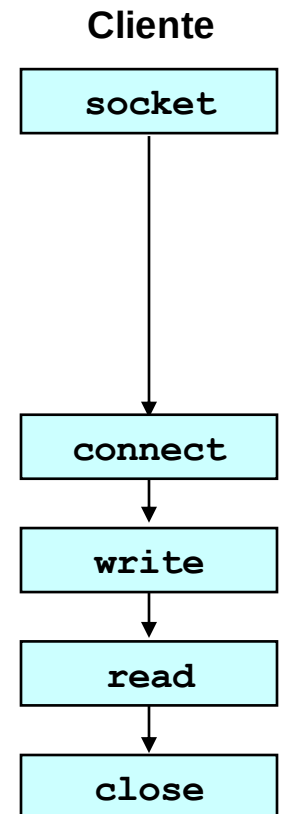
socket/client/client.py

```
'''
Cliente de socket simples.
Este cliente deve ser executado em uma rede Docker.

:Author: Alexandre Meslin
:Date: 2026-08-29
'''

# -----
# Módulos
# -----
import socket

# -----
# Constantes
# -----
SERVER_HOST = "server"
SERVER_PORT = 5000
```



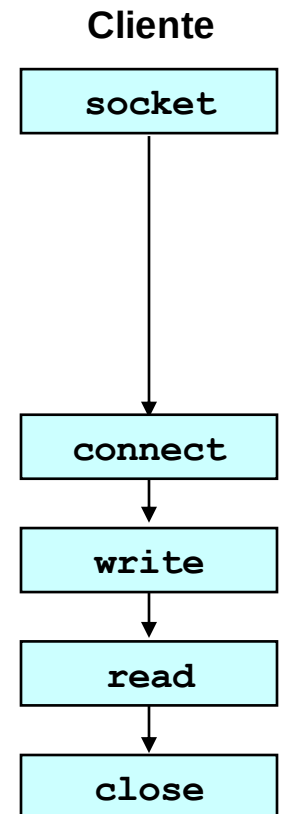
Exemplo: um cliente de chat

socket/client/client.py

```
def create_socket():  
    '''  
    Cria um socket TCP/IP para o cliente.  
    Retorna o socket criado.  
  
    :return: socket do cliente  
    '''  
    client_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)  
    return client_socket
```

AF_INET: IPv4
AF_INET6: IPv6

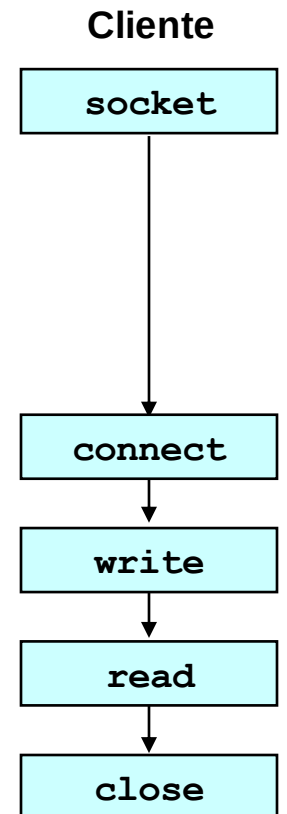
SOCK_STREAM: TCP
SOCK_DGRAM: UDP



Exemplo: um cliente de chat

socket/client/client.py

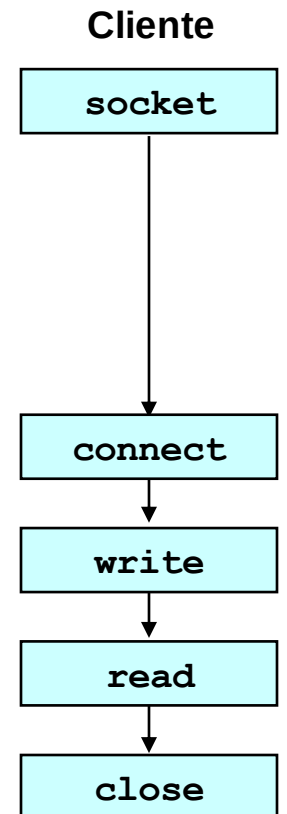
```
def connect(client_socket):  
    '''  
    Conecta o socket do cliente ao servidor.  
  
    :param client_socket: socket do cliente  
    :return: None  
    '''  
    print(f"Conectando ao servidor {SERVER_HOST}:{SERVER_PORT}...")  
    client_socket.connect((SERVER_HOST, SERVER_PORT))  
    print("Conectado ao servidor.")  
    return
```



Exemplo: um cliente de chat

socket/client/client.py

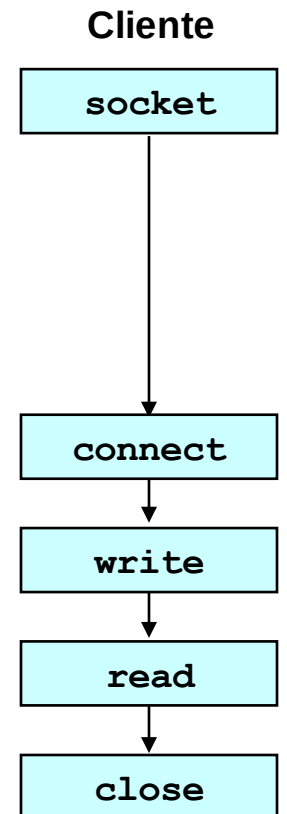
```
def sendall(client_socket):  
    ...  
    Envia mensagens para o servidor e recebe respostas.  
  
    :param client_socket: socket do cliente  
    :return: None  
    ...  
while True:  
    mensagem = input("Cliente: ")  
    if mensagem.lower() == "sair":  
        break  
    client_socket.sendall(mensagem.encode())  
    print("Aguardando resposta do servidor... ", end="", flush=True)  
    data = client_socket.recv(1024)  
    print(f"\nServidor: {data.decode()}")  
return
```



Exemplo: um cliente de chat

socket/client/client.py

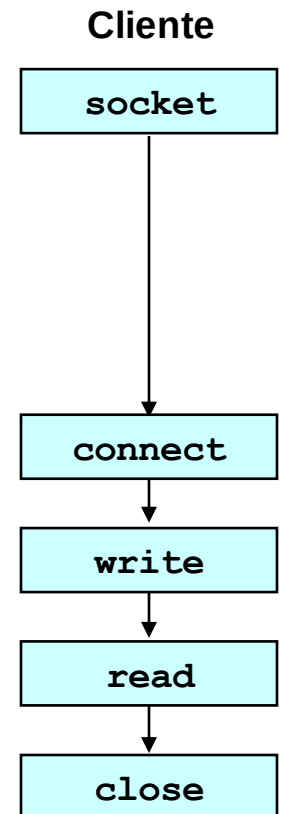
```
def close(client_socket):  
    '''  
    Fecha o socket do cliente.  
  
    :param client_socket: socket do cliente  
    :return: None  
    '''  
    client_socket.close()  
    print("Conexão encerrada.")  
    return
```



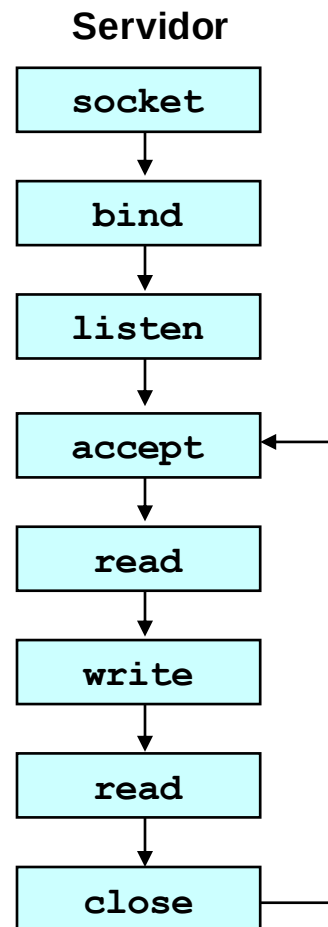
Exemplo: um cliente de chat

socket/client/client.py

```
def main():  
    client_socket = create_socket()  
    connect(client_socket)  
    sendall(client_socket)  
    close(client_socket)  
    return  
  
if __name__ == "__main__":  
    main()
```



Exemplo: um servidor de chat



Exemplo: um servidor de chat

Socket/server/server.py

```
'''
Servidor de socket simples.
Este servidor deve ser executado em uma rede Docker.
```

```
:Author: Alexandre Meslin
:Date: 2026-08-29
'''
```

```
# -----
# Módulos
# -----
```

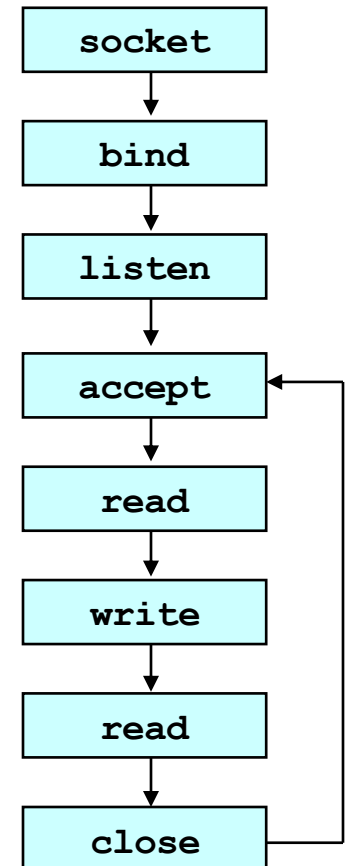
```
import socket
```

```
# -----
# Constantes
# -----
```

```
HOST = "0.0.0.0"
PORT = 5000
```

0.0.0.0: Qualquer interface

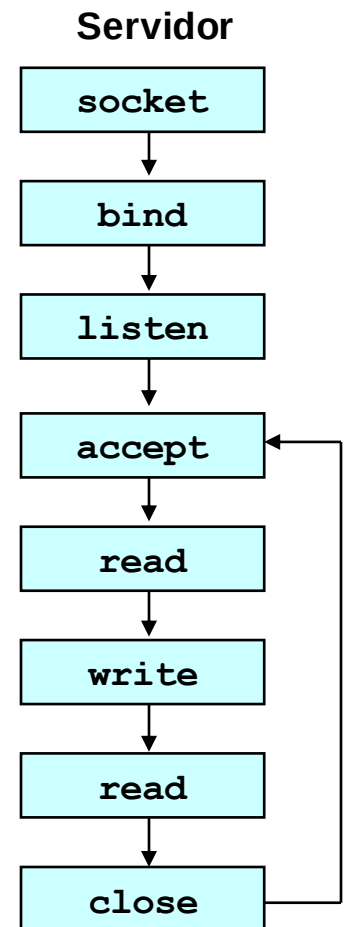
Servidor



Exemplo: um servidor de chat

Socket/server/server.py

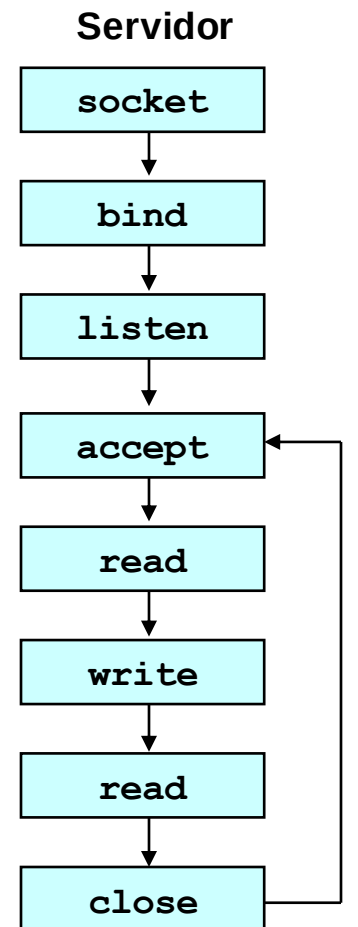
```
def create_socket():  
    '''  
    Cria um socket TCP/IP para o servidor.  
    Retorna o socket criado.  
  
    :return: socket do servidor  
    '''  
    server_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)  
    return server_socket
```



Exemplo: um servidor de chat

Socket/server/server.py

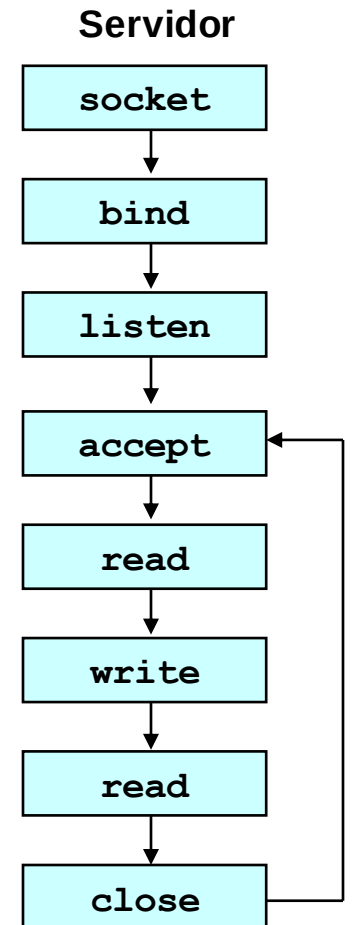
```
def bind(server_socket):  
    '''  
    Associa o socket do servidor a um endereço e porta específicos.  
  
    :param server_socket: socket do servidor  
    :return: None  
    '''  
    server_socket.bind((HOST, PORT))  
    return
```



Exemplo: um servidor de chat

Socket/server/server.py

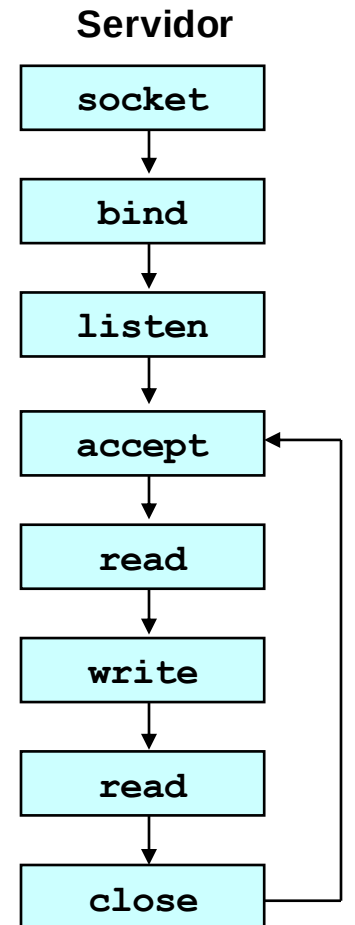
```
def listen(server_socket):  
    '''  
    Coloca o socket do servidor em modo de escuta para conexões.  
  
    :param server_socket: socket do servidor  
    :return: None  
    '''  
    server_socket.listen(1)  
    return
```



Exemplo: um servidor de chat

Socket/server/server.py

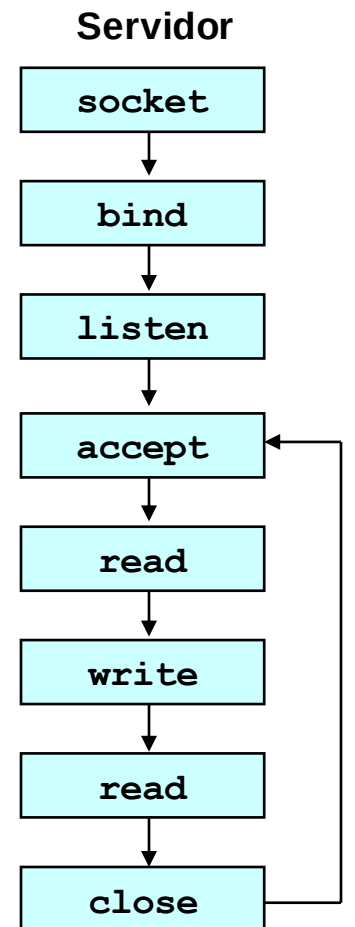
```
def accept(server_socket):  
    '''  
    Aceita uma conexão de um cliente.  
  
    :param server_socket: socket do servidor  
    :return: tupla com a conexão e o endereço do cliente  
    '''  
    print(f"Servidor aguardando conexão na porta {PORT}...")  
    conn, addr = server_socket.accept()  
    return conn, addr
```



Exemplo: um servidor de chat

Socket/server/server.py

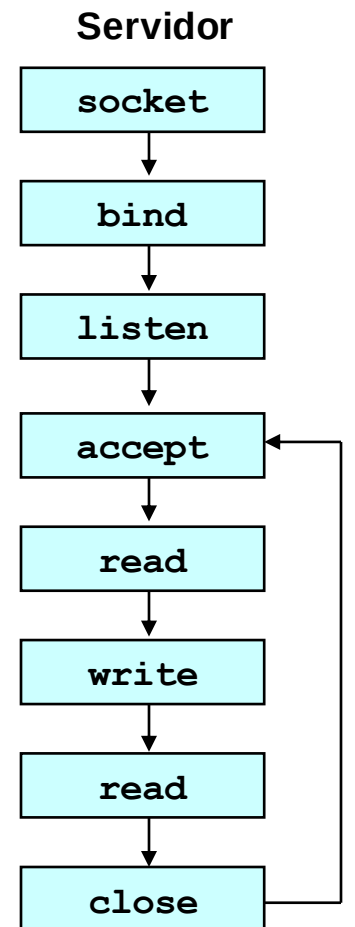
```
def recv(conn, addr):  
    '''  
    Recebe mensagens do cliente e envia respostas.  
  
    :param conn: conexão com o cliente  
    :param addr: endereço do cliente  
    :return: None  
    '''  
    print(f"Cliente conectado: {addr}")  
    while True:  
        print("Aguardando mensagem do cliente... ", end="", flush=True)  
        data = conn.recv(1024) # Recebe até 1024 bytes de dados do cliente  
        if not data:  
            break  
        mensagem = data.decode() # Decodifica os bytes recebidos em uma string  
        print(f"\nCliente: {mensagem}")  
        resposta = input("Servidor: ")  
        conn.sendall(resposta.encode()) # Envia a resposta de volta ao cliente  
    return
```



Exemplo: um servidor de chat

Socket/server/server.py

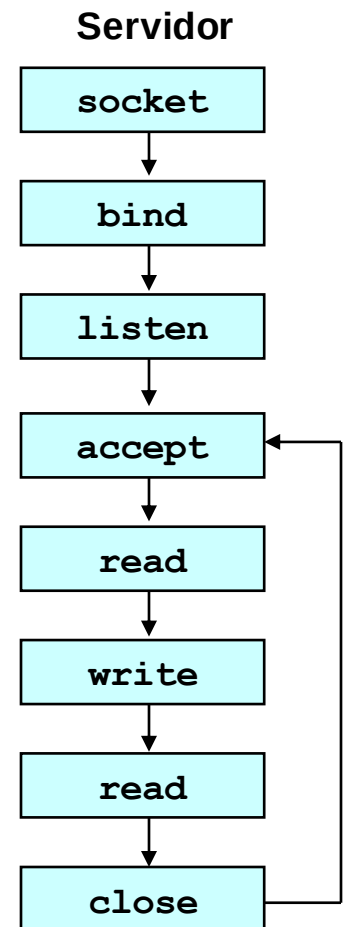
```
def close(conn, server_socket):  
    '''  
    Fecha a conexão com o cliente e o socket do servidor.  
  
    :param conn: conexão com o cliente  
    :param server_socket: socket do servidor  
    :return: None  
    '''  
    conn.close()  
    server_socket.close()  
    print("Conexão encerrada.")  
    return
```



Exemplo: um servidor de chat

Socket/server/server.py

```
def main():  
    '''  
    Função principal do servidor.  
    Cria o socket, associa, escuta, aceita conexões e recebe mensagens.  
    '''  
    server_socket = create_socket()  
    bind(server_socket)  
    listen(server_socket)  
    conn, addr = accept(server_socket)  
    recv(conn, addr)  
    close(conn, server_socket)  
    return  
  
if __name__ == "__main__":  
    main()
```



Testando o cliente-servidor

Servidor

```
root@server:/app# python3 server.py
Servidor aguardando conexão na
porta 5000...
Cliente conectado: ('172.18.0.3',
34398)
Aguardando mensagem do cliente...
Cliente: mensagem 1 do cliente para
o servidor
Servidor: mensagem 1 do servidor
para o cliente
Aguardando mensagem do cliente...
Cliente: mensagem 2 do cliente
Servidor: mensagem 2 do servidor
Aguardando mensagem do cliente...
Conexão encerrada.
```

Cliente

```
root@client:/app# python3 client.py
Conectando ao servidor
server:5000...
Conectado ao servidor.
Cliente: mensagem 1 do cliente para
o servidor
Aguardando resposta do servidor...
Servidor: mensagem 1 do servidor
para o cliente
Cliente: mensagem 2 do cliente
Aguardando resposta do servidor...
Servidor: mensagem 2 do servidor
Cliente: sair
Conexão encerrada.
```

Arquitetura TCP/IP

Protocolos de aplicação

- DHCP
- HTTP
- FTP
- SMTP
- POP3
- IMAP

DHCP

Dynamic Host Configuration Protocol

Como hosts obtêm automaticamente configurações de rede

O Problema

- Como gerenciar as configurações de rede TCP/IP em uma rede com diversos dispositivos?

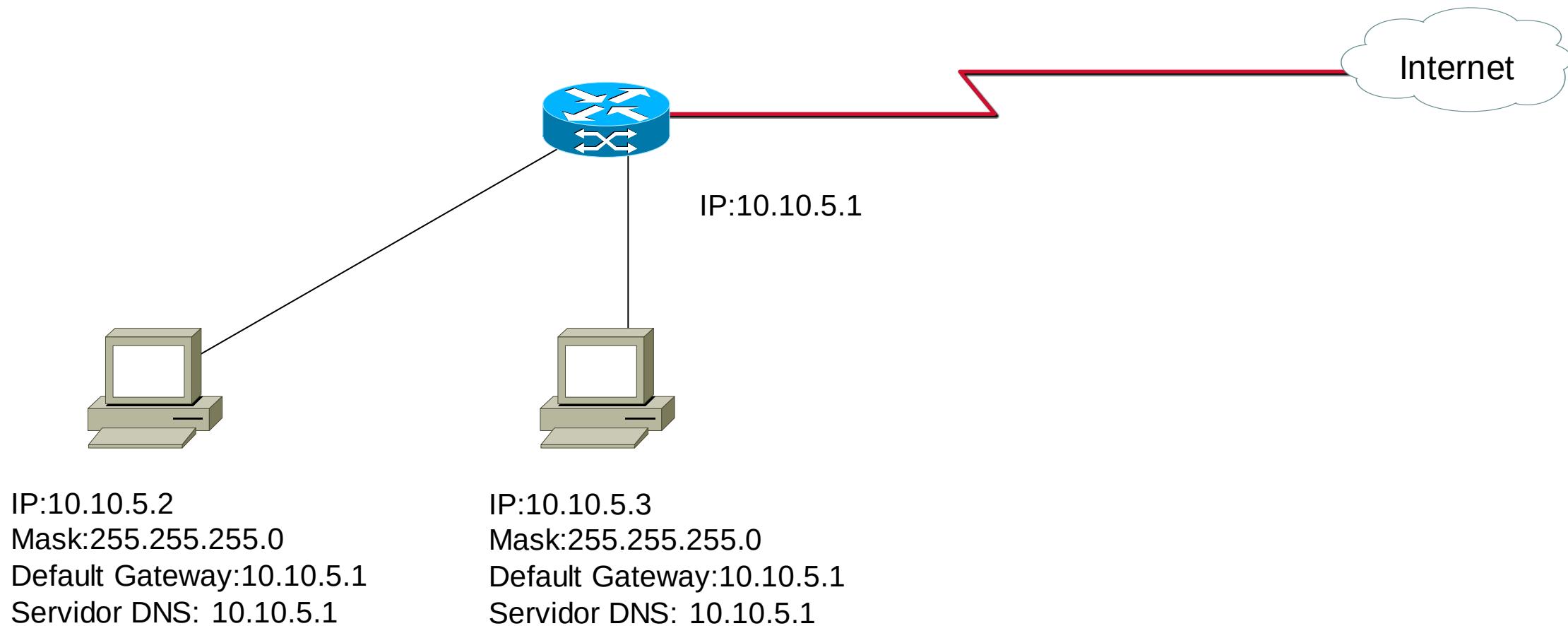
DHCP – Dynamic Host Configuration Protocol

- Automatiza a configuração de hosts em uma rede IP
- DHCPv4 é usado para IPv4
- DHCPv6 é usado para IPv6
- Pode fornecer IP, máscara/prefixo, gateway, servidor DNS e outros parâmetros
- Pode usar atribuição dinâmica, automática ou reservas

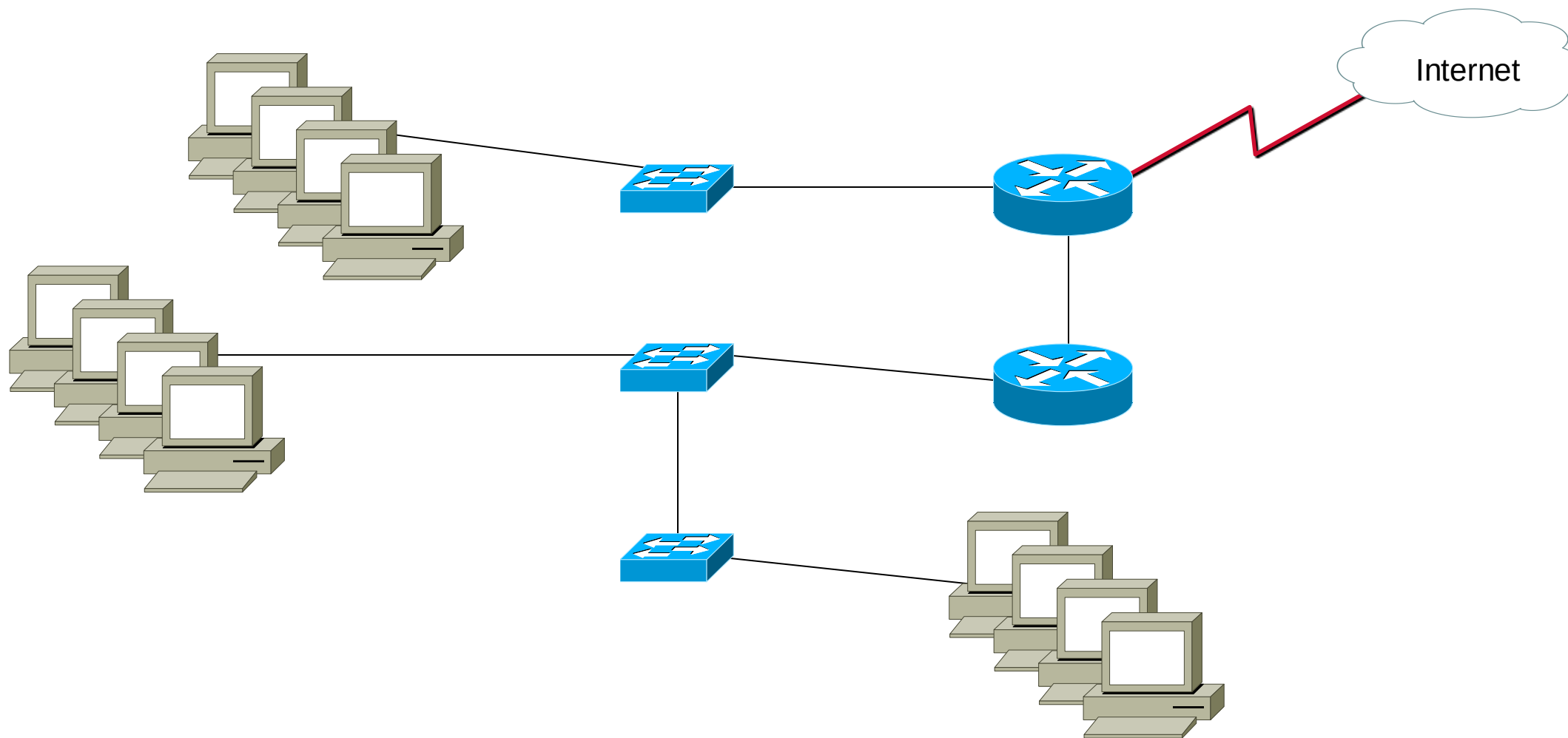
Por que usar DHCP?

- Evita configurar manualmente cada computador, telefone, impressora ou dispositivo IoT
- Reduz erros de configuração e conflitos de endereços
- Permite administrar centralmente os parâmetros da rede
- Facilita alterações de servidor DNS, gateway e outros parâmetros

Uma rede com apenas 2 computadores.



Uma rede corporativa



DHCP - Dynamic Host Configuration Protocol

- DHCP é um serviço utilizado para automatizar as configurações do protocolo TCP/IP nos dispositivos de rede
- O protocolo DHCP está descrito na RFC 2131
- Utiliza o UDP como protocolo de transporte:
 - Servidor DHCP: porta 67
 - Cliente DHCP: porta 68

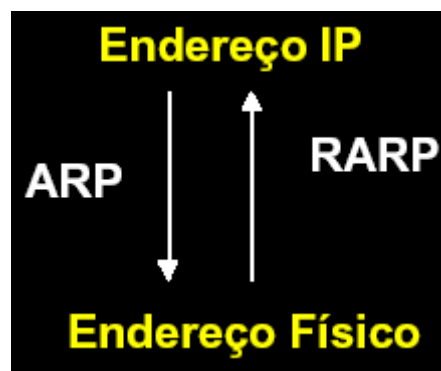
DHCP - Dynamic Host Configuration Protocol

■ Benefícios

- **Automação** do processo de configuração do protocolo TCP/IP nos dispositivos da rede
- Facilidade de alteração de parâmetros tais como **Default Gateway**, **Servidor DNS** e assim por diante, em todos os dispositivos da rede, através de uma simples alteração no servidor DHCP
- **Redução de erros de configuração**, tais como digitação incorreta de uma máscara de sub-rede ou utilização do mesmo número IP em dois dispositivos diferentes, gerando um conflito de endereço IP

RARP – Reverse Address Resolution Protocol

- Permite a obtenção de endereço IP a partir de seu endereço ethernet (MAC – Media Access Control)
- É necessário um servidor RARP
- Utilizado por estações diskless no momento de boot
- Descrito na RFC 903



BOOTP

- Protocolo de configuração de host
- Desenvolvido para ativar a configuração de estações de trabalho sem disco
- Definido na RFC 951
- Como antecessor do DHCP, o BOOTP tem algumas características operacionais semelhantes
 - A estrutura formal que cada um usa para trocar mensagens entre o servidor e os clientes
 - Uso de portas UDP conhecidas para comunicação entre cliente/servidor.
 - A distribuição de endereço IP como parte integral do serviço de configuração

Diferenças entre o BOOTP e o DHCP

BOOTP

- Destinado a configurar estações de trabalho sem disco com recursos limitados de inicialização.

DHCP

- Destinado a configurar computadores de rede realocados com frequência (como portáteis) que possuam recursos de inicialização total e unidades de disco rígido locais.

Diferenças entre o BOOTP e o DHCP

BOOTP

- Oferece suporte a um número limitado de parâmetros de configuração de cliente chamados área específica do fornecedor.

DHCP

- Oferece suporte a um conjunto maior e extensível de parâmetros de configuração de cliente chamados opções.

Diferenças entre o BOOTP e o DHCP

BOOTP

- BOOTP foi amplamente utilizado para configurar estações diskless, inclusive fornecendo informações necessárias para localizar uma imagem de inicialização transferida posteriormente, por exemplo, via TFTP.

DHCP

- Descreve um processo de configuração de inicialização de fase única no qual um cliente DHCP negocia com um servidor DHCP para determinar o endereço IP e obter quaisquer detalhes de configuração inicial necessários para a operação de rede.

Diferenças entre o BOOTP e o DHCP

BOOTP

- Os clientes BOOTP não religam ou renovam a configuração com o servidor BOOTP, exceto quando o sistema é reiniciado.

DHCP

- Os clientes DHCP não exigem que um sistema seja reiniciado para religar ou renovar a configuração com o servidor DHCP
- Em vez disso, os clientes entram automaticamente em um estado de religação em intervalos programados definidos para renovar a alocação de endereço concedida com o servidor DHCP
- Esse processo ocorre em segundo plano e é transparente para o usuário

Termos utilizados no DHCP

- **Servidor DHCP:** É um servidor onde foi instalado e configurado o serviço DHCP
- **Cliente DHCP:** É qualquer dispositivo de rede capaz de obter as configurações do TCP/IP a partir de um servidor DHCP

Termos utilizados no DHCP

■ Escopo

- Faixa configurada para uma sub-rede
- Ex: 10.10.10.100 a 10.10.10.150 - rede 10.10.10.0/255.255.255.0

■ Intervalo de exclusão

- Sequência limitada de endereços IP dentro de um escopo, que não devem ser fornecidos pelo DHCP
- Ex: 10.10.10.120 a 10.10.10.130

■ Pool de endereços

- Conjunto de endereços IP disponíveis, já descontados os endereços das faixas de exclusão
- Ex: 10.10.10.100 a 10.10.10.119 e 10.10.10.131 a 10.10.10.150.

Termos utilizados no DHCP

■ Concessão

- Intervalo de tempo que um cliente DHCP pode usar um endereço IP atribuído
- Ex: 5 dias

■ Reserva

- Atribuição de endereço IP permanente para um determinado dispositivo da rede
- Ex: MAC: ABCD.1234.6789 – IP 10.10.10.101

■ Tipos de opções

- Outros parâmetros de configuração do cliente que podem ser atribuídos pelo servidor DHCP.
- Ex: Gateway, WINS, Servidor DNS

Recursos do DHCP

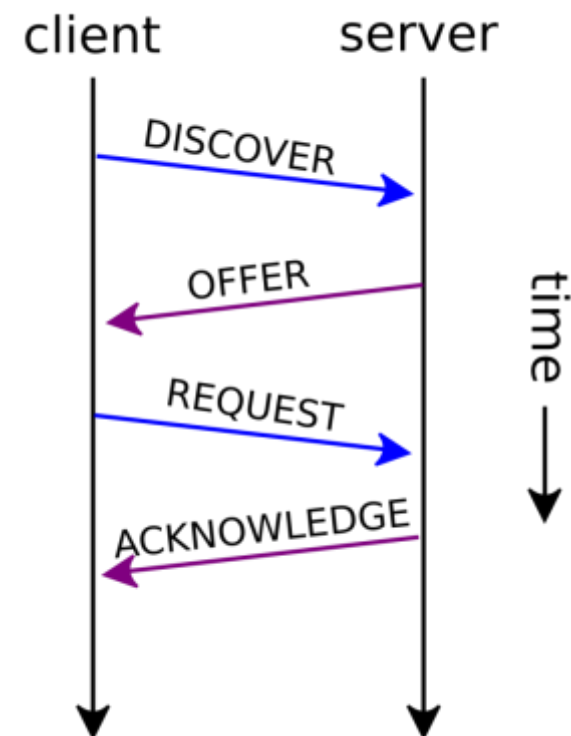
- O DHCP utiliza um modelo cliente/servidor
- O banco de dados do servidor inclui os seguintes itens:
 - Parâmetros de configuração válidos para todos os clientes na rede (número IP do Default Gateway, número IP de um ou mais servidores DNS e assim por diante) – estas configurações podem ser diferentes para cada escopo.
 - Endereços IP válidos mantidos em um pool para serem atribuídos aos clientes além de reservas de endereços IP
 - Duração das concessões oferecidas pelo servidor – a concessão define o período de tempo durante o qual o endereço IP atribuído pode ser utilizado pelo cliente

Atribuição de endereço IP

- Existem três mecanismos que são usados para atribuir um endereço IP ao cliente
 - **Alocação automática** – O DHCP atribui um endereço IP permanente ao cliente
 - **Alocação manual** – O administrador atribui o endereço IP ao cliente. O DHCP informa o endereço ao cliente
 - **Alocação dinâmica** – O DHCP atribui, ou aluga, um endereço IP ao cliente por um período de tempo limitado

Arquitetura Básica

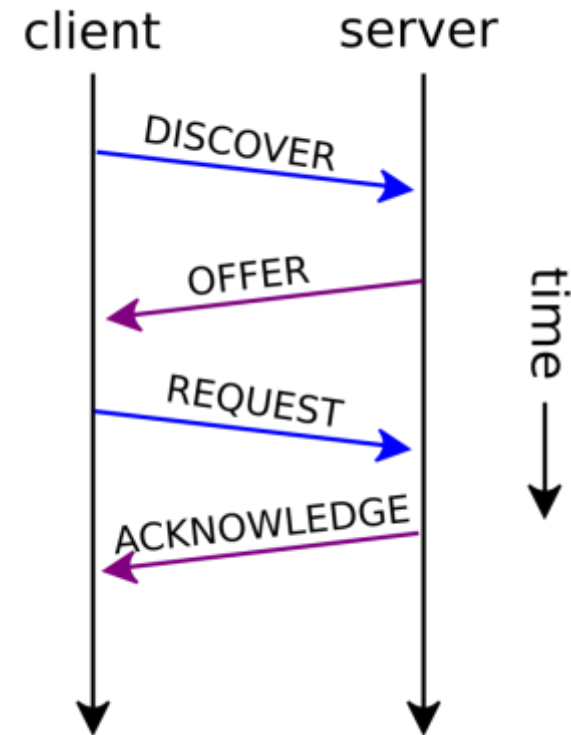
- O cliente solicita uma configuração
- O servidor possui um pool de endereços e opções
- O servidor oferece uma configuração ao cliente
- Em uma LAN, cliente e servidor normalmente comunicam-se localmente
- Entre redes, pode ser usado DHCP relay



DHCPv4 – DORA

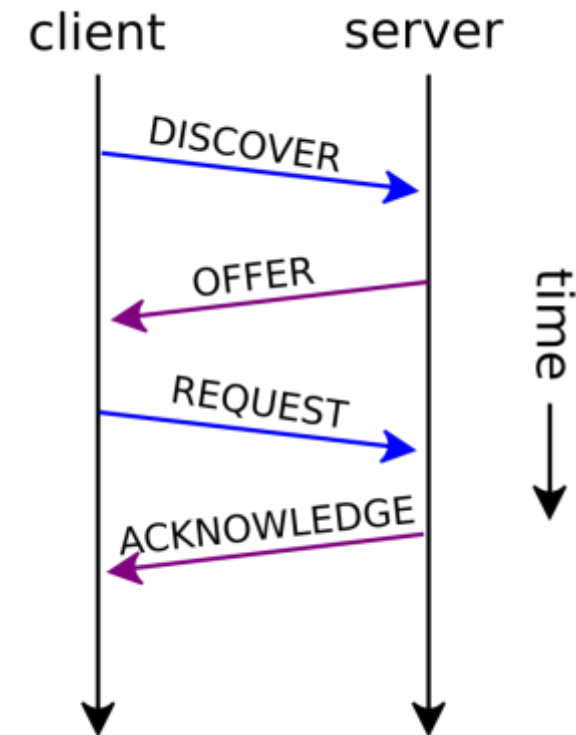
DORA – processo típico de obtenção inicial de uma configuração DHCPv4:

- **DISCOVER**: cliente procura servidores DHCP
- **OFFER**: servidor oferece endereço e parâmetros
- **REQUEST**: cliente solicita a oferta escolhida
- **ACK**: servidor confirma a configuração



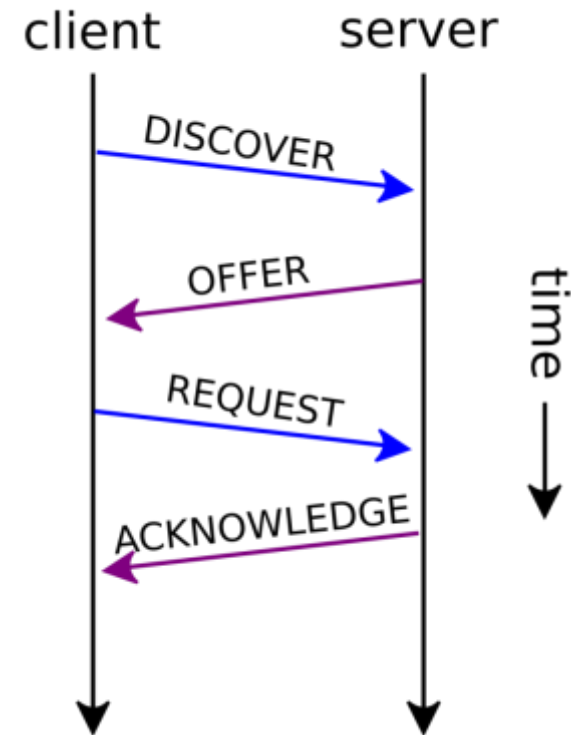
DHCP Discover

- O cliente ainda não possui uma configuração IPv4 válida
- Envia DHCP DISCOVER para localizar servidores DHCP
- Na rede local, a descoberta normalmente utiliza broadcast
- A mensagem contém informações para identificação do cliente
 - Ethernet:
 - destino = ff:ff:ff:ff:ff:ff
 - IPv4:
 - origem = 0.0.0.0
 - destino = 255.255.255.255
 - UDP:
 - origem = 68
 - destino = 67



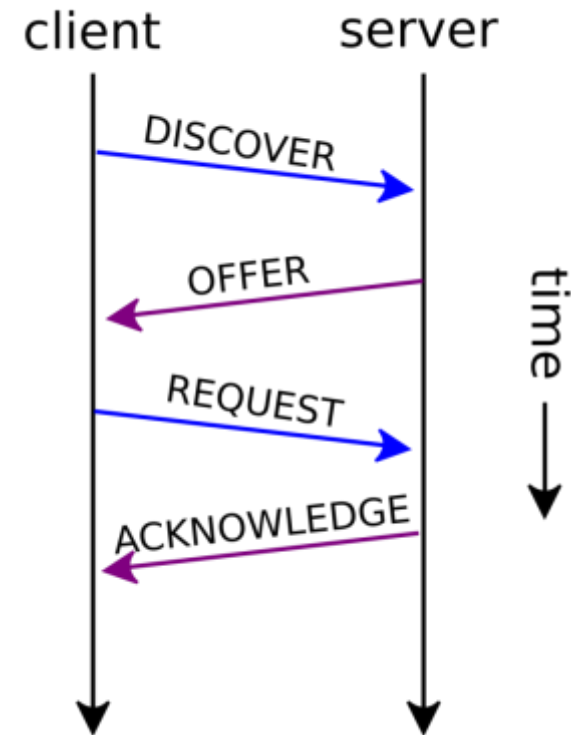
DHCP Offer

- Um servidor responde oferecendo uma configuração
- A oferta normalmente inclui um endereço IPv4
- Pode incluir máscara, gateway, servidores DNS, lease time e outras opções
- Mais de um servidor pode responder



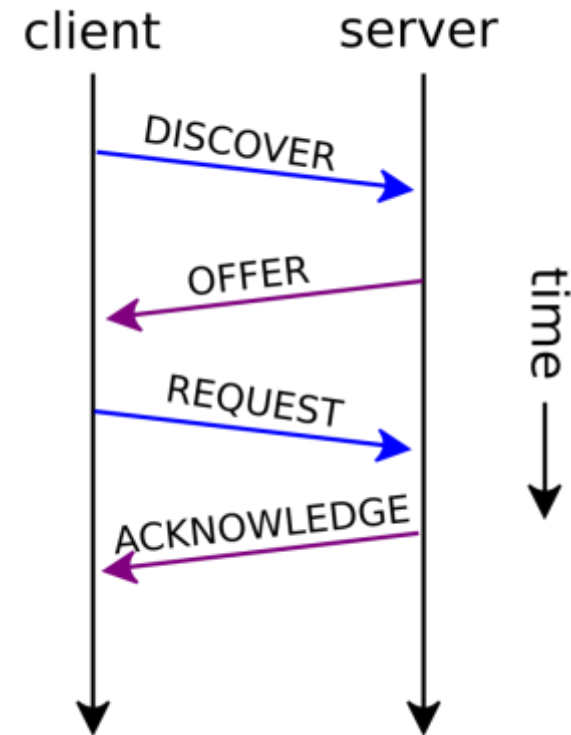
DHCP Request

- O cliente escolhe uma oferta
- Envia DHCP REQUEST indicando qual oferta deseja aceitar
- Na obtenção inicial, o cliente envia DHCP REQUEST indicando qual oferta deseja aceitar e, implicitamente, rejeitando as demais ofertas
- Também é usada posteriormente na renovação da concessão



DHCP Ack

- O servidor confirma a concessão com DHCP ACK
- O cliente configura sua interface com os parâmetros recebidos
- A concessão possui um tempo de validade — lease time
- Após o ACK, o host pode usar a configuração



Campos da Mensagem DHCP

0	7	8	15	16	23	24	31		
OP		HTYPE			HLEN			HOPS	
Transaction ID (xid)									
secs				Flags					
Client IP Address									
"Your" IP Address									
Next Server IP Address									
Relay IP Address									
Client Hardware Address (16 octetos)									
Server Host Name (64 octetos)									
Boot Filename (128 octetos)									
Opções (tamanho variável)									

Campos da mensagem DHCP

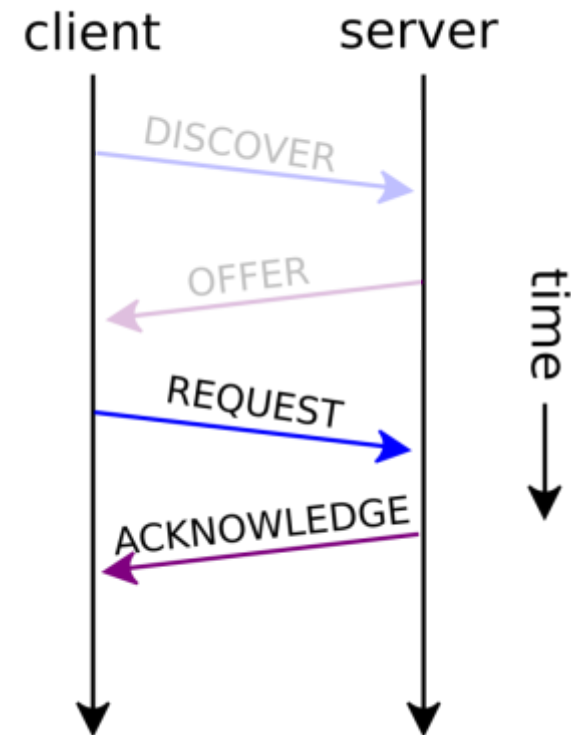
- **OP:** tipo da mensagem
 - 1: BootRequest
 - 2: BootReply
- **HType:** tipo do hardware
 - 1: Ethernet
- **HLEN:** Hardware address length
 - 6: Ethernet
- **HOPS:** geralmente 0 (utilizando para controlar passagem por agentes relays)
- **Transaction ID:** número da transação
- **Secs:** tempo em segundos preenchido pelo cliente desde que começou a transação
- **Flags:** flags de controle de mensagem
- **Client IP Address (ciaddr):** endereço IP atualmente utilizado pelo cliente, quando aplicável
- **Your IP address:** endereço IP do cliente (preenchido pelo servidor)
- **Next server IP address:** endereço do próximo servidor (se houver) para completar o boot
- **Relay IP address:** endereço do relay (se houver)
- **Cliente Hardware address**
- **Server hostname**
- **Boot filename**
- **Opções**

Lease – concessão temporária

- O endereço fornecido normalmente não é permanente
- O servidor define um tempo de concessão
- Antes de expirar, o cliente tenta renovar a concessão
- Se a renovação falhar, existem novas tentativas e eventualmente o endereço deixa de ser utilizável

Renovação do DHCPv4 - Renewal

- O cliente normalmente não executa DORA completo a cada renovação
- Tenta renovar diretamente com o servidor que concedeu o endereço ($T1 = 50\%$)
- Usa DHCP REQUEST e, se aceito, recebe DHCP ACK
- Se o servidor original não responder, o cliente pode tentar outros servidores ($T2 = 87,5\%$)
- O endereço expira ao chegar em 100%



DHCP RELEASE, DECLINE e NAK

- DHCP RELEASE: cliente informa que não precisa mais do endereço
- DHCP DECLINE: cliente informa que o endereço oferecido parece estar em uso
- DHCP NAK: servidor rejeita uma solicitação quando a configuração não é válida para aquela rede
- DHCPINFORM: cliente já possui endereço IP, mas solicita parâmetros de configuração adicionais ao servidor DHCP
- Essas mensagens ajudam a manter o pool consistente

Pool de Endereços

- O servidor mantém um conjunto de endereços que pode distribuir
 - Exemplo: 192.168.10.100–192.168.10.200
- Endereços podem ser excluídos para servidores, impressoras e infraestrutura
- O servidor controla endereços livres, concedidos e reservados

Reserva de Endereço

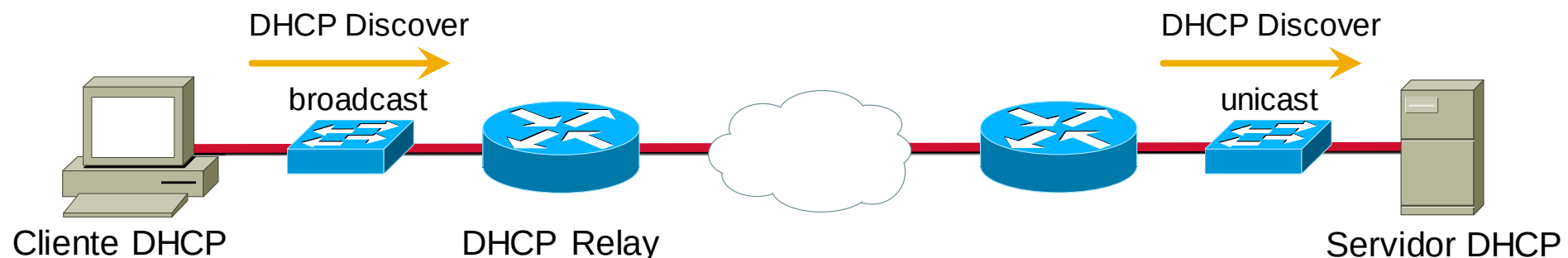
- Uma reserva associa um identificador do cliente a um endereço específico
 - Exemplo: uma impressora sempre recebe 192.168.10.20
- O endereço continua sendo administrado pelo DHCP
- É diferente de configurar manualmente um endereço estático no equipamento

Opções DHCP Importantes

- Endereço IP
- Máscara de sub-rede
- Gateway padrão
- Servidores DNS
- Tempo de concessão
- Outras opções: NTP, PXE/boot e parâmetros específicos do ambiente

DHCP e Roteadores — DHCP Relay

- DHCPv4 utiliza broadcast, que normalmente não atravessa roteadores
- Um DHCP relay encaminha as solicitações para um servidor DHCP em outra rede
- O relay permite centralizar um servidor DHCP para várias sub-redes
- O servidor consegue identificar a rede de origem da solicitação pelo campo GIADDR



Endereço APIPA – DHCP Indisponível

- Um host configurado para DHCP pode ficar sem endereço IPv4 válido
- Alguns sistemas podem utilizar IPv4 link-local/APIPA (169.254.0.0/16)
- Sem gateway ou servidores DNS adequados, a comunicação além da rede local pode falhar
- O comportamento depende do sistema operacional e da configuração

Segurança e Problemas Comuns

- Um servidor DHCP não autorizado pode fornecer gateway ou servidor DNS incorretos
- Mecanismo de switches que permite distinguir portas confiáveis e não confiáveis para ajudar a impedir respostas DHCP de servidores não autorizados
- Múltiplos servidores podem causar configurações inesperadas
- Pools muito pequenos podem ficar sem endereços

Arquitetura TCP/IP

Protocolos de aplicação

- DHCP
- HTTP
- **FTP**
- SMTP
- POP3
- IMAP

FTP

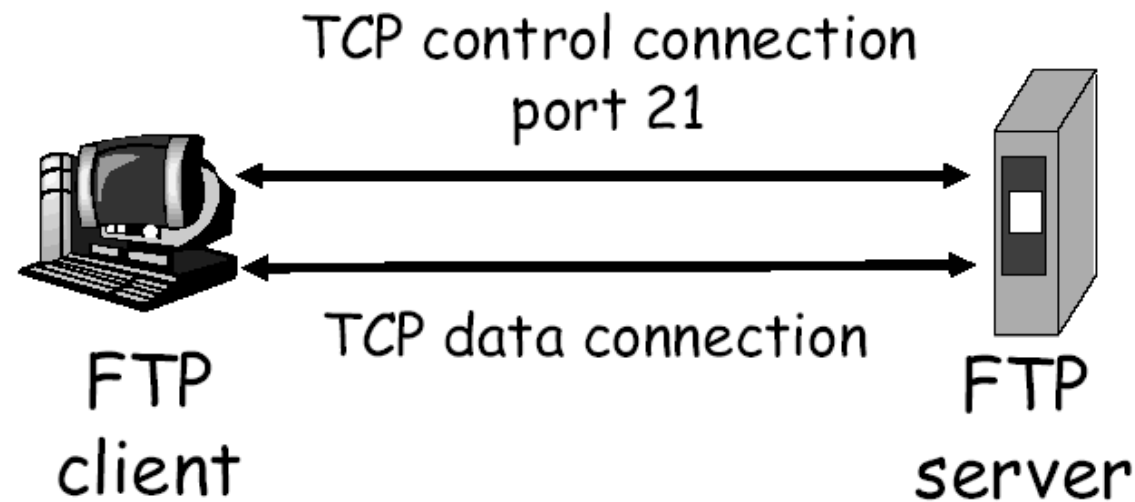
FTP - File Transfer Protocol

- Foi, durante muito tempo, o principal método de transferência de arquivos na Internet
 - Atualmente substituído por HTTPS, SFTP ou SCP
- Modelo cliente/servidor
 - cliente: inicia a sessão e solicita as operações
 - servidor: computador remoto
- Definido pela RFC 959
- Servidor ftp mantém o “estado”: diretório corrente, autenticação anterior

FTP - File Transfer Protocol

- **Tipo de transferência:** o cliente pode selecionar, por exemplo, **ASCII** ou **Image** (binário)
- **Controle de autenticação** – o servidor pode exigir autenticação do cliente, normalmente usando usuário e senha
- **Acesso concorrente** – Vários clientes acessando o servidor simultaneamente
- Transferência em ambos os sentidos

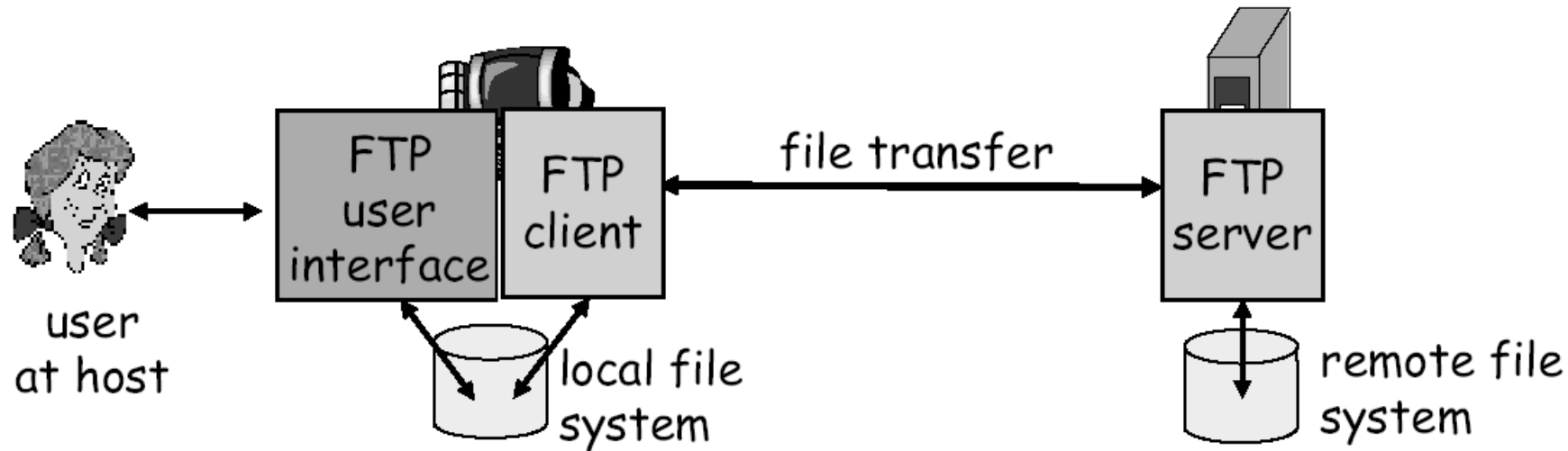
FTP - File Transfer Protocol



O FTP utiliza **dois canais TCP logicamente separados**

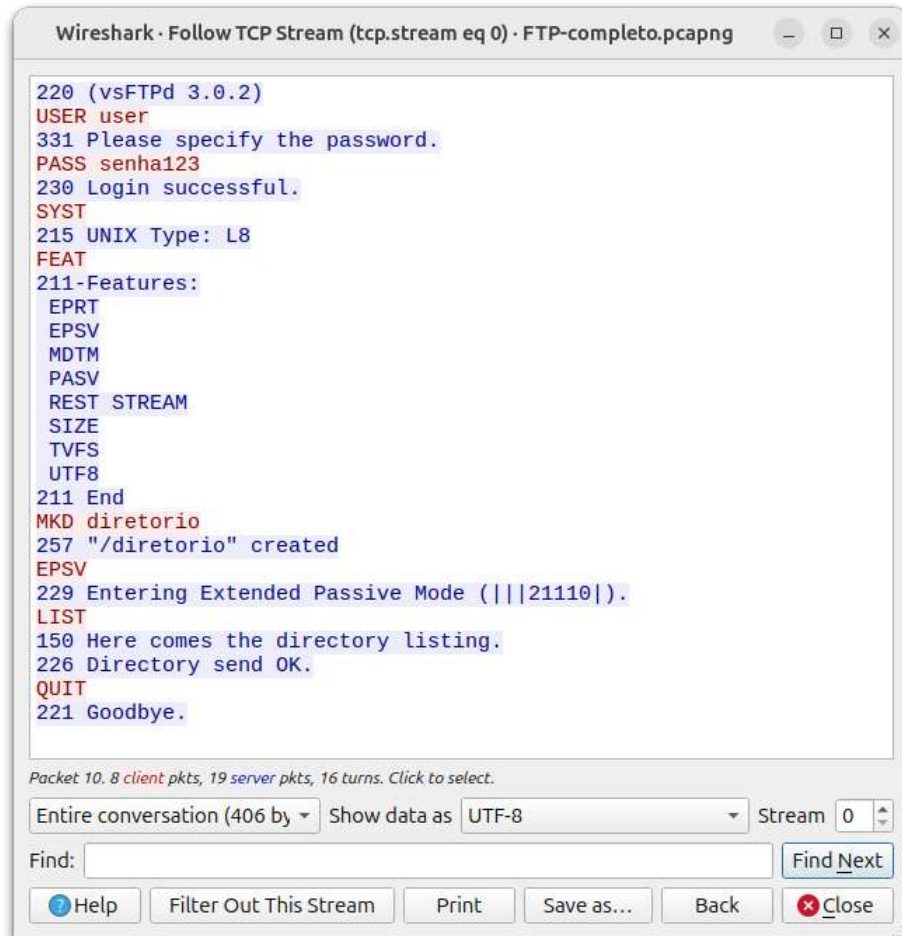
- **Controle:** comandos e respostas FTP
- **Dados:** transferência de dados entre o cliente e o servidor
 - criada para cada operação de transferência de dados, como **LIST**, **RETR** ou **STOR**

FTP - File Transfer Protocol



- Cliente se conecta ao servidor na porta 21(conexão de controle)
- Cliente executa comando solicitando dados do servidor
 - Ex.: LS, GET
- Servidor abre uma conexão com cliente a partir de uma outra porta (conexão de dados)

FTP - File Transfer Protocol



FTP - File Transfer Protocol

■ FTP Ativo

- Cliente se conecta ao servidor na porta 21(conexão de controle)
- Cliente informa ao servidor em que porta este deve se conectar para transferência dos dados.
 - Comando PORT ou EPRT
- Servidor conecta-se a porta informada pelo cliente a partir de sua porta 20, estabelecendo assim a conexão de dados

FTP - File Transfer Protocol

- FTP Ativo

172.18.0.3	172.18.0.2	FTP	93 Request: EPRT 1 172.18.0.3 55391
172.18.0.2	172.18.0.3	FTP	117 Response: 200 EPRT command successful. Consider using EPSV.
172.18.0.3	172.18.0.2	FTP	72 Request: LIST
172.18.0.2	172.18.0.3	TCP	74 20 → 55391 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 SACK_PERM TS
172.18.0.3	172.18.0.2	TCP	74 55391 → 20 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=1460 S
172.18.0.2	172.18.0.3	TCP	66 20 → 55391 [ACK] Seq=1 Ack=1 Win=64512 Len=0 TSval=121183146
172.18.0.2	172.18.0.3	FTP	105 Response: 150 Here comes the directory listing.
172.18.0.2	172.18.0.3	FTP-DATA	133 FTP Data: 67 bytes
172.18.0.3	172.18.0.2	TCP	66 55391 → 20 [ACK] Seq=1 Ack=68 Win=130256 Len=0 TSval=3701186
172.18.0.2	172.18.0.3	TCP	66 20 → 55391 [FIN, ACK] Seq=68 Ack=1 Win=64512 Len=0 TSval=121
172.18.0.3	172.18.0.2	TCP	66 55391 → 20 [FIN, ACK] Seq=1 Ack=69 Win=130256 Len=0 TSval=37
172.18.0.2	172.18.0.3	TCP	66 20 → 55391 [ACK] Seq=69 Ack=2 Win=64512 Len=0 TSval=12118314
172.18.0.2	172.18.0.3	FTP	90 Response: 226 Directory send OK.
172.18.0.3	172.18.0.2	TCP	66 54640 → 21 [ACK] Seq=34 Ack=115 Win=32406 Len=0 TSval=249545

FTP - File Transfer Protocol

■ FTP Passivo

- Cliente se conecta ao servidor na porta 21(conexão de controle)
- Cliente informa ao servidor que deseja fazer transferência de dados em modo passivo.
 - Comando PASV ou EPSV
- Servidor responde informando a porta à qual o cliente deve se conectar
- Cliente se conecta a porta informada e executa comando solicitando dados do servidor
 - Ex.: LS, GET
- Para cada solicitação do cliente na conexão de controle, o servidor informa a porta à qual o cliente deve se conectar para receber os dados

FTP - File Transfer Protocol

- FTP Passivo

172.18.0.3	172.18.0.2	FTP	72 Request: EPSV
172.18.0.2	172.18.0.3	FTP	115 Response: 229 Entering Extended Passive Mode (21107).
172.18.0.3	172.18.0.2	TCP	74 56814 → 21107 [SYN] Seq=0 Win=65535 Len=0 MSS=1460 SACK_PERM
172.18.0.2	172.18.0.3	TCP	74 21107 → 56814 [SYN, ACK] Seq=0 Ack=1 Win=65160 Len=0 MSS=1460
172.18.0.3	172.18.0.2	TCP	66 56814 → 21107 [ACK] Seq=1 Ack=1 Win=129940 Len=0 TSval=11735
172.18.0.3	172.18.0.2	FTP	72 Request: LIST
172.18.0.2	172.18.0.3	FTP	105 Response: 150 Here comes the directory listing.
172.18.0.2	172.18.0.3	FTP-DATA	133 FTP Data: 67 bytes (EPASV) (LIST)
172.18.0.3	172.18.0.2	TCP	66 56814 → 21107 [ACK] Seq=1 Ack=68 Win=129876 Len=0 TSval=11735
172.18.0.2	172.18.0.3	TCP	66 21107 → 56814 [FIN, ACK] Seq=68 Ack=1 Win=65536 Len=0 TSval=11735
172.18.0.3	172.18.0.2	TCP	66 56814 → 21107 [FIN, ACK] Seq=1 Ack=69 Win=129876 Len=0 TSval=11735
172.18.0.2	172.18.0.3	TCP	66 21107 → 56814 [ACK] Seq=69 Ack=2 Win=65536 Len=0 TSval=83800
172.18.0.2	172.18.0.3	FTP	90 Response: 226 Directory send OK.
172.18.0.3	172.18.0.2	TCP	66 53312 → 21 [ACK] Seq=13 Ack=113 Win=32406 Len=0 TSval=214437

FTP - File Transfer Protocol

■ FTP ativo

- servidor inicia a conexão de dados;
- pode ser problemático quando o cliente está atrás de firewall/NAT;
- usa PORT ou, na versão estendida, EPRT.

■ FTP passivo

- cliente inicia a conexão de dados;
- normalmente funciona melhor quando o cliente está atrás de firewall/NAT;
- usa PASV ou, modernamente, EPSV.
- A RFC 2428 introduziu EPRT e EPSV, inclusive para tratar IPv6 e melhorar o comportamento com NATs.

FTP - File Transfer Protocol

- FTP Anônimo
 - Permite o acesso de qualquer usuário a uma área pública no servidor
 - Usa uma conta e senha padrão
 - Conta: anonymous ou ftp
 - Senha: normalmente o e-mail do usuário (configurável)

FTP - File Transfer Protocol

- Transferência segura de arquivos
 - Secure Sockets Layer FTP – Extensão FTP que utiliza o SSL para adicionar criptografia na transferência dos dados
 - FTPS (FTP over TLS) – Extensão do FTP que utiliza TLS para proteger a conexão de controle e, quando configurado, a conexão de dados
 - Secure File Transfer Program (SFTP) – Alternativa ao FTP criado para utilização com o SSH
 - Secure Copy (SCP) – Outra alternativa ao FTP também utilizado com base no SSH.

FTP - File Transfer Protocol

Protocolo	Base	Porta (típica)	É FTP?
FTP	TCP	21	Sim
FTPS	FTP + TLS	21*	Sim
SFTP	SSH	22	Não
SCP	SSH	22	Não

FTP - File Transfer Protocol

■ Alguns comandos FTP

- `binary`
 - Ou simplesmente `bin`.
 - Estabelece como binário o tipo de representação dos arquivos a serem manipulados.
 - Use este comando sempre que for lidar com arquivos de imagem, documentos formatados, executáveis e arquivos compactados.
- `cd diretório_remoto`
 - Muda o diretório de trabalho na máquina remota.
- `cdup`
 - Muda o diretório de trabalho para o diretório "pai" (superior) do diretório atual.
- `lcd [ diretório ]`
 - Muda o diretório de trabalho na máquina local. Se nenhum diretório for especificado, o diretório "home" do usuário é utilizado.
- `get arq_remoto [ arq_local ]`
 - Recupera o arquivo remoto e o armazena na máquina local. Se um nome de arquivo local não for especificado, é dado o mesmo nome do arquivo na máquina remota.
- `mget arquivos_remotos`
 - Faz um `get` para cada arquivo remoto cujo nome esteja na lista de arquivos remotos.
- `put arq_local [ arq_remoto ]`
 - Armazena um arquivo local na máquina remota. Se não for especificado um nome arquivo remoto, ele terá o mesmo nome do arquivo local.

FTP - File Transfer Protocol

■ Alguns comandos FTP

- `mput arquivos_locais`
 - Faz um put para cada arquivo local cujo nome esteja na lista de arquivos locais
- `help [ comando ]`
 - Ou apenas `?`, escreve uma mensagem explicativa sobre o significado do comando.
 - Quando nenhum comando é especificado, apresenta uma lista de comandos.
- `ls [ dir_remoto ] [ arq_local ]`
 - Dá uma listagem do conteúdo de um diretório da máquina remota. Se não for especificado um diretório remoto, é mostrada a listagem do diretório de trabalho usado. Se nenhum arquivo local for especificado, a listagem é mostrada na tela.
- `dir [ dir_remoto ] [ arq_local ]`
 - Lista o conteúdo do diretório da máquina remota, colocando o resultado na máquina local.
 - Se nenhum diretório remoto for especificado, o diretório de trabalho atual na máquina remota será utilizado.
 - Se nenhum arquivo na máquina local for especificado para receber a lista do diretório remoto, o resultado é enviado para o terminal.
- `!dir [ dir_local ] [ arq_local ]`
 - Em alguns sistemas ftp, este comando lista o conteúdo de um diretório da máquina local.
- `pwd`
 - Retorna o nome do diretório atual na máquina remota.
- `quit`
 - Termina uma sessão ftp

FTP - File Transfer Protocol

- 1xx resposta preliminar
- 2xx conclusão positiva
- 3xx informação adicional necessária
- 4xx erro temporário
- 5xx erro permanente
- 220 Service ready
- 331 User name okay, need password
- 230 User logged in
- 150 Opening data connection
- 226 Transfer complete

TFTP - Trivial File Transfer Protocol

- Protocolo que desempenha o mesmo tipo de serviço que o FTP – transferência de arquivos – mas ao contrário do FTP usa o protocolo UDP na camada de transporte.
- Menor utilização de banda e processamento.
- Normalmente utilizado em dispositivos com boot remoto, por oferecer um cliente bem menor que o cliente FTP
- Transferência simples de arquivos sem autenticação

TFTP não é uma versão simplificada do **FTP**; é um protocolo diferente

Arquitetura TCP/IP

Protocolos de aplicação

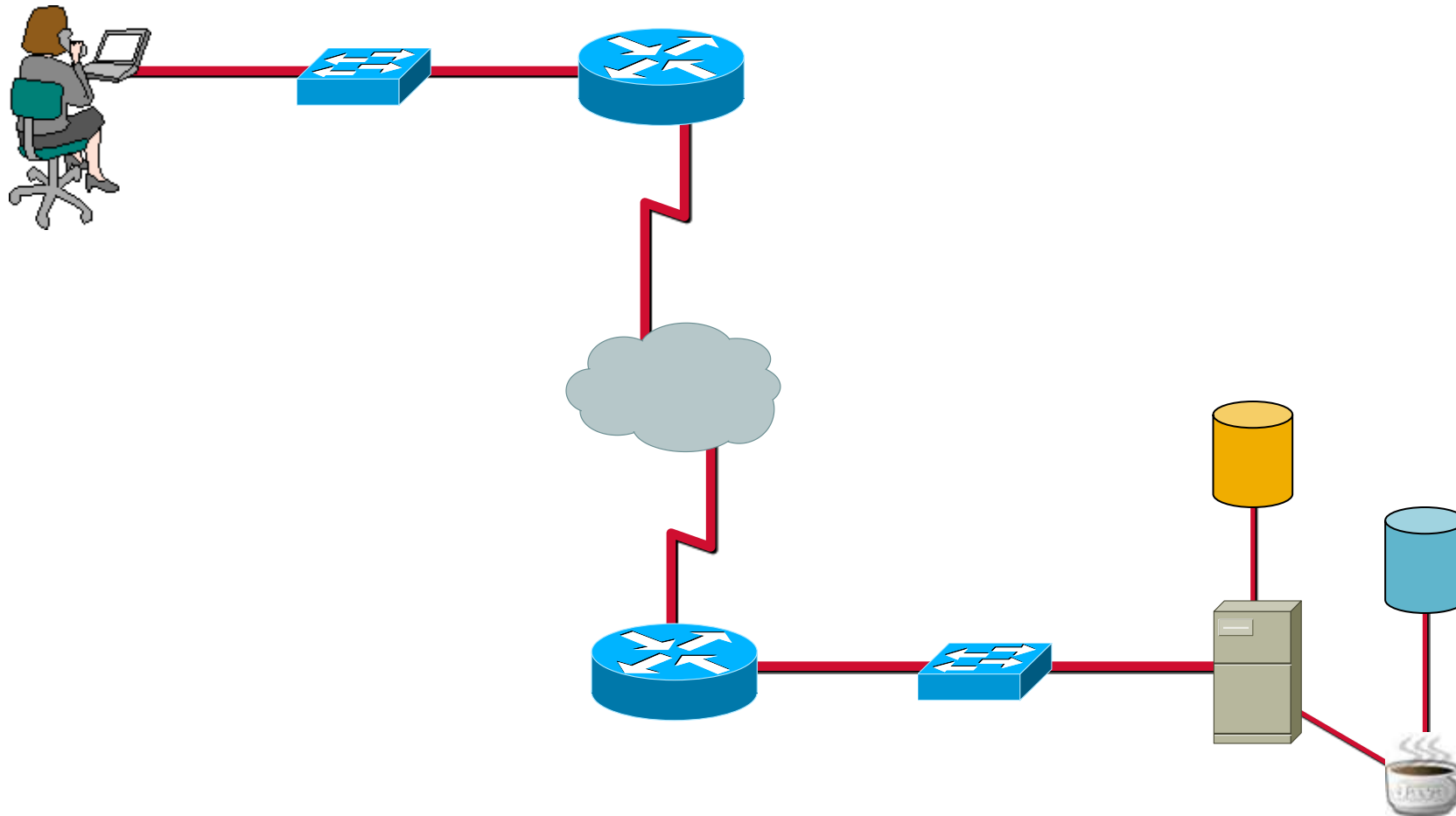
- DHCP
- **HTTP**
- FTP
- SMTP
- POP3
- IMAP

HTTP

Introdução ao HTTP

- Aplicação cliente-servidor
- Protocolo específico para interação cliente-servidor:
 - HTTP (Hyper Text Transfer Protocol)
 - Protocolo para transferir documentos em diversos formatos
- O aplicativo do cliente se encarrega de interpretar os documentos e exibi-los para o usuário

Arquitetura do Modelo Web



Cliente Web

- Para o usuário a web é conjunto de páginas
- Páginas são visualizadas por programas específicos:
 - Browsers
- Browser normalmente possui internamente vários interpretadores (plug-in) permitindo a carga e a execução de documentos de diferentes tipos

Servidor Web

- Cada site web possui um daemon (servidor Web)
- Daemon espera requisições de clientes na porta 80 (TCP)
- Após conexão estabelecida:
 - Cliente envia uma requisição ao servidor
 - Servidor atende requisição e envia resposta
 - Conexão é desfeita
- Interação baseada em dois elementos:
 - URL: Uniform Resource Locator
 - HTTP: HyperText Transfer Protocol

HyperText Markup Language (HTML)

- Linguagem que descreve como documentos devem ser formatos por um cliente web (browser)

HTTP - HyperText Transfer Protocol

- Protocolo Web que atua no nível de aplicação para a transferência de hipertexto operando sobre o protocolo TCP/IP para estabelecer um mecanismo de serviço com estrutura requisição-resposta.

HTTP - HyperText Transfer Protocol

- Permite a transferência de dados de um ponto a outro sem se importar com o tipo do dado e como ele é processado
 - HTTP transfere qualquer tipo de dados e não apenas páginas web
 - Não faz nenhum tipo de processamento
- Por padrão utiliza a porta 80 (TCP)
- Browser é que determina como os dados são processados
- URL (Uniform Resource Locator) é utilizado para endereçar uma página
 - protocolo://nome_máquina.domínio:porta/caminho/recurso
 - <http://www.paiva.com:80/docs/index.html>

HTTP - HyperText Transfer Protocol

- **HTTP** transporta bytes
- **Content-Type** informa como o conteúdo deve ser interpretado

Content-Type: text/html

Content-Type: image/png

Content-Type: application/json

Content-Type: video/mp4

Content-Type: application/pdf

HTTP - HyperText Transfer Protocol

- Versão 1.0 [RFC1945](#)
 - Conexão não persistente (versões 0.9 e 1.0)
 - Uma conexão TCP é feita para cada requisição/resposta
 - Cliente abre a conexão, envia a requisição, o servidor envia a resposta e fecha a conexão
 - Ex: uma página com 5 figuras envolve seis conexões, uma para página e uma para cada figura

HTTP - HyperText Transfer Protocol

- Versão 1.1 [RFC2616](#)
 - Conexão persistente
 - A conexão é fechada pelo servidor em resposta a uma requisição de encerramento feita pelo cliente ou por time-out
 - Uma conexão tem capacidade de atender várias requisições/respostas
 - Pode trabalhar no modo não persistente
 - Principal vantagem é a otimização do processo
 - Uso de Servidores Proxy
 - Novos métodos de requisição

HTTP - HyperText Transfer Protocol

- Versão 2
 - mensagens binárias
 - multiplexação de múltiplas requisições/respostas sobre uma conexão TCP
 - compressão de cabeçalhos
 - priorização de streams
 - redução do problema de *head-of-line blocking* existente no nível HTTP/1.1

HTTP - HyperText Transfer Protocol

- Versão 3
 - **UDP/QUIC**: substitui o TCP pelo **QUIC sobre UDP**, eliminando problemas de *head-of-line blocking* no transporte e permitindo estabelecimento mais rápido da conexão
 - **QUIC**: protocolo de transporte que inclui **TLS 1.3**

QUIC: protocolo de transporte que oferece retransmissão, **ordenação** dos dados, controle de **congestionamento**, controle de **fluxo** e conexão **confiável** sobre **UDP**

HTTP

- Principais características:
 - HTTP/1.0, HTTP/1.1 e HTTP/2 utilizam TCP
 - HTTP/3 utiliza QUIC sobre UDP
 - Requisição/Resposta – Navegador envia uma requisição à qual o servidor HTTP responde
 - Sem estado – O servidor não mantém um histórico das requisições/sessões anteriores
 - Transferência bidirecional – Ambos, servidor e navegador, fazem transferências entre si
 - Ex: Servidor → Navegador – Página Web
 - Navegador → Servidor – “Formulário”

HTTP e estado

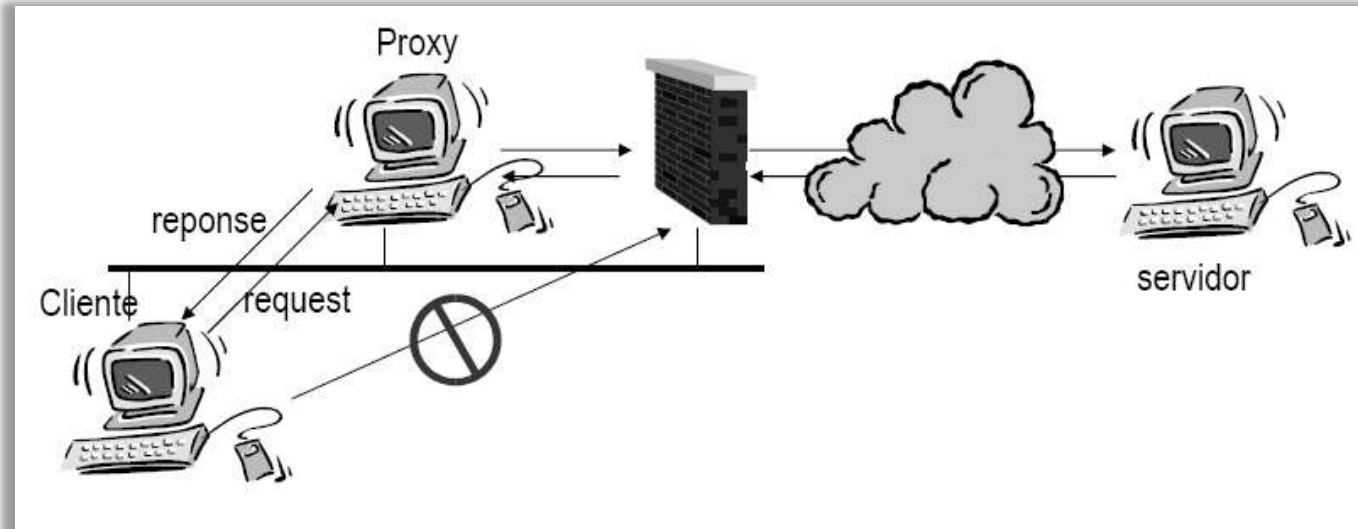
- HTTP não mantém estado por si só
- O servidor pode enviar Set-Cookie
- O cliente envia Cookie
- A aplicação pode usar o cookie para associar requisições a uma sessão

HTTP

- Principais características:
 - Negociação de capacidade
 - Ex: Conjunto de caracteres a ser usado durante a transferência
 - Suporte para uso de cache – Navegador verifica junto ao servidor se o conteúdo da página em cache foi alterado
 - Suporte para intermediários – Permite a utilização de Servidor Proxy entre o navegador e o servidor HTTP

Proxy Web

- Melhora o desempenho realizando caching das páginas acessadas
- Proxy pode atuar como intermediário entre cliente e servidor, permitindo, dependendo da configuração, cache, controle de acesso, filtragem e registro das requisições
- Segurança
 - Máquina cliente não é visível do exterior
 - Possibilidade de monitorar sites, palavras chaves, etc



HTTP

- Os serviços de HTTP incluem:
 - GET: solicita ao servidor o envio de um recurso
Ex: GET http://www.paiva.com/ HTTP/1.1
 Accept: text/html, image/gif, image/jpeg
 User-Agent: Mozilla/3.0
 - HEAD: variante de GET que solicita ao servidor o envio apenas de informações sobre o recurso.
 - PUT: permite que o cliente autorizado armazene ou altere o conteúdo de um recurso mantido pelo servidor.

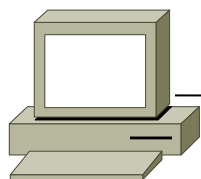
HTTP

- Os serviços de HTTP incluem:
 - POST: permite que o cliente envie mensagens e conteúdo de formulários para servidores que irão manipular a informação de maneira adequada.
 - DELETE: permite que o cliente autorizado remova um recurso mantido pelo servidor
 - OPTIONS: permite descobrir opções/capacidades do servidor
 - PATCH: usado para modificações parciais de um recurso

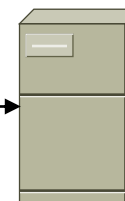
HTTP

■ Ex. do método GET

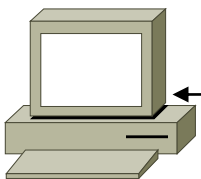
Requisição



```
GET /caminho/arquivo.html HTTP/1.1
Host: www.paiva.com
User-Agent: Firefox/1.0.7
Accept: text/plain,image/png
```



Resposta



```
HTTP/1.1 200 OK
Date: Mon, 12 Dec 2005 04:15:03 GMT
Server: Apache/1.3.27 (Unix) (Red-Hat/Linux)
Content-Type: text/html; charset=UTF-8
Content-Length: 1354
```

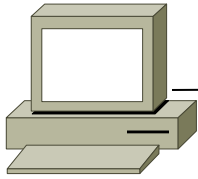
```
<html>
<body>
<h1>Título da página</h1>
(mais conteúdo do arquivo) . . .
</body>
</html>
```



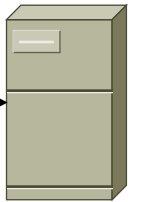
HTTP

■ Ex. do método GET

Requisição



```
GET /caminho/arquivo.html HTTP/1.1  
Host: www.paiva.com  
User-Agent: Firefox/1.0.7  
Accept: text/plain,image/png
```

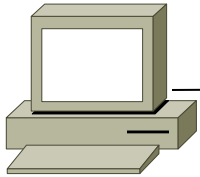


- 1ª linha Get = método
 /caminho/arquivo.html = URL
 HTTP1.1 = versão do protocolo
- 2ª linha Host: www.paiva.com = máquina onde o recurso se encontra
- 3ª linha User-Agent: Firefox/1.0.7 = Informa qual o Browser
- 4ª linha Accept: text/plain,image/png = Tipos de recursos que está apto a receber

HTTP

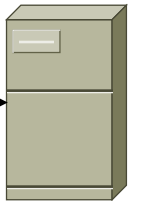
■ Ex. do método GET

Resposta



```
HTTP/1.1 200 OK
Date: Mon, 12 Dec 2005 04:15:03 GMT
Server: Apache/1.3.27 (Unix) (Hed-Hat/Linux)
Content-Type: text/html; charset=UTF-8
Content-Length: 1354

<html>
...
```



- 1ª linha HTTP/1.1 → versão do protocolo
200 OK → código de status
- 2ª linha Date: Mon, 12 Dec 2005 04:15:03 GMT → Data e horário
- 3ª linha Content-Type: text/html; charset=UTF-8 → Tipo de recursos que está enviando
- 4ª linha Content-Length: 1354 → Tamanho do conteúdo
- 5ª linha Linha em branco

Corpo ou conteúdo do recurso solicitado

HTTP – Códigos de Status

- 1xx: Informational (Informativo) – utilizado para enviar informações para o cliente de que sua requisição foi recebida e está sendo processada
 - 200 OK
 - 201 Created
 - 204 No Content
- 2xx: Success (Sucesso) – indica que a requisição do cliente foi bem sucedida
 - 301 Moved Permanently
 - 302 Found
 - 304 Not Modified
- 3xx: Redirection (Redirecionamento) – informa a ação adicional que deve ser tomada para completar a requisição
 - 400 Bad Request
 - 401 Unauthorized
 - 403 Forbidden
 - 404 Not Found
- 4xx: Client Error (Erro no cliente) – avisa que o cliente fez uma requisição que não pode ser atendida
 - 500 Internal Server Error
 - 502 Bad Gateway
 - 503 Service Unavailable
- 5xx: Server Error (Erro no servidor) – ocorreu um erro no servidor ao cumprir uma requisição válida

Exemplo

```
$ curl -v http://example.com
* Host example.com:80 was resolved.
* IPv6: 2606:4700:10::ac42:93f3, 2606:4700:10::6814:179a
* IPv4: 104.20.23.154, 172.66.147.243
* Trying [2606:4700:10::ac42:93f3]:80...
* Connected to example.com (2606:4700:10::ac42:93f3) port 80
> GET / HTTP/1.1
> Host: example.com
> User-Agent: curl/8.5.0
> Accept: */*
>
< HTTP/1.1 200 OK
< Date: Mon, 21 Sep 2026 01:19:55 GMT
< Content-Type: text/html
< Transfer-Encoding: chunked
< Connection: keep-alive
< Server: cloudflare
< Last-Modified: Tue, 15 Sep 2026 23:41:26 GMT
< Allow: GET, HEAD
< Accept-Ranges: bytes
< Age: 13689
< cf-cache-status: HIT
< CF-RAY: a3e53e95ea28e2b8-GIG
<
<!doctype html><html lang="en"><head><title>Example Domain</title><link rel="icon" href="data:,"><meta name="viewport" content="width=device-width,
initial-scale=1"><style>body{background:#eee;width:60vw;margin:15vh auto;font-family:system-ui,sans-serif}h1{font-
size:1.5em}div{opacity:0.8}a:link,a:visited{color:#348}</style></head><body><div><h1>Example Domain</h1><p>This domain is for use in documentation
examples without needing permission. Avoid use in operations.</p><p><a href="https://iana.org/domains/example">Learn more</a></p></div></body></html>
* Connection #0 to host example.com left intact
```

Virtual Hosts

- Virtual Host é a capacidade de hospedar mais do que um web site numa única máquina
- A implementação pode ser feita utilizando um ou vários endereços IP
 - IP-based (vários IPs)
 - Name-based (apenas 1 IP)
- Transparente para o cliente
- Cada domínio virtual trabalha de modo separado, com seus próprios arquivos HTML e sua própria configuração.

127.0.0.53

www.empresa.com (site A)

www.loja.com (site B)

Virtual Hosts

- Name-based
 - Cabeçalho `host` obrigatório a partir da versão 1.1
 - Permite compartilhar o mesmo IP e porta entre os diversos Virtual Hosts
 - Maior economia e flexibilidade na utilização de endereços IP

Virtual Hosts

- IP-based
 - Normalmente é utilizado quando há necessidade de um IP dedicado para cada Virtual Host
 - Alguns SO permitem a atribuição de mais de 1 endereço IP para mesma placa de rede

Secure Socket Layer (SSL)

- Desenvolvido originalmente pela Netscape.
- Versão 3.0 foi publicada como draft da Internet
 - Base para a definição do Transport Layer Security (TLS)
 - TLS é compatível com SSL versão 3.0
- O objetivo é oferecer um canal seguro para comunicação entre cliente e servidor
- SSL está obsoleto (RFC 7568) – atualmente HTTPS usa TLS

Transport Layer Security (TLS)

- Autenticação: verificação de identidade
- Confidencialidade: criptografia de dados
- Integridade: detecção de alteração de dados durante transporte
- RFCs
 - 8446 (TLS 1.3)
 - 5246 (TLS 1.2)
 - 4346 (TLS 1.1)
 - 2246 (TLS 1.0)

HTTPS

- O HTTPS combina TLS e HTTP
- HTTPS (HTTP sobre TLS) e HTTP utilizam portas diferentes (443 e 80, respectivamente)

Arquitetura TCP/IP

Protocolos de aplicação

- DHCP
- HTTP
- FTP
- SMTP
- POP3
- IMAP

Correio Eletrônico

- POP3
- SMTP
- IMAP

Correio eletrônico

- Permite a troca de mensagens não interativas entre usuários
- Formato do endereço de correio eletrônico (e-mail address)
 - user@domain
 - user – nome da caixa postal/usuário
 - domain – domínio responsável pelo endereço
- Alias
 - Um apelido (alias) é um endereço de correio alternativo para um usuário real
 - É possível mapear vários endereços para uma única pessoa, ou mapear um único endereço para várias pessoas.

Correio eletrônico

- O registro **MX** do DNS associa um domínio a um ou mais nomes de servidores de correio
- Uma consulta **A/AAAA** deve ser feita para obter o endereço IP do servidor

```
$ nslookup -type=mx puc-rio.br
Server:          127.0.0.53
Address:         127.0.0.53#53
```

```
Non-authoritative answer:
puc-rio.br      mail exchanger = 0
correio.rdc.puc-rio.br.
```

```
Authoritative answers can be found from:
```

Correio eletrônico

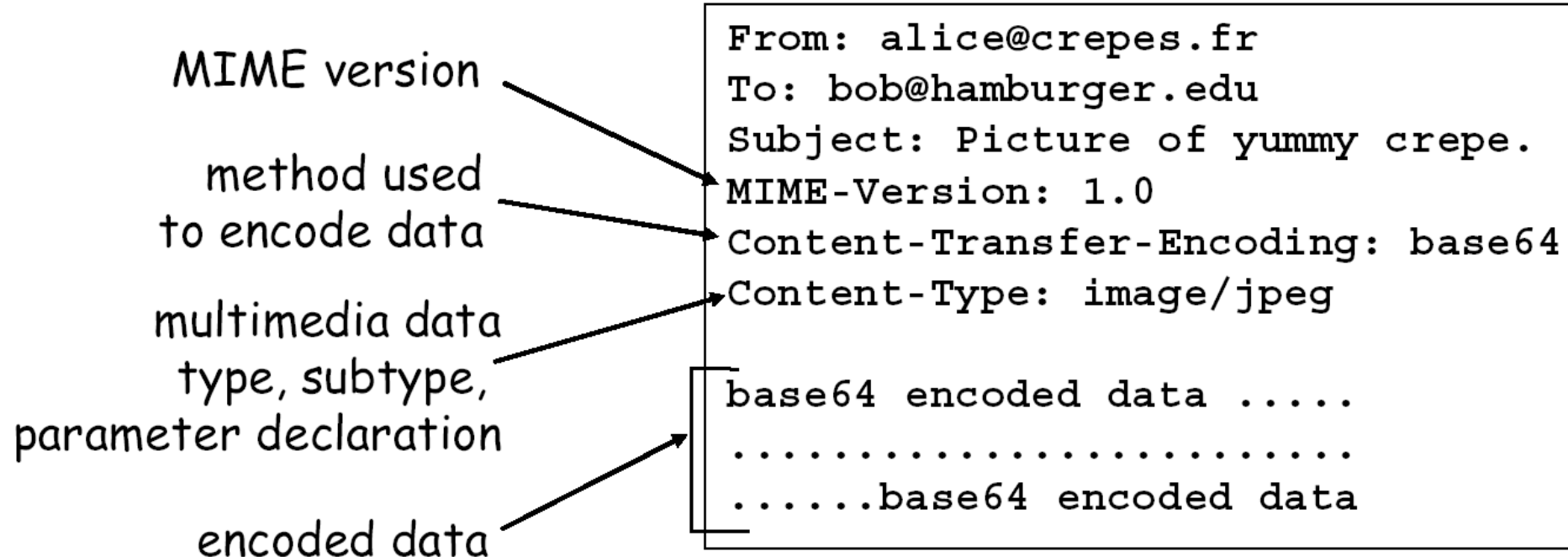
- Tem seu funcionamento baseado no paradigma “**store-and-forward**”, utilizando a técnica de **spooling**
 - **Store-and-forward**: é o princípio segundo o qual um servidor recebe/armazena uma mensagem antes de encaminhá-la ao próximo servidor.
 - **Spool**: é a área/fila de armazenamento utilizada para mensagens que aguardam processamento ou entrega
- A **RFC 5322** especifica que as mensagens são representadas em formato texto e divididas em duas partes - Cabeçalho e Corpo (Header – Body)

Formato da mensagem

- Cabeçalho (header) - é estruturado em campos que contém o remetente, destinatário e outras informações sobre a mensagem
 - Date:
 - Subject:
 - From:
 - To:
 - Cc:
- Uma linha em branco separando o cabeçalho do corpo da mensagem
- Corpo (body) - contém o texto da mensagem

Formato da mensagem: extensões

- Padrões como o MIME (RFC 2045 e 2046) foram desenvolvidos para codificar arquivos binários para transferência através do protocolo SMTP



Formato da mensagem: extensões

- Tipos MIME - Tipo/subtipo
 - Text
 - plain, html
 - Image
 - jpeg, gif
 - Audio
 - basic (8-bit mu-law), 32kadpcm
 - Video
 - mpeg, quicktime
 - Application - dados que têm de ser processados por uma aplicação
 - msword, msexcel

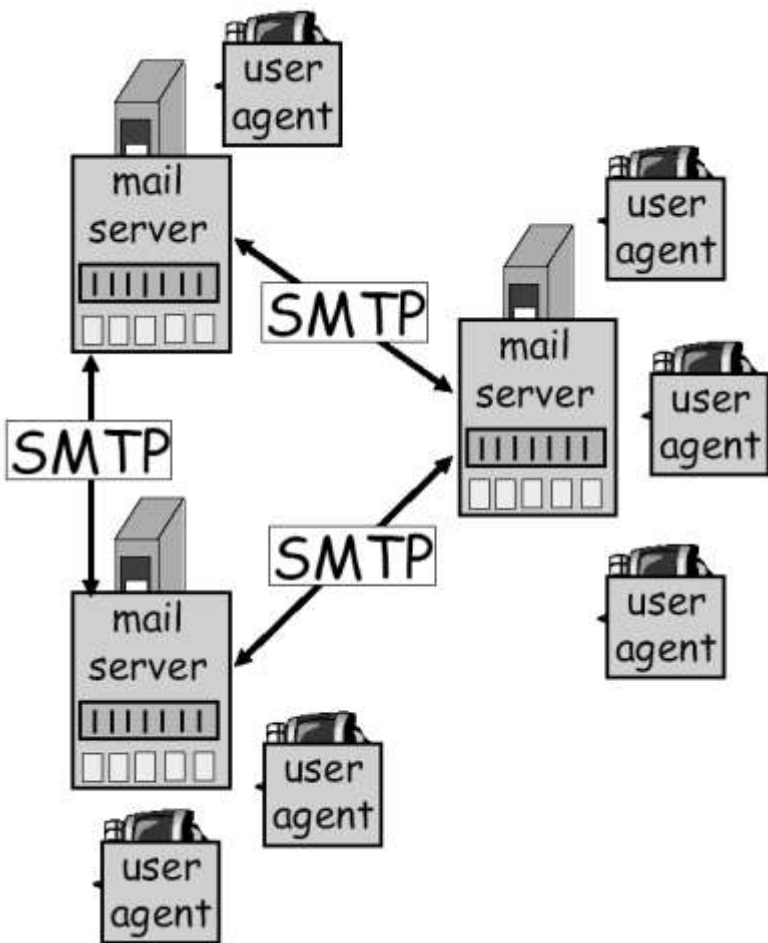
Componentes de um sistema de correio eletrônico

- MUA (Mail User Agent) – Aplicativo cliente de e-mail. O MUA entra em contato com o servidor para que este envie a mensagem ao seu destinatário
 - Ex.: Outlook, Eudora, Thunderbird
- MDA - Mail delivery agent – Aplicativo responsável pela transferência das mensagens recebidas pelo MTA para a mailbox do usuário
- MTA (Mail Transfer Agent) – Aplicativo servidor, responsável pela transferências de mensagens
 - Ex.: PostFix, SendMail, Exchange Server, Merak

Componentes de um sistema de correio eletrônico

- Protocolos
 - SMTP - Simple Mail Transfer Protocol
 - POP3 - Post Office Protocol
 - IMAP4 - Internet Message Access Protocol - versão 4

Componentes de um sistema de correio eletrônico



- SMTP - Simple Mail Transfer Protocol é o protocolo padrão para envio de e-mail e transferência entre servidores
- Modelo cliente/servidor. Usa o protocolo de transporte TCP (porta 25) para transferência das mensagens entre servidores, 587 para submissão autenticada e 465 para submissão com TLS implícito
- Recebe as mensagens de clientes e de outros servidores de correio
- Envia as mensagens para a caixa postal de destino
- Três fases na comunicação entre cliente e servidor
 - apresentação ("handshaking")
 - transferência de mensagens
 - fechamento
- Definido na RFC 5321
- Alguns comandos:
 - Hello, mail from:, rcpt to:, data, quit, vrfy, expn, noop, ser

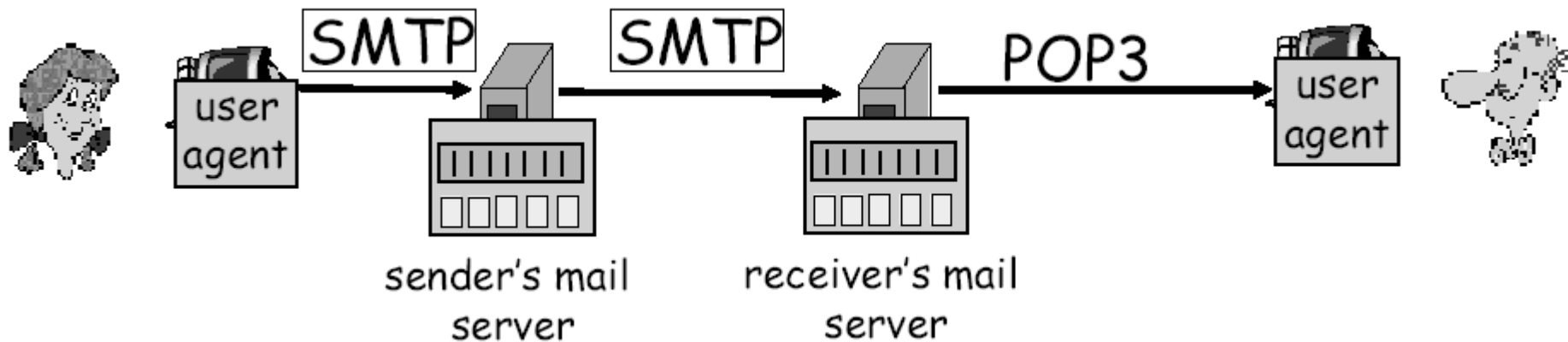
Componentes de um sistema de correio eletrônico

- Exemplo de uma sessão SMTP

```
S: 220 hamburger.edu
C: HELO crepes.fr
S: 250 Hello crepes.fr, pleased to meet you
C: MAIL FROM: <alice@crepes.fr>
S: 250 alice@crepes.fr... Sender ok
C: RCPT TO: <bob@hamburger.edu>
S: 250 bob@hamburger.edu ... Recipient ok
C: DATA
S: 354 Enter mail, end with "." on a line by itself
C: Olá! Testando o SMTP.
C: .
S: 250 Message accepted for delivery
C: QUIT
S: 221 hamburger.edu closing connection
```

Componentes de um sistema de correio eletrônico

- **POP3 – Post Office Protocol** é um protocolo utilizado no acesso remoto a uma caixa de correio eletrônico e download das mensagens (para a máquina local)
- Definido na **RFC 1939**



Componentes de um sistema de correio eletrônico

■ Fases do POP

Fase de autorização

- comandos do cliente:
 - user: username
 - pass: password
- respostas do servidor
 - +OK
 - -ERR

Fase de transação, cliente:

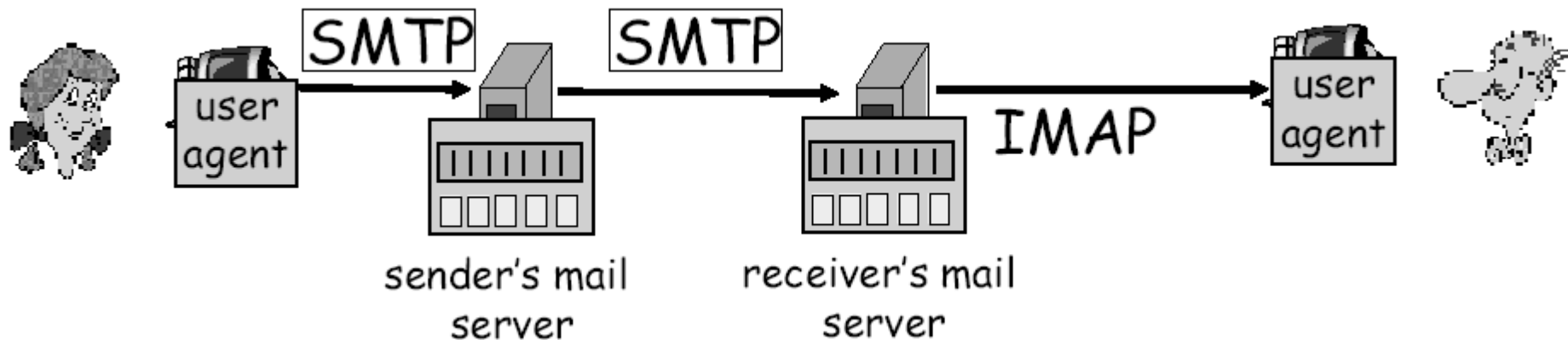
- list: mostra o nº das mens.
- retr: transfere a mens. pelo nº
- dele: apaga
- quit

S: +OK POP3 server ready
C: user alice
S: +OK
C: pass hungry
S: +OK user successfully logged on

C: list
S: 1 498
S: 2 912
S: .
C: retr 1
S: <message 1 contents>
S: .
C: dele 1
C: retr 2
S: <message 1 contents>
S: .
C: dele 2
C: quit
S: +OK POP3 server signing off

Componentes de um sistema de correio eletrônico

- IMAP - Internet Message Access Protocol - versão 4 (IMAP4)
 - também utilizado no acesso remoto a uma caixa de correio, porém permite que um cliente acesse e manipule mensagens de correio eletrônico diretamente em um servidor
- Definido na RFC 9051



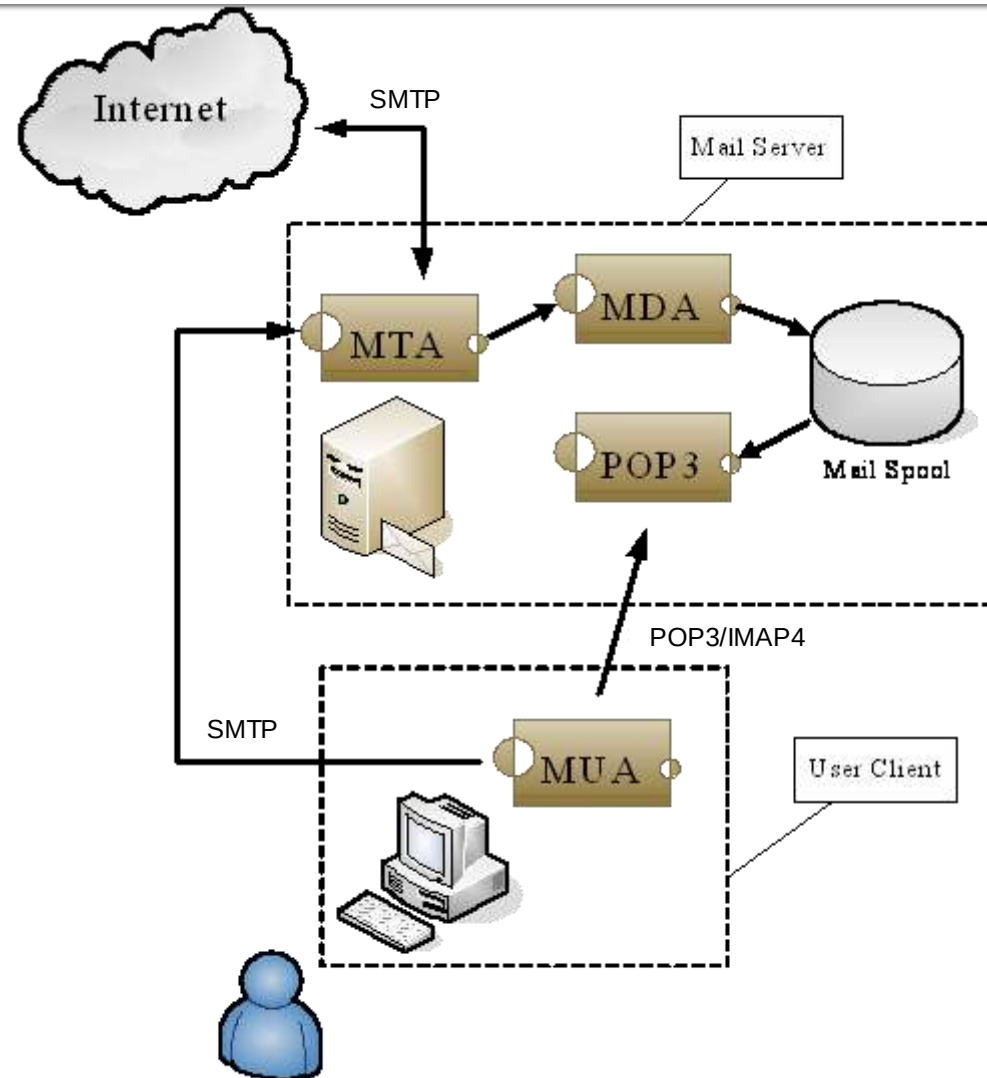
Processo de envio/recebimento de mensagens

1. Usuário acessa o aplicativo cliente (MUA) para redigir a mensagem e enviá-la para o servidor (MTA) – esta transferência é feita através do SMTP
2. O servidor armazena a mensagem em sua área de spool juntamente com as seguintes informações:
 - Emissor, destinatário, máquina destino e horário da requisição de envio
3. O servidor origem inicia, em “background” uma conexão TCP com o servidor destino na porta 25
4. Estabelecida a conexão a mensagem é transferida, armazenada no spool do servidor destino e a cópia no servidor origem é removida

Processo de envio/recebimento de mensagens

5. Quando não é possível estabelecer a conexão ou esta falha, o processo registra o horário da tentativa de envio da mensagem
6. Quando a entrega falha temporariamente, o MTA mantém a mensagem em uma fila e realiza novas tentativas segundo uma política de retry
7. O servidor destino coloca as mensagens recebidas na caixa postal dos usuários
8. Aplicação cliente estabelece uma conexão TCP com servidor onde está a caixa de correio, utilizando o protocolo POP (porta 110) ou IMAP (143), para transferir as mensagens para a máquina local ou, se estiver utilizando o IMAP, manipulá-las no próprio servidor

Processo de envio/recebimento de mensagens



Correio eletrônico – Webmail

Webmail

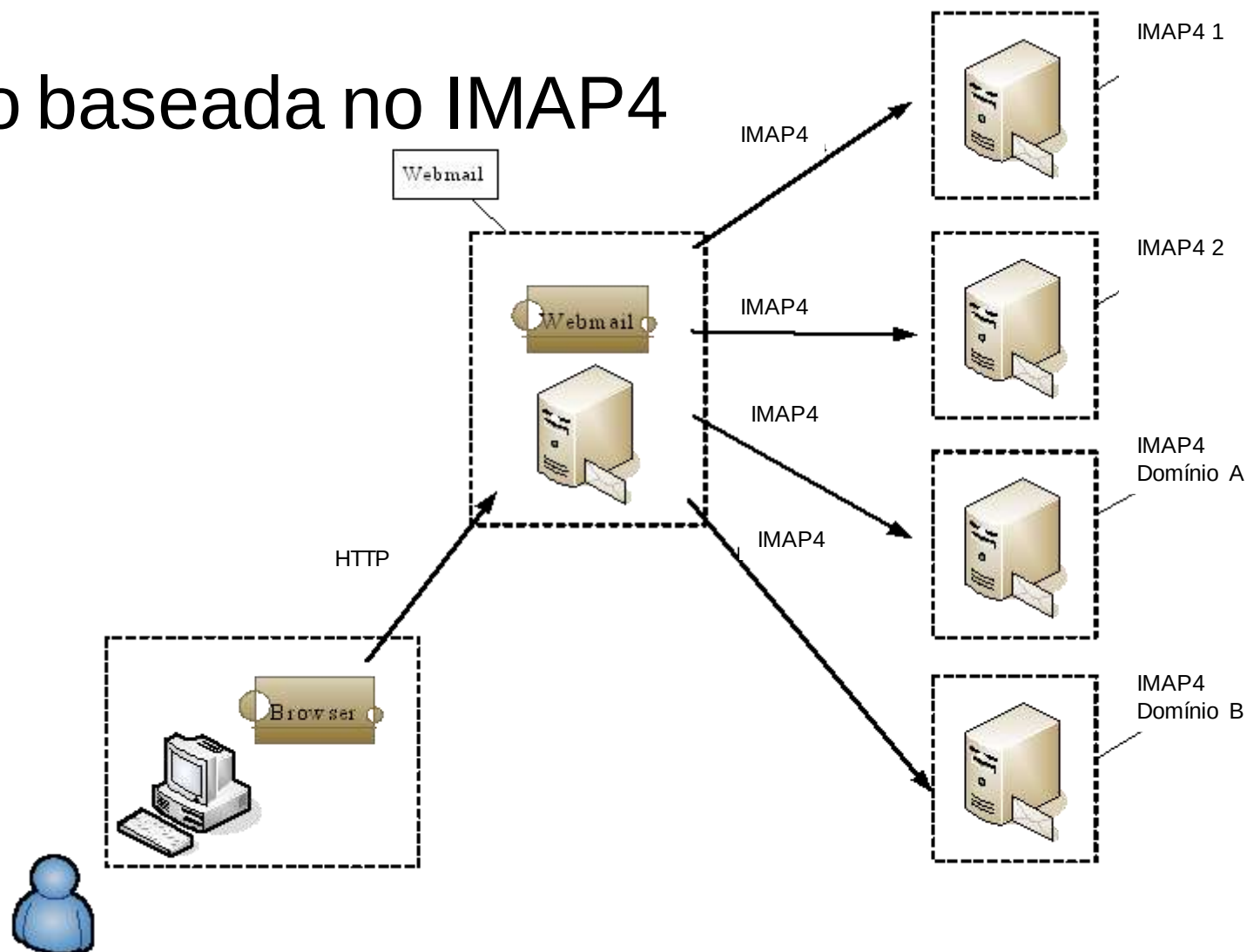
- Um webmail é um sistema que permite aos seus usuários acessar suas mensagens utilizando apenas um navegador Internet, sem que haja a necessidade de instalar e configurar programas clientes de e-mail
- O navegador não fala diretamente IMAP – fala com o backend que fala IMAP com o servidor de correio

Correio eletrônico – Webmail

- Configuração baseada no IMAP4
 - Acesso a mailbox via IMAP4 local ou remotamente.
 - Flexibilidade, escalabilidade
 - A velocidade pode ser limitada pelo desempenho da rede.

Correio eletrônico – Webmail

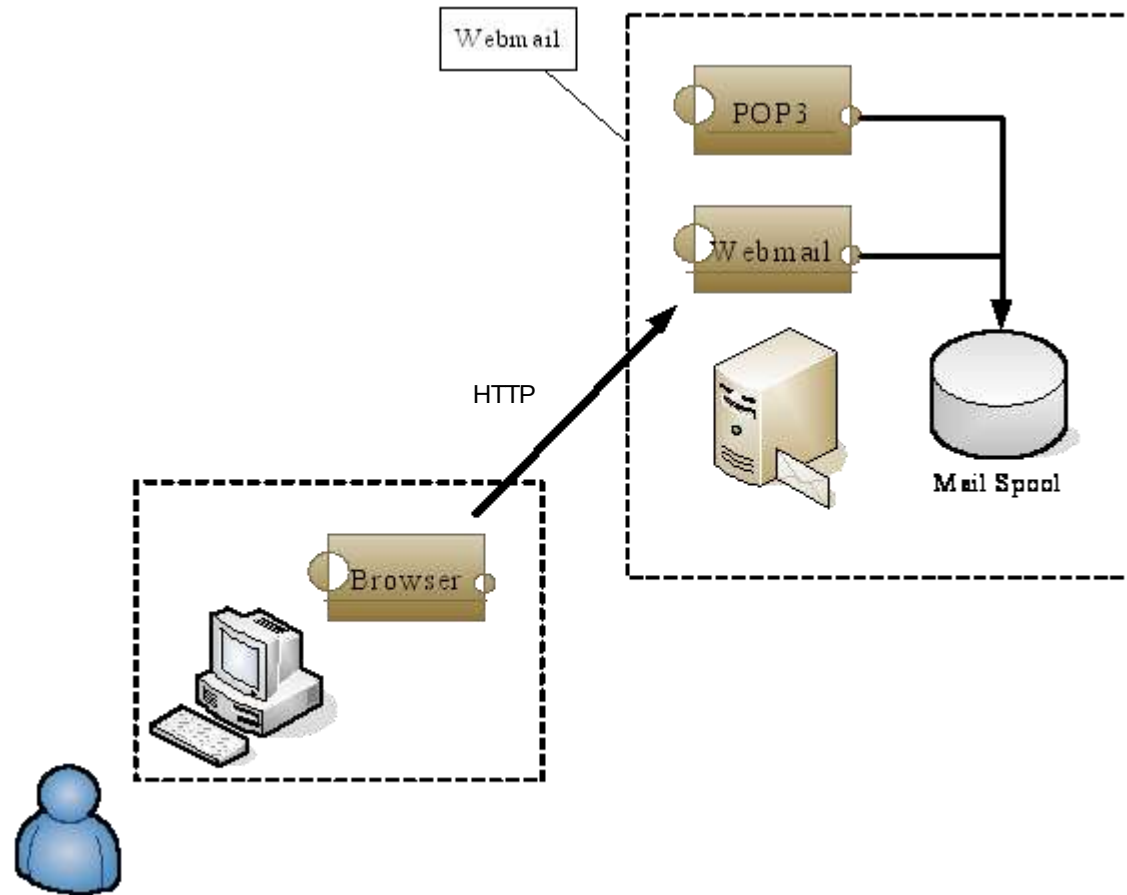
- Configuração baseada no IMAP4



Correio eletrônico – Webmail

- Acesso local direto
 - Acesso local a mailbox
 - Protocol proprietário.
 - Somente utilizado para mailbox local.
 - O uso de diversos servidores requer compartilhamento de disco pela rede local
 - Bastante rápido, se o sistema não estiver sobre carregado.

Correio eletrônico – Webmail



- Acesso local direto

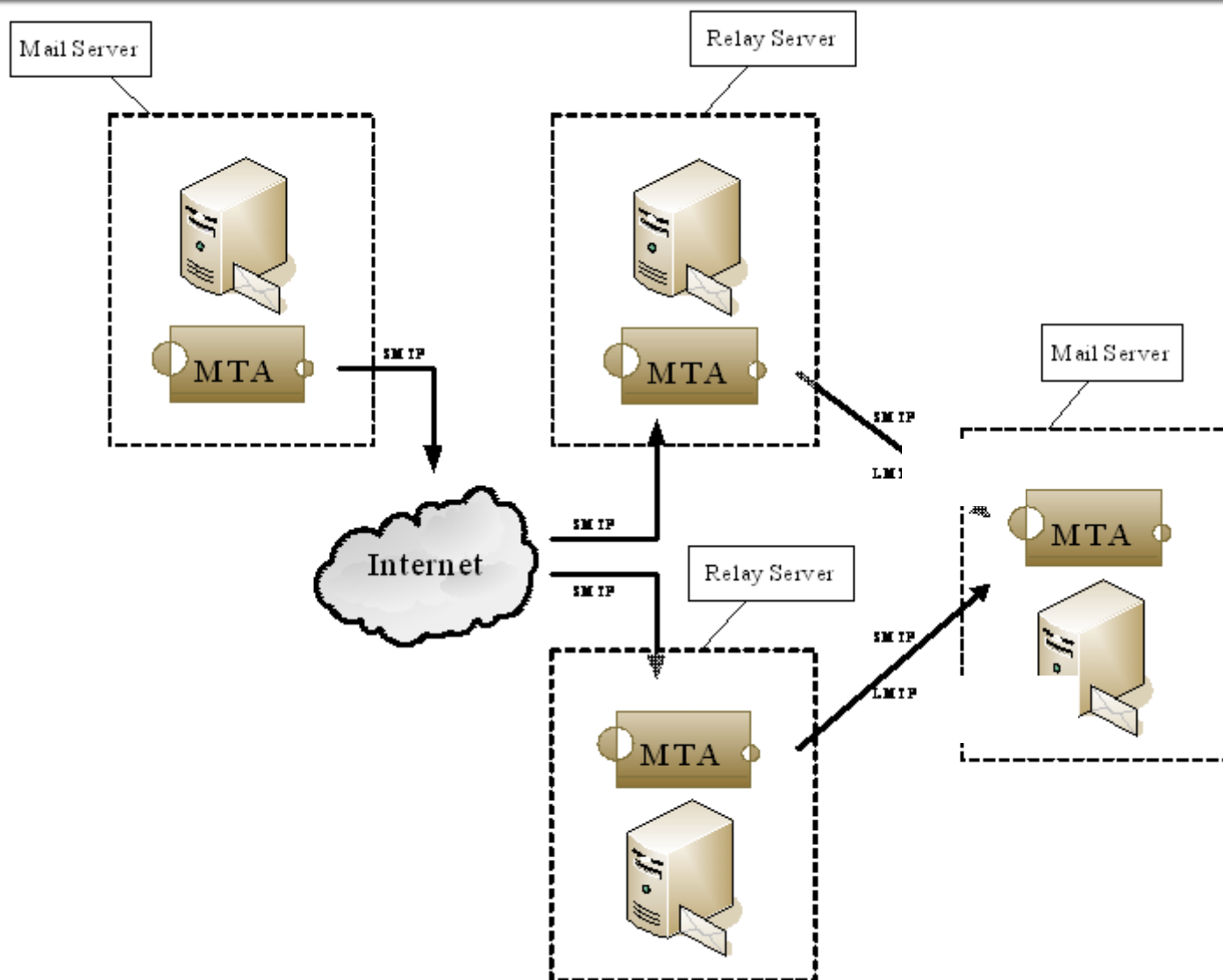
Correio eletrônico – Relay

- Processo de recebimento e envio de mensagens de correio
- Utiliza protocolo SMTP para receber e encaminhar mensagens

Correio eletrônico – Relay

- Balanceamento e compartilhamento de carga
- Contingência
- Proteção e prevenção contra ataques externos aos servidores de e-mail interno
- Possibilidade de uso de diferentes links de acesso à Internet (diferentes ISPs)
- Filtro de Spam e Vírus
- Maneira mais simples de diminuir os gargalos e melhorar a performance

Correio eletrônico – Relay



Correio eletrônico – Relay

- Servidores que permitem que qualquer máquina faça relay neles (relay aberto) correm sérios riscos de serem utilizados para o envio de spam
- Como evitar:
 - Somente permitir que máquinas da rede local façam relay no servidor
- Mecanismos alternativos
 - ~~POP before SMTP~~
 - SMTP Auth (SMTP autenticado)

Correio eletrônico – Relay

- POP before SMTP
 - Primeiramente o usuário acessa o servidor POP (autenticando-se) e baixa suas mensagens
 - Servidor adiciona o IP da máquina a uma lista de máquinas autorizadas a fazerem relay durante um determinado tempo (ex. 5min)
 - Compatível com todos os programas
- SMTP Auth (SMTP autenticado)
 - Servidor solicita senha que permitirá o envio da mensagem
 - Necessita de criptografia mais forte de modo que as senhas não sejam capturadas

DNS

Domain Name System

- Conceitos
- Tradução de nome e IP
- Consultas
- Hierarquia
- Tipos de registros

DNS

Domain Name System – Sistema de Nomes de Domínio

- Pessoas preferem nomes, como `www.example.com`
- A comunicação IP precisa de endereços para encaminhar pacotes
- DNS associa nomes a endereços e outras informações
- É um serviço distribuído e hierárquico
- É o sistema distribuído que traduz nomes legíveis por humanos (como `www.google.com`) em endereços IP numéricos que os computadores usam para se comunicar (e vice-versa)

DNS

Nome de domínio

- www.example.com
- Fácil de memorizar
- Amigável para humanos
- Pode mudar de IP sem afetar o nome

Endereço IP

- 104.20.23.154
- Difícil de lembrar
- Usado pelas máquinas
- Pode ser dinâmico (DHCP)

DNS

- Memorização
 - Humanos não conseguem lembrar dezenas de endereços IP de 32 ou 128 bits
- Balanceamento
 - Um nome pode apontar para vários IPs (round-robin, anycast, CDN)
- Flexibilidade
 - Um domínio pode mudar de servidor (IP) sem que os usuários precisem atualizar nada
- Escalabilidade
 - Sistema hierárquico e distribuído que escala para bilhões de nomes no mundo

Nome de domínio, hostname e FQDN

- Domínio: **puc-rio.br**
- Hostname: **www**
- FQDN (Fully Qualified Domain Name): **www.puc-rio.br**
- O ponto final representa a raiz da hierarquia
- Os níveis são separados por pontos

Hierarquia do DNS – Árvore Invertida

Root Servers

- 13 clusters de servidores raiz (a.root-servers.net ... m.root-servers.net)
- Conhecem os TLDs

TLD Servers

- Top-Level Domains: .com, .org, .br, .edu...
- Responsáveis pelos domínios de segundo nível

Authoritative

- Servidores autoritativos do domínio (ex: ns1.exemplo.com)
- Possuem a zona oficial

Resolver

- DNS recursivo do ISP ou público (8.8.8.8, 1.1.1.1)
- Faz as consultas em nome do cliente

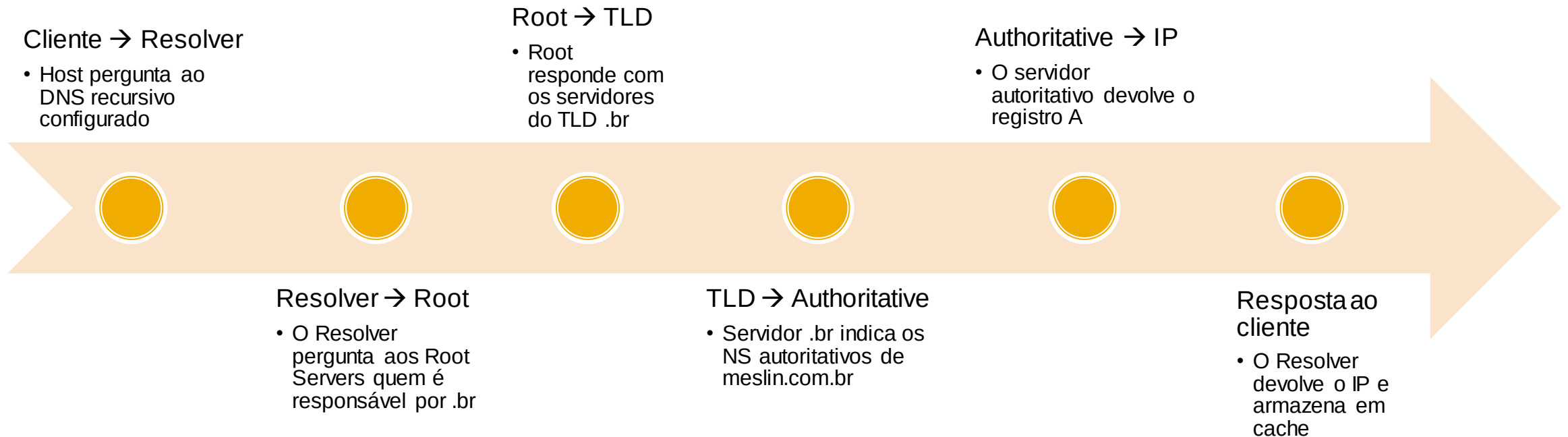
Resolver Recursivo vs Autoritativo

- Resolver recursivo: atende clientes e mantém cache
- Servidor autoritativo: possui os registros oficiais da zona
- O cliente normalmente consulta um resolver configurado no sistema
- O resolver pode consultar outros servidores DNS para obter a resposta

Consulta Recursiva vs Iterativa

- Recursiva
 - “resolva este nome para mim”
- Iterativa
 - “se não souber, indique quem pode saber”
- Cliente → resolver
 - Normalmente recursiva
- Resolver → DNS superiores
 - Normalmente iterativa

Funcionamento – Quem é www.meslin.com.br?



Principais Tipos de Registros

Tipo	Descrição	Exemplo
A	Mapeia nome → IPv4	www → 192.16.2.1
AAAA	Mapeia nome → IPv6	www → 2001:C3D2::1
CNAME	Alias para outro nome	www → git
MX	Servidores de e-mail do domínio	mail.example.com
NS	Servidores autoritativos	ns1.example.com
SOA	Início de autoridade de zona	Parâmetro de zona
PTR	Resolução reversa (IP → nome)	192.16.2.1 → www
TXT	Texto (uma chave)	akjlsdfjaosdfhjakkjfkkeix

Servidor de e-mail

- Exemplo: meslin@puc-rio.br
- MX puc-rio.br: correio.rdc.puc-rio.br
- correio.rdc.puc-rio.br ainda precisa ser resolvido

Cache e TTL

- TTL (Time To Live) define por quanto tempo um registro pode ser armazenado em cache
- Cache
 - Cliente: armazenado pelo SO ou pelo aplicativo (navegador)
 - Resolver: armazena respostas e reutiliza para vários clientes

Segurança do DNS

- DNSSEC
 - Assinaturas criptográficas que garantem a autenticidade e integridade das respostas DNS
- DNS over HTTPS (DoH)
 - Criptografa as consultas DNS dentro de HTTPS, impedindo interceptação e manipulação
- DNS over TLS (DoT)
 - Canal TLS dedicado (porta 853) para proteger o tráfego DNS entre cliente e resolver

Portas e Transporte

- DNS tradicional usa UDP/53 na maioria das consultas
- TCP/53 também é usado em situações como respostas grandes e transferências de zona
- DoT e DoH oferecem transporte DNS criptografado
- No laboratório introdutório, observe principalmente UDP/53

Ferramentas Práticas

dig

Linux e macOS

- **dig example.com**
 - Consulta A padrão
- **dig example.com MX**
 - Registros de e-mail
- **dig @8.8.8.8 example.com**
 - Usar Resolver específico
- **dig -x 8.8.8.8**
 - Resolução reversa

nslookup

Linux, Windows e macOS

- **nslookup example.com**
 - Consulta A padrão
- **nslookup -type=A+AAAA example.com**
 - Consulta A e AAAA
- **nslookup -type=MX example.com**
 - Registros de e-mail
- **nslookup 8.8.8.8**
 - Resolução reversa

Alexandre Meslin
meslin@puc-rio.br

Redes de Comunicação de Dados